MANGALORE INSTITUTE OF TECHNOLOGY & ENGINEERING

(A Unit of Rajalaxmi Education Trust ®, Mangalore)
Autonomous Institute Affiliated to V.T.U, Belagavi, Approved by AICTE, New Delhi
Accredited by NAAC with A+ Grade & ISO 9001:2015 Certified Institution



PROJECT REPORT

23AISE409 PROJECT PHASE – II

"PROBABILISTIC CASCADING FAILURE AND AGENT ASSISTED ROOT CAUSE ANALYSIS FOR MICROSERVICE SYSTEMS"

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

by

ANISH C	**4MT23AI008**
ROYSON SALIS	**4MT23AI046**
DHANUSH K MURTHY	**4MT23AI401**
K BHARATH PARMAR	**4MT24AI402**

Under the Guidance of
Dr. Manjunath Prasad H R
Assistant Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

2026–2027

MANGALORE INSTITUTE OF TECHNOLOGY & ENGINEERING

(A Unit of Rajalaxmi Education Trust ®, Mangalore)
Autonomous Institute Affiliated to V.T.U, Belagavi, Approved by AICTE, New Delhi
Accredited by NAAC with A+ Grade & ISO 9001:2015 Certified Institution

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CERTIFICATE

This is to certify that the Project Phase – II (23AISE409) work entitled "PROBABILISTIC CASCADING FAILURE AND AGENT ASSISTED ROOT CAUSE ANALYSIS FOR MICROSERVICE SYSTEMS" is a bonafide work carried out by ANISH C (4MT23AI008), ROYSON SALIS (4MT23AI046), DHANUSH K MURTHY (4MT23AI401), K BHARATH PARMAR (4MT24AI402) in partial fulfillment for the award of degree of Bachelor of Engineering in Artificial Intelligence and Machine Learning of Mangalore Institute of Technology & Engineering (Autonomous) during the year 2026–2027. It is certified that all corrections and suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The project has been approved as it satisfies the academic requirements in respect of project work prescribed for the Bachelor of Engineering degree.

Dr. Manjunath **Prasad H R** **Project Guide**	Dr. Sunil Kumar S Head of Dept. AI & ML

Dr. Prashanth C M
Principal

DECLARATION

We declare that this Project Phase – II (23AISE409) report titled “Probabilistic Cascading Failure and Agent Assisted Root Cause Analysis for Microservice Systems” submitted in partial fulfillment of the degree of B.E. in Artificial Intelligence and Machine Learning is a record of original work carried out by us under the supervision of Dr. Manjunath Prasad H R, and has not formed the basis for the award of any other degree, in this or any other Institution or University. In keeping with ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Sr. No.	Name	Signature
1	ANISH C (4MT23AI008)	
2	ROYSON SALIS (4MT23AI046)	
3	DHANUSH K MURTHY (4MT23AI401)	
4	K BHARATH PARMAR (4MT24AI402)	

ACKNOWLEDGEMENTS

The successful completion of the Project Phase – II (23AISE409) work titled “Probabilistic Cascading Failure and Agent Assisted Root Cause Analysis for Microservice Systems” would not have been possible without the support of those, whose constant guidance and encouragement helped turn our efforts into success.

We would like to thank our Project Guide Dr. Manjunath Prasad H R, Assistant Professor, Department of Artificial Intelligence and Machine Learning for his/her constant encouragement and support extended throughout.

We would like to express appreciation to Dr. Sunil Kumar S, Head of Department of Artificial Intelligence and Machine Learning for his/her support and guidance.

We would like to thank our Principal Dr. Prashanth C M, for encouraging us and giving us an opportunity to accomplish the project.

We also thank our management who helped us directly or indirectly in the completion of Phase – II of the project.

Our special thanks to faculty members and others for their constant help and support.

Above all, we extend our sincere gratitude to our parents and friends for their constant encouragement with moral support.

ANISH C (4MT23AI008)

ROYSON SALIS (4MT23AI046)

DHANUSH K MURTHY (4MT23AI401)

K BHARATH PARMAR (4MT24AI402)

ABSTRACT

A failing microservice drags its callers down, and within a minute operators face dozens of alarms with no sign of which service started it. This project builds an unsupervised diagnosis platform that watches Docker containers from outside, over SSH, and needs no instrumentation, labels or GPU. It recovers the call graph from socket tables, models normal behaviour while removing the workload shared by all services, ranks the root cause with a random walk, and learns online how failures propagate. A hybrid of this model and robust rules alarms on container facts and statistical deviation. Evaluated on 62 blind faults in two real systems, 84 live faults and 178 synthetic faults, the hybrid names the true cause first for 71% of real faults against 63% for rules alone (not statistically significant), keeps false alarms on workload changes at 0%, and costs 0.37 ms per service.

TABLE OF CONTENTS

Sr. NO.	CONTENTS	PAGE NO.
1.	INTRODUCTION	13
2.	LITERATURE REVIEW	18
3.	PROPOSED SYSTEM AND SCOPE	22
4.	METHODOLOGY	32
5.	IMPLEMENTATION & EXPERIMENTAL WORK	44
6.	RESULTS, ANALYSIS AND DISCUSSION	66
7.	CONCLUSION AND FUTURE SCOPE	87
	REFERENCES	91
	APPENDIX A: ENGINE CONSTANTS AND THRESHOLDS	96
	APPENDIX B: FAULT CLASSES	97
	APPENDIX C: USER GUIDE	98

LIST OF FIGURES

Sr. No.	TITLE	Page No.
Figure 3.1	System architecture	24
Figure 3.2	One 5-second cycle: observation, graph update, the three engines in parallel, ranking and…	24
Figure 3.3	Root-cause ranking: split each deviation into own and explained parts, run the random walk, …	27
Figure 3.4	The three engines in parallel	28
Figure 4.1	The cascade model on a four-service example	39
Figure 4.2	Evaluation method	42
Figure 5.1	Architecture view: the dependency graph of the simulated shop, with arrangement and filter…	52
Figure 5.2	An active incident after orders-db was killed: root cause, cascade, ranking, evidence…	53
Figure 5.3	The parallel view during a slow-payment fault	54
Figure 5.4	An enlarged chart with its time brush and hover read-out	54
Figure 5.5	Cascade risk during the fault: the origin and the services failing on their own…	55
Figure 5.6	The investigation pages: (a) telemetry with fleet totals and a per-service table, (b)…	56
Figure 5.7	The live ShopFlow bench (13 services and its load generator) in the four 3D arrangements,…	58
Figure 5.8	The live LedgerLine bench (12 services and its load generator) in the four 3D arrangements,…	59
Figure 5.9	(a) The Islands view with every project shown: each bench is a district with its own role…	60
Figure 5.10	The Fault Lab control page of the ShopFlow bench: a random campaign, one fault by hand, …	63
Figure 6.1	Two fault-free recordings with the load stepped (left ShopFlow, right LedgerLine; dotted…	67

Figure 6.2	The candidate models	68
Figure 6.3	Sensitivity of the load-aware model to its alarm line (the engine alarms at level 2, that…	70
Figure 6.4	Operating curves	72
Figure 6.5	Paired differences in caught-and-named rate between engines (percentage points, 95%…	73
Figure 6.6	Caught-and-named rate by fault class (percent; faults per class), for four rankers on the…	73
Figure 6.7	Detection delay after fault onset as a cumulative distribution (symmetric-log axis); …	74
Figure 6.8	Ablation of the corrected ML engine	75
Figure 6.9	Amplification study on soft faults (106 per level)	76
Figure 6.10	Who-is-next prediction on the synthetic held-out set	77
Figure 6.11	Call directions	77
Figure 6.12	Early warning	78
Figure 6.13	Fault-type diagnosis	78
Figure 6.14	Cost of one decision window against the number of services (single core, logarithmic axes;…	79
Figure 6.15	Three real incidents	80

LIST OF TABLES

Sr. No.	TITLE	Page No.
Table 2.1	Review of existing work (what each documents, what it needs, and where it stops)	20
Table 2.2	Approaches compared on practical properties (✓ yes, – no, partly)	20
Table 3.1	Functional requirements and where they are met	25
Table 3.2	Non-functional requirements	25
Table 3.3	PEAS description of the two agents	27
Table 3.4	Main components	28
Table 3.5	The design principles of the platform	29
Table 3.6	Design decisions and the alternatives they were chosen over	30
Table 4.1	Measures used in the evaluation	33
Table 4.2	Edge evidence and its confidence	35
Table 4.3	Error sources and how each is handled	40
Table 4.4	Cost per decision window	41
Table 5.1	Implementation environment	44
Table 5.2	Engine parameters (defaults)	50
Table 5.3	The test suites	61
Table 5.4	The two benchmark hosts	62
Table 5.5	ShopFlow scenarios and their true root causes	64
Table 5.6	The evaluation harness and the experiments built on it	64
Table 6.1	Models of normal compared inside the same engine	67
Table 6.2	The three engines on the same recordings	70

Table 6.3	Detection and root cause in one harness (point estimates with 95% bootstrap intervals over…	72	
Table 6.4	Service-level detection by model of normal (mean s.d	74	
Table 6.5	One simulated fault (slow payment): each engine’s first suspect and its peak against its…	80	
Table 6.6	Live campaigns, same fault plan, auto-restart on and off (21 faults each; 95% Wilson intervals)	82	
Table 6.7	Top-1 and Top-3 root-cause accuracy (%) by fault class in the live campaigns (on / off)	82	
Table 6.8	Time a stopped container stayed down, with auto-restart on and off (injector releases it…	83	
Table 6.9	This project beside the approaches it builds on	85	
Table A.1	Rule-layer thresholds (defaults, adjustable on the Incidents page)	96	
Table A.2	Edge evidence confidence	96	
Table B.1	Fault classes, how they are applied, and how many faults of each class the evaluation sets…	97	

Abbreviations / Notations

AIOps – Artificial Intelligence for IT Operations

API – Application Programming Interface

AUROC – Area Under the Receiver Operating Characteristic curve

CI – Confidence Interval

C+N – Caught and named (detected, and the true cause ranked first)

ECE – Expected Calibration Error

FA/h – False-alarm episodes per fault-free hour

GPU – Graphics Processing Unit

MAD – Median Absolute Deviation

ML – Machine Learning

RCA – Root Cause Analysis

REST – Representational State Transfer

SSH – Secure Shell

TCP – Transmission Control Protocol

WS – WebSocket

Notations

S, N – the set of services and its size

$x_s(t)$ – the reading of service s at time t (eight metrics)

Δ – the collection period, 5 seconds

w(e), c(e) – weight and evidence confidence of edge e

level – 0 typical, 1 on the service's alarm line

L(t) – load factor (the median shift of the traffic-carrying signals)

λ_s, ρ_s – model level and rule score of service s

π_s – random-walk share of service s

α – restart probability of the walk (0.15)

$\hat{p}(c \leftarrow x)$ – learned probability that trouble in x reaches its caller c

P(c) – risk that service c is reached from a suspected root

CHAPTER 1

INTRODUCTION

1.1 Background

Most large applications today are built as many small services that talk to each other over the network. A shopping site, for example, has separate services for the catalogue, the cart, payment, shipping and orders, each with its own database or cache [1]. This style allows teams to deploy and scale each piece independently.

Take a slow database. Its client waits, so it answers slowly. The services that call that service begin to time out, retry, and add load on top of load. A dozen services alarm within a minute, and the on-call engineer faces a wall of red in which the first failure is no louder than its victims. Practitioners call this an alert storm. The alerts themselves cost little; deciding which one deserves action costs hours [2, 3].

Two facts make the problem tractable. Trouble follows the call graph, from a callee to the services that wait on it. And it follows the graph *probabilistically*: a caller with a timeout, a cache or a circuit breaker can survive a failing dependency, while one without them cannot. A tool that knows the graph and the chance that trouble crosses each edge can answer what a ranked list of alarms cannot. Which service started it? Which healthy services will fall next?

Recent research answers the first question with heavy machinery: models that fuse metrics, traces, and logs into learned representations and localise the fault with graph neural networks [4, 5, 6]. They work. They also demand labelled failures that most teams never collect, accelerated hardware at inference time, and they return output (an embedding ranking, a cluster number) that an engineer cannot check against the dashboard. Classic observability products take the opposite trade. They show nothing until someone adds client libraries, agents or a sidecar to every pod.

It watches the containers from outside, over SSH and the Docker socket, so that nothing in the monitored system is modified. From these observations, it builds a live dependency graph, and on top of that graph, it runs an unsupervised, single-core diagnosis pipeline that proposes a root cause, a prediction of what will fail next, a fault type, and a short runbook, each backed by numbers for the operator to inspect.

1.2 Problem Statement

The project addresses four linked problems that a team running containerised microservices encounters in practice:

- Invisible dependencies. The real call graph of a running system is rarely documented and drifts over time. Compose files list what was declared, not what is actually communicating. Without instrumentation, no one can say which service calls which, or in which direction.

- Alert storms. When one service fails, its callers fail too. Ranking alarms by severity often points to the loudest service, which is often a victim and not the cause.

- No view of what comes next. Operators learn that a service is affected only after it alarms. A model of how failure spreads on this particular system could warn earlier.

- Black-box diagnosis. Methods that need training data and return an opaque score are hard to trust at three in the morning. An engineer needs to see the evidence and how it was combined.

The problem is therefore to observe a containerised system without touching it, recover its call graph, detect trouble per service against that service's own normal, rank the likely origin of an incident, predict how it will spread, and show every step in a form an engineer can verify, with no labelled data and no specialised hardware.

1.3 Motivation and Need for the Project

Small teams and student projects run the systems that heavyweight AIOps products serve badly: a few Docker hosts on a cloud account, no budget for per-host licences, applications nobody wants to re-instrument. These systems still fail in cascades. We watched it on our own benchmarks: killing one datastore container raised alarms on its callers within seconds.

Three observations shaped the work. The host already shows most of what an operator needs: container state, CPU and memory against the container's own limit, network counters and each container's table of open TCP sockets. A per-service model of "normal" replaces global thresholds, which fail because services differ by orders of magnitude in every metric. And a result nobody can reproduce or audit is worth little, so the evaluation had to meet the standard of the tool: ground truth the detector never sees, confidence intervals, paired tests and an honest list of failures.

1.4 Objectives

1. To build a platform that connects to Docker hosts over SSH, discovers the service topology, and shows live health, metrics and dependencies in a 2D and a 3D view, without changing the monitored applications.

2. To recover call directions from observed sockets, and where names carry no information, from the services' own behaviour.

3. To detect trouble per service against its own recent history with a hybrid of a load-aware learned model and a robust statistical rule layer, treating container facts such as "stopped" or "crash-looping" as evidence rather than degree.

4. To rank the root cause of an incident by separating a service's own deviation from what its neighbours explain, and by running a random walk over the call graph.

5. To model cascading failure with learned per-edge propagation probabilities, predict which services are next, and warn early about gradual faults.

6. To classify the fault type and attach a short runbook, so the output is something an engineer can act on.

7. To make the three engines (Rules, ML, Hybrid) switchable and to show, side by side and on the same readings, what each one detects, computes, and predicts.

8. To evaluate all of this on real and synthetic fault campaigns with ground truth that the detector never sees, reporting confidence intervals, paired tests, an ablation, and the negative results.

1.5 Scope of the Project

The system targets Docker-based microservice stacks on Linux hosts that can be reached over SSH. It reads container state, statistics, and socket tables; it does not read application payloads and does not require code changes. It was developed and measured on the following applications:

- ShopFlow, a 13-service retail checkout system and **LedgerLine**, a 12-service retail-banking transfer system. Both were built for this study as test benches, each on its own EC2 host with its own Fault Lab and a Prometheus–Grafana–Loki monitoring stack (Section 5.10); both were run under open-loop traffic for 217 minutes with 62 faults injected;

- Sock Shop, the open microservices demo, used for development;

- a social-network application from the DeathStarBench suite [7] (27 containers), and the OpenTelemetry demo, used to check that discovery works on third-party systems;
- a synthetic generator that writes runs in the same recorded format over 16 random topologies, used to enlarge the evaluation.

The mapper is not a distributed tracing tool: it does not include request identifiers or spans, and the “Traces” page displays sampled TCP connection snapshots. It does not replace an application performance monitoring suite. No language model is involved in the diagnosis; the “assistant” is a transparent composition of statistical models, and Chapter 7 discusses where a language-model agent could be added.

1.6 Significance and Expected Contribution

The project shows that useful root-cause analysis does not need instrumentation, labels or a GPU, and it measures how far that goes. Its contributions are:

- an **instrumentation-free observation layer** that recovers topology and call direction from container sockets and Docker state;
- a **probabilistic cascading-failure model** whose per-edge propagation probabilities are Beta–Binomial posteriors learned online from the system’s own incidents, with risk flowing over the graph by noisy-OR;
- a **load-aware model of normal** that explains away the workload every service shares and judges only what is left, so a change of traffic is not mistaken for a fault (it replaced a per-service Isolation Forest that was in alarm for most of a load test), and a **hybrid detector** in which that model and a robust rule layer jointly raise the alarm and container facts count as evidence, with an operating contract that reports alarm *time* as well as false-alarm episodes;
- a **fact-aware, decomposition-based ranker** for the root cause, and **call directions learned from telemetry**;
- a **Detection engines page** that runs Rules, ML and Hybrid in parallel and shows what each detected, computed and predicted;
- a **reproducible evaluation** with a blind fault injector, a held-out synthetic benchmark, bootstrap intervals, paired tests, an ablation and an amplification study, together with the results that did not help and a measurement bug that had hidden half of the real faults;

- a working **platform** of about 36 000 lines of TypeScript with 339 automated tests, a traffic generator and a host-check tool, used throughout the experiments.

1.7 Organization of the Report

Chapter 2 reviews related work and states the gap. Chapter 3 formulates the problem and presents the system, its architecture and its two agents. Chapter 4 sets out the models, the evaluation measures, the blind fault injector and the statistics. Chapter 5 covers the implementation: modules, core code, algorithms, configuration, the prototype and testing. Chapter 6 reports the experiments, compares the system with existing approaches and records what the benchmark hosts taught us. Chapter 7 concludes with limitations and future work. The appendices give the engine constants, the fault classes and a short user guide.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This project draws on five bodies of work: anomaly detection on time series, root-cause localisation in microservice systems, models of how failure spreads, evaluation practices for anomaly and diagnosis methods, and the observability tools that supply the data. Each statement about a cited work below restates what that work documents; where we compare, we specify what we compared.

2.2 Review of Existing Work

2.2.1 Anomaly detection

Isolation Forest [8, 9] isolates an outlier with a few random cuts and needs no density estimate, which suits per-service streams of mixed, skewed metrics. It powered the first version of this engine, and Chapter 6 shows where it breaks: a change of workload. Network monitoring met the same problem earlier. Traffic on many links follows a few shared causes, so researchers model the readings as a low-dimensional subspace and call anomalous whatever falls outside it [10, 11]. Distance and density methods [12, 13], the histogram score HBOS [14] and Hotelling's multivariate control chart [15, 16] with a shrinkage covariance [17] are the obvious alternatives; Chapter 6 tests all of them. The median and the median absolute deviation give robust location and scale [18, 19]. They underpin our rule layer and, with a tail-aware spread [20], the load-aware model. Surveys of streaming and learned detection [21, 22, 23] and deployed monitoring detectors [24, 25] describe our setting: unlabelled streams, drifting baselines and costly false alarms.

2.2.2 Evaluation of anomaly detectors

Several studies fault the common habit of scoring detectors point by point and crediting a whole event when any point inside it is flagged [26, 27]. They argue for range-based precision and recall that respect an event's duration [28]. We follow them. Chapter 4 scores at the event level: a fault counts as detected only when an alarm names a truly involved service while the fault is active, and a false alarm is an episode outside every fault window.

2.2.3 Root-cause localisation in microservices

MonitorRank [29] introduced a personalised random walk over the call graph, biased by how anomalous each service looks, and ranks the services by the walk's stationary share.

MicroRCA [30], CloudRanger [31], Microscope [32], AutoMAP [33] and MicroRank [34] extend this idea with anomaly-weighted transitions, causal-graph construction and trace spectra. Causal approaches recover structure using the PC algorithm or hierarchical discovery [35, 36, 37]. Learned methods localise with graph autoencoders and multi-source fusion of metrics, traces and logs [4, 5, 6, 38]. A survey of AIOps approaches to failure management [3] and one specific to anomaly and root-cause work in microservices [39] place these methods in context. Most of them require traces, labelled failures or training, and some need accelerated hardware; the random-walk family, however, needs only the call graph and an anomaly score, which is the basis for this project.

2.2.4 Cascading failure and change detection

The independent-cascade model of influence spread [40] forms the basis of our propagation model, with edge probabilities estimated online rather than assumed. Probabilistic forecasts are assessed using proper scoring rules, reliability diagrams, and expected calibration error [41, 42, 43]. Detection latency can be addressed with classical change-point detection [44, 45]. PageRank [46] and reciprocal-rank fusion [47] inform ranking; Granger causality [48] and transfer entropy [49] inform the study of call direction; random forests and CART [50, 51] support the fault-type classifier.

2.2.5 Benchmarks and chaos engineering

Open benchmark applications such as DeathStarBench [7] and Train-Ticket [52] provide realistic microservice systems to test on, and chaos-engineering practice [53] provides the method for injecting faults on purpose. Our fault injector follows that practice but, importantly, keeps its ledger private: nothing inside the system under test is labelled.

2.2.6 Observability tooling

Production observability usually combines three kinds of tools. Metric systems scrape numbers from endpoints that each service exposes. Distributed tracing follows a request across services and needs the application, or a proxy in front of it, to propagate trace identifiers. Application performance monitoring products add agents inside the runtime. All three give an accurate picture once the instrumentation exists; none works on a stack that nobody is willing to modify. Service meshes can supply topology without code changes, but they require a sidecar in every pod. The site-reliability literature [2] is clear. That the hard part is not collecting signals but deciding what to look at first, which is the gap that root-cause ranking aims to address.

Table 2.1: Review of existing work (what each documents, what it needs, and where it stops).

Work	Approach	Needs	Limitation for our setting
Isolation Forest [8]	Isolate outliers with random trees; score by path length	Unlabelled history	Gives a score, not a cause; blind to the workload
Subspace residual [10]	Model the shared causes of many signals as a subspace; score the residual	A matrix of signals	Learns the subspace from history; slow to adapt to a new load pattern
MonitorRank [29]	Personalised random walk over the call graph	Call graph, anomaly score	Direction and edge weights are inputs
MicroRCA [30], CloudRanger [31], Microscope [32], AutoMAP [33]	Extend the walk with anomaly-weighted transitions and graph construction	A graph and monitoring data (sources differ by method)	The graph is built from data the host alone does not give
MicroRank [34]	Trace spectrum analysis with PageRank	Request traces	Needs tracing in every service
RCD [35], CIRCA [36]	Causal discovery or structural causal graph for localisation	Metrics, structure	Assumes a faithful causal model
Eadro [6], DeepHunt [5], DiagFusion [4]	Learned fusion of metrics, traces, logs; graph neural or autoencoder models	Labelled or curated failures; GPU	Not auditable; training data needed
Independent cascade [40]	Probabilistic spread over a graph	Edge probabilities	Probabilities must be estimated
Change-point tests [44, 45]	Detect a shift in a stream	A stream	Alone, no cause or graph

2.3 Comparative Analysis

Table 2.2 compares the approaches on the properties that decide whether a method can be used on an unmodified Docker stack. The proposed system is deliberately not the most accurate method one could imagine; it is the one that can be switched on in minutes, explains itself, and has been measured honestly.

Table 2.2: Approaches compared on practical properties (✓ yes, – no, ∼ partly).

Property	Metric tools	Tracing	Learned fusion	Walk RCA	This project
No change to applications	–	–	–	∼	✓
Works without labelled failures	✓	✓	–	✓	✓
Recovers the call graph itself	–	✓	∼	–	✓
Learns call direction	–	✓	–	–	✓
Per-service learned baseline	∼	–	✓	∼	✓
Ranks a root cause	–	∼	✓	✓	✓
Predicts what fails next	–	–	–	–	✓
Needs a GPU	–	–	✓	–	–

Property	Metric tools	Tracing	Learned fusion	Walk RCA	This project
Every score can be audited	✓	✓	–	∼	✓

2.4 Research and Design Gap

Four gaps motivate the design.

1. Observation without instrumentation. The graph that random-walk and causal methods assume is usually given; recovering it, with direction, from the host alone is rarely addressed.

2. Facts are not degrees. A stopped container is not “very anomalous”; it is a fact. Methods that fold container state into one score lose the most reliable signal there is, and our ablation shows how much that costs.

3. Propagation is assumed, not learned. Cascade prediction typically uses fixed edge probabilities. Learning these probabilities from the system’s own incidents, with a prior that ensures rarely used edges behave sensibly, is both cheap and measurable.

4. Evaluation is optimistic. Point-adjusted scoring, tuned thresholds and a single data set flatter results; we use event-level scoring, a held-out split and paired tests, and report what failed.

2.5 Summary

Existing root-cause methods are accurate but assume instrumentation, labels or hardware that small teams do not have, and are often evaluated optimistically. The proposed system, described next, is instrumentation-free, unsupervised and auditable.

CHAPTER 3

PROPOSED SYSTEM AND SCOPE

3.1 Problem Formulation

A system is a set S of N services with a directed call graph. An edge $u \to v$ means that u calls v, so a failure travels *against* the arrow, from v to the callers that wait on it. Every $\Delta = 5$ s the collector produces, for each service s, a reading

$$x_s(t) \in \left(R \cup \{\varnothing\}\right)^8,$$

holding CPU, memory, CPU throttling, network in and out, request rate, error share and 95th-percentile latency. The symbol $\varnothing$ marks a metric that was not reported. It is never replaced by zero, as a stopped container must be distinguishable from an idle one. Each reading comes with a container state (running, paused, restarting, exited, detached).

At time t the pipeline must decide four things, using only readings up to t:

1. whether an incident is in progress;
2. which service is its root cause r;
3. which currently healthy services will be affected before the incident ends;
4. what kind of fault it is.

The pipeline must achieve this without labelled failures, without modifying any application, and fast enough for the decision window to finish well inside the 5 s collection period.

3.2 Proposed Solution

The proposed system is an agent-assisted pipeline in a concrete sense. A **collector agent** observes the containers from outside the application. A **diagnosis assistant** turns those observations into a cause, a forecast of what fails next, a fault type, a runbook and an explanation the operator can check. Between the two sits the evidence-weighted dependency graph. Figure 3.1 shows the whole system, and Figure 3.2 illustrates one 5-second cycle.

The solution has seven steps, each described in Chapter 4.

1. Observe. Read container state, statistics and socket tables over SSH. Install nothing on the host.

2. Build the graph. Merge declared dependencies (from the Compose file) with observed ones; orient each observed socket by the listening port; assign a confidence and a dependency share to each edge.

3. Learn directions. Where the heuristic cannot determine who calls whom, vote over dynamic and structural signals.

4. Detect. Judge each service against what the current load predicts for it, using its own recent past, with a robust rule layer beside the model. Treat container facts as evidence.

5. Rank. Split each service’s deviation into what its callees explain and what they do not; run a personalised random walk over the call graph; present facts first.

6. Predict. Spread risk over the graph by using noisy-OR with edge probabilities learned from the system’s own incidents; warn early about gradual faults.

7. Diagnose and explain. Classify the fault type, attach a short runbook, and show the evidence behind every number.

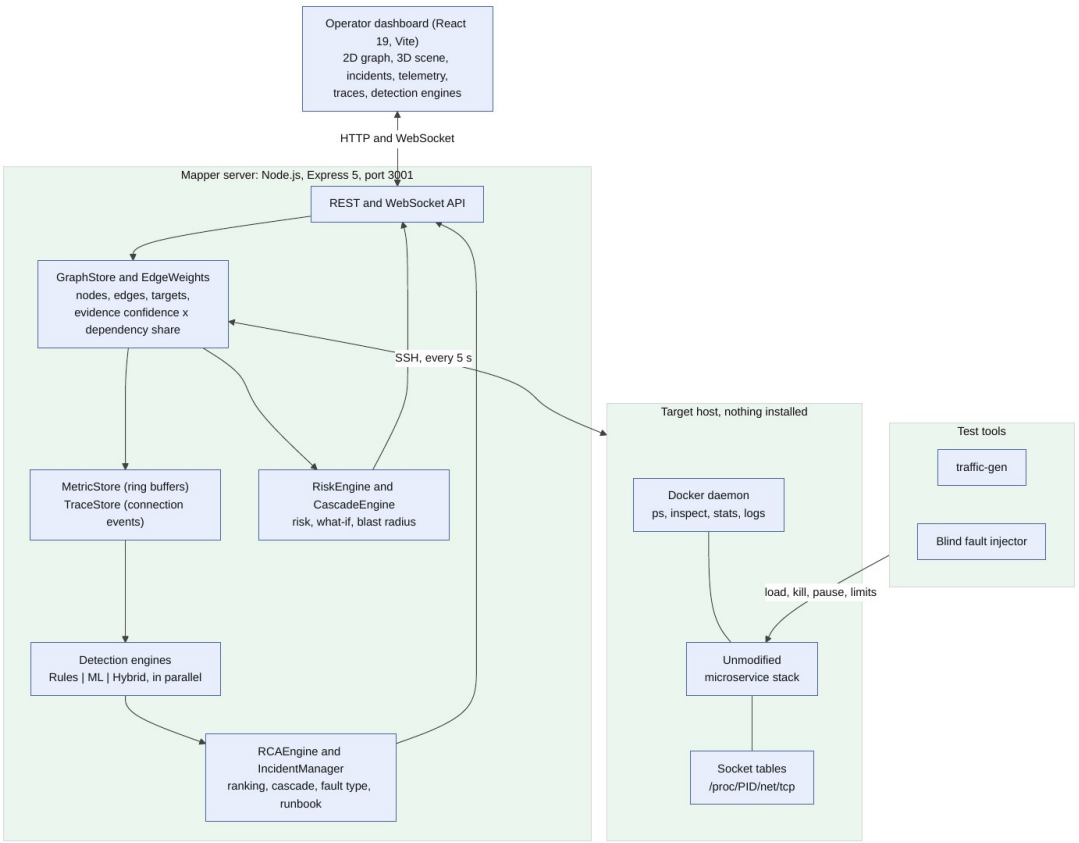


Figure 3.1: System architecture. The server reaches the target host over SSH and reads Docker and the container socket tables; nothing is installed on the host. The three detection engines share the same readings and graph; the dashboard receives updates over a WebSocket.

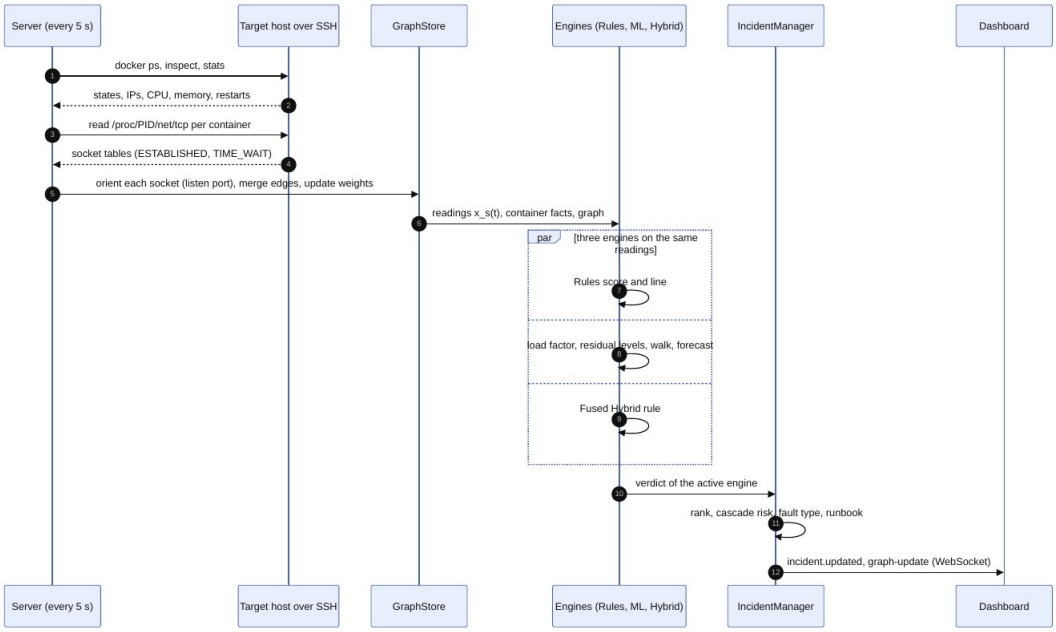


Figure 3.2: One 5-second cycle: observation, graph update, the three engines in parallel, ranking and the push to the dashboard.

3.3 Requirements

3.3.1 Functional requirements

Table 3.1: Functional requirements and where they are met.

ID	Requirement	Component
FR1	Connect to a Docker host over SSH with a key pair and discover its containers and Compose dependencies	Connection manager, discovery engine
FR2	Collect CPU, memory, throttling, network, restarts and request statistics every 5 s	Metric collector, MetricStore
FR3	Recover observed dependencies and their direction from container socket tables	Collector, GraphStore, EdgeWeights
FR4	Show the live topology in 2D and in 3D, with health, link activity and layouts to choose from	Dashboard (graph canvas, 3D scene)
FR5	Detect abnormal behaviour per service against its own baseline	Rules, ML and Hybrid engines
FR6	Raise an incident and rank the root cause with an evidence breakdown	RCAEngine, IncidentManager
FR7	Predict which healthy services are likely next and give an early warning for gradual faults	CascadeEngine, ML engine
FR8	Classify the fault type and give a runbook	Diagnosis layer, remediation guide
FR9	Let the operator switch the engine and compare all three on the same readings	Engine switch, Detection engines page
FR10	Generate load and run experiments against a target	TrafficController, traffic-gen
FR11	Record runs and score the engines against a private fault ledger	Evaluation harness (run/cli)
FR12	Report host and mapper problems separately	check-host, mapper-edges

3.3.2 Non-functional requirements

Table 3.2: Non-functional requirements.

Quality	Requirement
Non-intrusive	No code, agent or sidecar is added to the monitored application; the host needs only Docker and SSH access
Unsupervised	No labelled failures and no training phase beyond the first minutes of normal readings
Auditable	Every score shown to the operator can be traced to a measurement and a stated rule
Fast	A decision window finishes in a small fraction of the 5-second collection period on one CPU core
Honest	Container state is shown as a fact, missing metrics as missing, and measured limits are stated rather than hidden
Safe to connect	Remote commands come from a fixed catalogue whose arguments are sanitised; keys stay outside the repository

Quality	Requirement
Reproducible	Every reported number is regenerated by a script in the repository
Portable	Runs on macOS, Linux and Windows with Node.js; a Python sidecar is optional

3.4 System Architecture

The architecture has five layers (Figure 3.1).

Dashboard. A React 19 application built with Vite. It never polls: the server pushes graph and incident updates over a WebSocket, and REST calls fetch history and configuration. The views are described in Section 5.7.

Server. A Node.js process (Express 5, TypeScript) that owns all state and is the only component that writes to disk. The GraphStore holds nodes, edges and targets, the MetricStore a ring buffer of readings per service, and the TraceStore the connection events. The IncidentManager drives the engines and turns their verdicts into incidents.

Connection and collection. A connection manager keeps one SSH session per target and runs commands from a fixed catalogue (docker ps, inspect, stats, logs, and reads of /proc/<pid>/net/tcp). Reading the socket table from the host through the container's process ID works for every image, including those with no shell. An optional standalone collector can run on the host and push to the same ingest endpoint.

Target workloads. Unmodified Docker Compose applications.

Test tools. A traffic generator with per-application workflows and profiles, and a fault injector that works on containers (kill, pause, resource limits, network detach) and writes its ledger to a file the detector never reads.

The server listens on port 3001 (HTTP and WebSocket), the dashboard on 5173 and the traffic generator on 4400. The server reaches each host over SSH on port 22, and the traffic generator reaches the application's front end, typically on port 8080.

3.5 Agent Design

3.5.1 The two agents

Table 3.3 describes the two agents with the PEAS scheme (performance measure, environment, actuators, sensors).

Table 3.3: PEAS description of the two agents.

	Collector agent	**Diagnosis assistant**
Performance	Complete, timely and truthful readings; correct edges and directions	Caught-and-named rate; alarm time; calibration of predictions
Environment	A Docker host reached over SSH, running an unknown microservice stack	The evidence-weighted graph and the readings it holds
Actuators	SSH commands from a fixed catalogue	Incidents, rankings, predictions, runbooks, explanations in the dashboard
Sensors	Docker state and statistics, socket tables, logs, interface counters	Readings, container facts, graph, learned directions

3.5.2 Decision logic of the diagnosis assistant

The assistant runs one decision window per cycle (alarm rule in Figure 3.3, ranking in Figure 3.4). An incident opens as DEGRADED, or as INCIDENT when a service is critical. It updates while the alarm holds and resolves when the alarm clears.

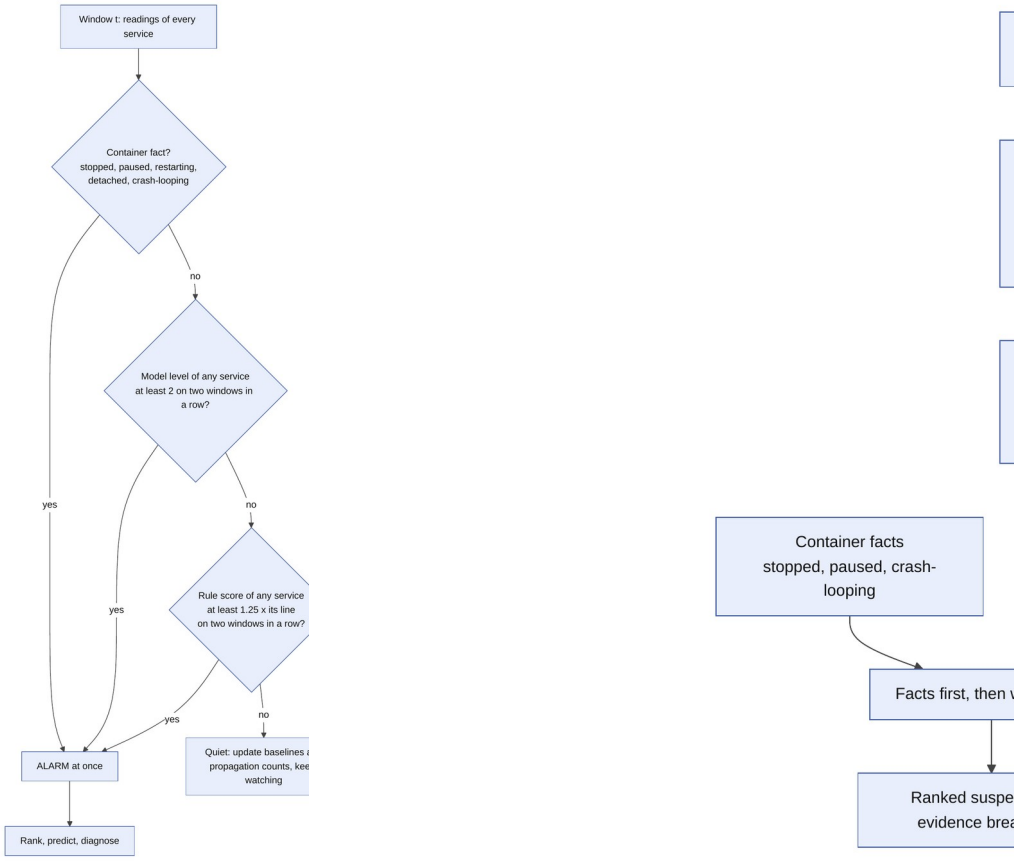


Figure 3.3: Root-cause ranking: split each deviation into own and explained parts, run the random walk, put facts first.

3.5.3 Three engines on the same readings

The dashboard has a switch with three positions: Rules, Hybrid, and ML. Only the active engine decides the incident, but all three are computed on every cycle from the same

readings and graph (Figure 3.5). This makes two things possible: switching engines does not start the learned detector from scratch, and the Detection engines page can show what each engine detected, computed, and predicted at the same moment.

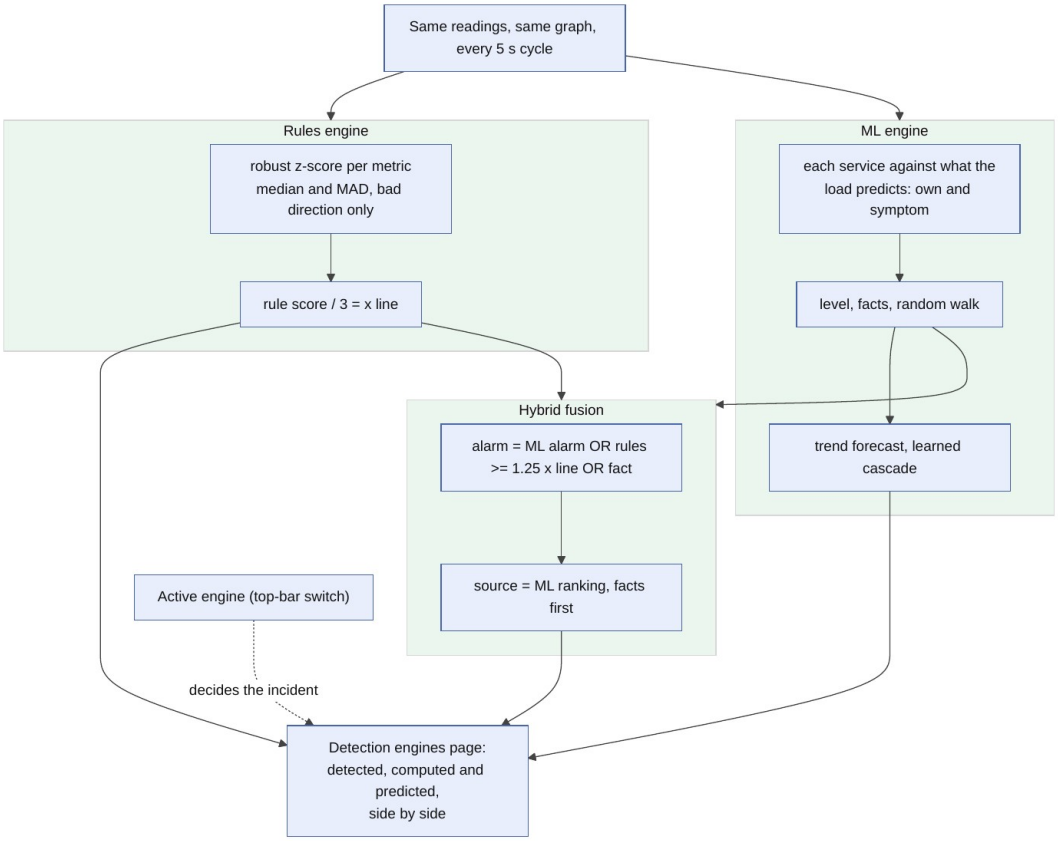


Figure 3.4: The three engines in parallel. The rules score, the ML engine and the fused Hybrid rule are computed on the same inputs; the switch only chooses which one decides the incident.

3.6 Functional Components

Table 3.4 lists the main components with their responsibility and location in the repository.

Table 3.4: Main components.

Component	Responsibility	Location
ConnectionManager	One SSH session per target; fixed command catalogue; timeouts sized for a loaded host	server/telemetry-platform
MetricCollector	Docker statistics, socket tables, interface counters; orients each socket	server/telemetry-platform
EndpointDiscoveryEngine	Topology from Compose and running containers; routes and endpoint health	server/discovery
GraphStore	Nodes, edges, targets; merges declared and observed edges; prunes stale ones	server/graph

Component	Responsibility	Location
EdgeWeights	Evidence confidence and dependency share per edge	server/graph
MetricStore, TraceStore	Ring buffers of readings; connection events with a 24-hour window	server/telemetry, server/traces
Detection algorithms	Rule scores, container facts, features, thresholds	server/algorithms/detection
ML engine	Load-aware residual model, random walk, forecast, learned cascade, hybrid fusion	server/algorithms/ml
RCAEngine, IncidentManager	Ranking, explanation, incident lifecycle	server/algorithms/rca, server/rca
RiskEngine, CascadeEngine	Per-service risk, what-if failure, blast radius	server/graph
TrafficController	Starts and stops load; resolves the target URL; any project is accepted	server/traffic
traffic-gen	Workflows, profiles, virtual users	traffic-gen
Evaluation harness	Records runs, scores engines against a ledger	run/cli, v2
Dashboard	Views, 3D scene, engine switch	client/src

3.7 Design Principles and Decisions

Most of the code follows from a short list of rules. When a piece of code looks stranger than it needs to be — for example, a field left empty where zero would be simpler, a score split into named parts, or a loop that waits for itself — one of these rules is usually the reason.

Table 3.5: The design principles of the platform.

Principle	What it means and why	Where it shows
Nothing is installed on the target	The tool watches over one SSH connection with a fixed set of Docker and /proc reads, so it can monitor a system its operator does not own without changing it. The cost: no per-request data by default	Command catalogue; RemoteCommand.ts
Observed beats declared	A depends_on line is a claim; an open socket is a fact. The graph keeps both and never weighs them equally: each edge carries a confidence from its evidence	Edge confidence (Table 4.2)
Unmeasured is missing, never zero	Zero means “measured, and it was zero”. A made-up zero looks real on screen, and “idle” and “not measured” call for opposite decisions	Metrics, edge activity, health factors; the 2D cards show a dash

Principle	What it means and why	Where it shows
Every number can be checked by hand	Every score is arithmetic over named inputs, and the parts sit beside it	Evidence breakdown on the incident page
An arrow means “calls”	An edge $A \rightarrow B$ means A calls B, so a failure at B hits A and moves against the arrow. Reversing it flips everything downstream	Orientation by listening port
Compare like with like	Degree, traffic share and criticality are computed inside one project, traffic shares split functional from telemetry calls, and every model trains on one service’s own history	Per-service baselines
Rules and models stay separate	Incidents come from the detection engines. Findings from the optional learned models stay apart, so no model is graded on its own output	Findings page
Be light on the target	Each loop waits for itself (the next sweep starts 5 s after the last one ended), reads each container once per sweep and rediscovers everything every 180 s; going deeper needs per-target consent	Collector and discovery

Table 3.6: Design decisions and the alternatives they were chosen over.

Decision	Alternatives	Why this one
SSH, Docker CLI and /proc	An agent per host, eBPF, a service mesh	Needs nothing installed; works on any Docker host that can be reached over SSH
Direction from the listening port	Trusting the scanned container as the source	Both ends see every connection; the scan’s own view produced a reversed duplicate of every link
Byte counters from /proc/<pid>/net/dev	docker stats network figures	Docker rounds to three significant figures, so a 9.87 MB counter moves in 10 kB steps
Additive ranking factors	A trained classifier	There are no labelled incidents to train on, and additive factors can be explained
Noisy-OR cascade	Plain reachability	Reachability treated any chain as total failure
Islands layout for the 3D scene	A grid, a “server room”, a building	Sections read at a glance and each project gets its own district; we tried and rejected the room and building versions

3.8 Scope and Constraints

In scope. Docker-based services on Linux hosts reachable over SSH; container-level metrics, socket tables and logs; unsupervised detection; ranking; cascade prediction; fault typing; a dashboard; a reproducible evaluation.

Out of scope. Distributed tracing (the “Traces” page shows sampled TCP connection snapshots, not requests); application payloads; Kubernetes clusters (not tested); a language-model agent (discussed as future work); training on labelled data.

Constraints that shape the design.

- *Coarse signals.* Observing from outside yields eight metrics per service every 5 s. A brownout on an almost idle service moves none of them enough to alarm, and Chapter 6 reports it as an open problem.

- *Two views of each socket.* Both ends see every connection, so direction must be recovered; otherwise every dependency looks like a cycle.

- *Slow answers from loaded hosts.* Command timeouts are sized for a host under heavy load. A host that cannot answer ‘docker ps‘ at all shows nothing, and Section 6.1.15 gives a measured example.

- *One write.* The collection path only reads. The mapper makes one write to a host, which restarts a container that exited with a non-zero code, limited to a few attempts per container and switched off with MAPPER_AUTO_RESTART=off. Load generators are never restarted by this.

CHAPTER 4

METHODOLOGY

4.1 Objective of the Study

The study asks how far an instrumentation-free, unsupervised pipeline can go in answering four operational questions about a running microservice system: is something wrong, where did it start, who is next, and what kind of fault is it. It also asks, with equal weight, how reliable the answers are. On the synthetic track, the settings were chosen on a tuning split kept apart from the data used for the reported numbers; on the real track, they were not tuned on a held-out system, and the report states this where it matters.

4.2 Method and Approach

The approach is a pipeline of small, separately testable models rather than one large model. Each stage has a stated input, a stated output, and a measured contribution (Section 6.1.7).

1. an **evidence-weighted graph** from observed sockets and declared dependencies;

2. learned call directions for pairs the heuristic cannot orient;

3. a **hybrid detector** of a load-aware residual model and a robust rule layer, with container facts;

4. a **ranker** built on a deviation decomposition and a personalised random walk;

5. a **cascade model** with Beta–Binomial edge probabilities and noisy-OR risk, plus trend-based early warning and change-point alarms;

6. a **fault-type classifier** and runbook.

The evaluation then replays recorded runs through the engines and scores them against a ledger that the engines never see.

4.3 Measurement and Evaluation Parameters

Table 4.1 defines the measures. They are event-level on purpose: point-wise scoring with a window adjustment rewards an engine that flags one point of a long fault, which is not how an operator experiences an alarm [26, 27, 28].

Table 4.1: Measures used in the evaluation.

Measure	Definition
Detected (Det.)	Share of faults for which an alarm during the fault (or within a 60 s grace period) places a truly involved service in the top three
Caught and named (C+N)	Share of all faults that are detected and for which the faulted service is first at the last alarming window before the fault heals
A@k	Share of detected faults whose true cause is in the top k
False alarms per hour	Alarm episodes outside every fault window, per fault-free hour
Alarm time	Fraction of fault-free time spent in alarm
Detection delay	Seconds from fault onset to the first qualifying alarm
AUROC, average precision	Service-level discrimination: positives are the faulted service during a fault; negatives are any service when no fault is active; container facts are excluded
Brier score, ECE	Accuracy and calibration of who-is-next probabilities
Direction accuracy	Share of observed true-call pairs oriented correctly, wrongly or left unresolved
Cost	Milliseconds per decision window on one core

Two operating contracts. An operator can budget false-alarm *episodes* per fault-free hour. But one unbroken alarm counts as a single episode, so an engine can meet an episode budget while alarming almost continuously. We therefore also report alarm time. An earlier version of our own detector had a low episode rate and 86% alarm time on the real systems; the refinements of Section 4.6.3 cut it to 5%.

4.4 Tools, Equipment and Resources

The server and the dashboard ran on an Apple Mac mini with Node.js 22. The target hosts were AWS EC2 c7i-flex.large instances (2 vCPUs, 4 GiB) in ap-south-1 running Ubuntu, Docker and Docker Compose. For the live recordings of Section 4.10.4 both benches ran in Docker on one Linux machine. ShopFlow (13 services) and LedgerLine (12 services) were the applications under test; Sock Shop, a DeathStarBench social network and the OpenTelemetry demo served during development. traffic-gen and the benches' own generators supplied load, and the blind Fault Lab supplied faults. Matplotlib drew the figures, and LaTeX and Mermaid typeset this report. Table 5.1 lists the software stack.

4.5 Experimental and Implementation Procedure

1. Prepare the applications. Start ShopFlow and LedgerLine with Docker Compose on the lab hosts.

2. Connect the mapper. Add each host (address, user, key) in the dashboard; the server discovers containers and Compose dependencies and starts the 5-second cycle.

3. Drive traffic. Run open-loop traffic with a daily wave, bursts, and flash crowds for 217 minutes per system.

4. Inject faults blind. The injector draws faults from a Poisson process whose rate follows the load, applies them physically to containers, and writes the ledger to a private file.

5. Record. The collector's readings and container facts are recorded in a trace format that the engines can replay offline.

6. Replay and score. Each engine runs on the recorded trace, and scores are computed against the ledger after the run.

7. Extend synthetically. A generator writes runs in the same format over 16 further topologies, tuning on one split and scoring once on a disjoint split.

8. Report. Intervals, paired tests, an ablation study, and an amplification study are generated by scripts in the repository.

4.6 The Models

4.6.1 The evidence-weighted graph

Each edge has a *confidence* $c(e)$, the strongest evidence behind it: 1.0 for a call seen in request logs or oriented by a listening port, 0.85 when only the ephemeral-port range orients it, 0.70 when observed without direction evidence, 0.50 when the direction is unresolved, and 0.40 when the Compose file declares it but no traffic was ever seen. The *dependency share* $d(e)$ of an edge leaving a caller is its slow-baseline traffic normalised over the caller's edges. The slow baseline matters: a link goes quiet exactly when its callee dies, so weighting by the instantaneous rate would make a dead dependency look less and less important. The weight is

$$w(e) = clip(c(e)\,d(e)\,\phi, 0, 1),$$

with ϕ discounting fire-and-forget telemetry edges to a tenth. Table 4.2 lists the confidence of each kind of evidence. *The offline benchmarks replay recordings that keep no edge evidence, so they use unit weights, and this report does not measure the effect of the evidence weighting.*

Table 4.2: Edge evidence and its confidence $c(e)$.

Evidence	**Confidence** $c(e)$
Seen in request logs, or oriented by a listening port	1.00
Oriented only by the ephemeral-port range	0.85
Observed, no direction evidence recorded	0.70
Observed, direction could not be decided	0.50
Declared in the Compose file, never seen carrying traffic	0.40

4.6.2 Learned call directions

The collector sees most pairs from both ends (in the recorded runs 15 of 32 pairs in one system and 18 of 36 in the other), so direction must be recovered before a walk can follow it. The shipped heuristic reads names (an entry point calls a service, a service calls a datastore) and fails when names carry no role. We instead learn the direction of each observed pair online from the services' own histories, with no labels and no names. Four *dynamic* signals use what services do during trouble: upward symptoms, downward traffic drops, implication ($P(T_a / T_b) - P(T_b / T_a)$, with T meaning "in trouble"), and a Granger-style F-test on the symptom series. Two structural signals use the observed graph: the depth of each service from the entry point, and whether a service reports request metrics of its own (a datastore does not). The learned direction is the sign of $\sum_m \omega_m s_m(a,b)$ with weights $\omega_m \in \{0.5, 1, 2\}$ chosen by grid search on the tuning split. A pair stays undecided if the magnitude falls below 0.05.

4.6.3 A load-aware model of normal

A service is judged against its own recent past, never against a global threshold, because services differ by orders of magnitude in every metric. That alone falls short. CPU and network follow the work a service receives, so a model of "normal readings" fitted on a quiet stretch calls a busy stretch anomalous, and the reverse. The first version of this engine used a per-service Isolation Forest and made exactly that mistake: when traffic stopped, every service read as anomalous at once (Section 6.1.1). The model below strips out the part of "unusual" that only means "busier" or "quieter" and judges what remains. A fault in one service changes that service, not all of them by the same amount.

Readings. Each signal goes on a log scale with a unit below which a change is noise, $x_{s,f} = \log(v_{s,f} + u_f)$. The units are $u_f = 5$ for CPU (so a one-point blip on an idle service is no event), 1 for memory, 2000 B/s for network and 0.1 for request rate. A missing signal stays missing and never becomes zero.

Reference and load factor. The reference $r_{s,f}$ is the median of a service's last 360 readings (half an hour), so a short quiet spell does not move it. A signal *carries traffic* if its reference sits at least 0.4 above its idle level, $r_{s,f} - \log u_f \geq 0.4$; an idle database reads the same zeros whatever the load, so it says nothing about it. The *load factor* is the median shift of the carrying signals of network in and out, request rate and CPU,

$$L(t) = \underset{(s,f) \in A(t)}{median} \left(x_{s,f}(t) - r_{s,f} \right), \tag{4.1}$$

where $A(t)$ is the set of carrying signals, used only when at least two services contribute. A fault in a few services cannot move a median. e^L is the factor by which the project's traffic changed, and the Detection engines page shows it.

Response. A signal may ignore the load (a message bus idling at the same rate) or follow it (a gateway). The project's own data set the slope b_f: a ridge regression of the shifts $d_{s,f}(\tau) = x_{s,f}(\tau) - r_{s,f}$ on $L(\tau)$ over the carrying signals in the stored window,

$$b_f = \frac{\sum_{s,\tau} d_{s,f}(\tau) L(\tau) + \kappa b_f^0}{\sum_{s,\tau} L(\tau)^2 + \kappa}, \kappa = 60, \tag{4.2}$$

pulled towards a prior b_f^0 (1 for network and requests, 0.4 for CPU, 0 for memory, errors, latency and throttling) while the load has barely varied, so it is usable before any load change has been seen.

Residual and level. A signal is judged under both hypotheses, $b = 0$ and $b = b_f$. For each, the window of the service's accepted readings gives a baseline $m^{(b)}$ (the median of $x_{s,f}(\tau) - b\,L(\tau)$) and a robust spread

$$\sigma^{(b)} = max\left(1.4826\,MAD, q_{0.9} - m^{(b)} / 1.2816, \phi_f \right), \tag{4.3}$$

the larger of the usual MAD spread and the distance to the 90th percentile in the direction that matters (both directions for traffic), with a floor ϕ_f at the smallest change worth noticing. The MAD alone treats a bursty signal as Gaussian and alarms on its ordinary bursts; the tail term keeps the alarm quiet (Section 6.1.1). The deviation is

$$z_{s,f}(t) = \min_{b \in \{0, b_f\}} \frac{\left| x_{s,f}(t) - b\,L(t) - m^{(b)} \right|}{\sigma^{(b)}}, \tag{4.4}$$

so a signal counts as unusual only if *neither* way of following the load explains it. When the load holds steady, both hypotheses agree and nothing is lost. Only increases of CPU, memory, throttling, errors and latency count, because less of them never signals a fault of

the service itself and follows every restart; traffic counts in both directions. The service's level is

$$level(s,t) = \max_{f} z_{s,f}(t) / z_{line}, z_{line} = 4, \tag{4.5}$$

so level 1 is the service's own alarm line, as before, and the hybrid alarm uses level 2 ($z = 8$). The signals split into *own* (CPU, memory, throttling), which a dependency cannot cause, and *symptom* (traffic, errors, latency, network), which a service can inherit; the ranker uses the two levels separately. The model keeps the strongest signal and its z, so a verdict reads "CPU 9.3 spreads from what the load predicts".

Gate and drift. Only readings under level 1 join the window of $W = 120$ readings, so a fault never becomes the baseline. A small steady shift (level below 3) becomes the new normal after two minutes (24 readings), a large one after thirty. That has a price: a mild fault lasting longer than two minutes gets absorbed and the model stops alarming on it (Section 6.1.2), while the rule layer and the container facts of the hybrid keep alarming. A service is judged only after 30 readings. The model needs no labels, installs nothing on the target and makes one pass over the readings the collector already gathers; a median and a sort per signal, refreshed every four readings, dominate its cost.

4.6.4 Why the first model failed, and what was compared

The first engine kept two Isolation Forests per service, trained on a sliding window of that service's readings. They learn the shape of recent readings, and a change of workload falls outside that shape. Stopping the traffic made every service alarm together, and restoring it did so again. The three other detectors of the study (k-NN, Mahalanobis, HBOS) and their ensemble fail the same way, because all of them model the readings themselves. We therefore compared families of model that treat the workload explicitly, all unsupervised, all computed from telemetry the mapper already collects:

- the old per-service forest, the ensemble of four detectors, k-NN, Mahalanobis and HBOS (*the workload-blind family*);
- a **control chart**: the robust residual model above without a load factor (MAD spread, or the tail spread);
- a **subspace residual** model: the standardised readings of all services together, the top one or two principal directions of recent healthy readings taken as the shared causes, and the residual off that subspace scored (the multivariate process-monitoring approach [10, 11]);

• the **load-aware residual model**: the control chart with the load factor, with either the MAD spread or the tail spread.

Every candidate runs inside the same engine (container facts, persistence, random walk, forecast) so that only the model of normal changes.

4.6.5 Rules and container facts

A robust one-sided z-score per metric (median and MAD, bad direction only) gives a rule score ρ_s normalised so that 1 is its three-sigma line. A container that is exited, paused, restarting, or network-detached is a fact: evidence, not degree. Two facts that raw container state misses were added after inspecting the real runs. A docker stop exits with code zero, which looks like a finished job, so a service that ran and then exits counts as *stopped*. A crash loop looks healthy between kills, so two outages within three minutes mark it.

4.6.6 The hybrid alarm

Let λ_s be a service's model level and $f_s \in \{0,1\}$ its fact flag. The hybrid alarms at window t if

$$\max_s f_s = 1 \text{ or } \max_s \lambda_s \geq \tau_\lambda \text{ or } \max_s \rho_s \geq \tau_\rho, \tag{4.6}$$

with $\tau_\lambda = 2$ and $\tau_\rho = 1.25$: the rules alone earn trust only well past their own line. In the live system, a fact raises an alarm immediately, and the other two conditions must hold for two consecutive windows. The offline evaluation demands two consecutive windows for all three, the stricter reading.

4.6.7 Deviation decomposition and the random walk

With own level $o(s)$, symptom level $\sigma(s)$ and callees $C(s)$,

$$\begin{aligned} x(s) &\leftarrow \gamma \max_{c \in C(s)} a(c), \\ raw(s) &\leftarrow \max\big(o(s), \max(0, \sigma(s) - x(s))\big), \end{aligned} \tag{4.9}$$

so a service whose symptoms a sick callee fully explains drops out, while one that is abnormal in its own resources stays. The ranker is a personalised-PageRank walk in the MonitorRank form [29]. A walker on the directed graph steps to a callee v with weight $w(u \to v)a(v)$, back to a caller with weight $\rho_b a(u)$, and stays with weight $\max\big(0, a(u) - \max_c a(c), o(u)\big)$, restarting on the anomalous set with probability $\alpha = 0.15$. The stationary share π_s is the score, and facts rank first. The stay term lets the source, whose trouble nothing downstream explains, collect the most mass.

4.6.8 The cascade model

Each call edge $c \to x$ carries an unknown probability $p(c \leftarrow x)$ that trouble in x reaches its caller c. Whenever a service x turns abnormal, each healthy caller c becomes one trial, and the trial succeeds if c turns abnormal within one minute. With k successes in n trials and a Beta prior of mean π_0 and strength m, the posterior mean is

$$\hat{p}(c \leftarrow x) = \frac{k + m\pi_0}{n + m}, \tag{4.7}$$

so a rarely exercised edge starts at its prior and moves with evidence. No labels are needed; the trials are the system's own history. All experiments use an uninformative prior ($\pi_0 = 0.5$, $m = 2$). Given a suspected root r, risk flows up the call graph by noisy-OR, the independent-cascade model [40]:

$$P(r) = 1, P(c) = 1 - \prod_{x \in C(c)} \left(1 - P(x)\hat{p}(c \leftarrow x)\right). \tag{4.8}$$

Services above a risk cut are reported as likely next, and $\sum_c P(c)$ estimates the blast radius. Three tools assess calibration: the Brier score $BS = \frac{1}{n}\sum_i (p_i - y_i)^2$ [41], the expected calibration error $ECE = \sum_b \frac{n_b}{n} \vee \acute{p}_b - \acute{y}_b \vee ¿$ over equal-width bins [43], and reliability diagrams.

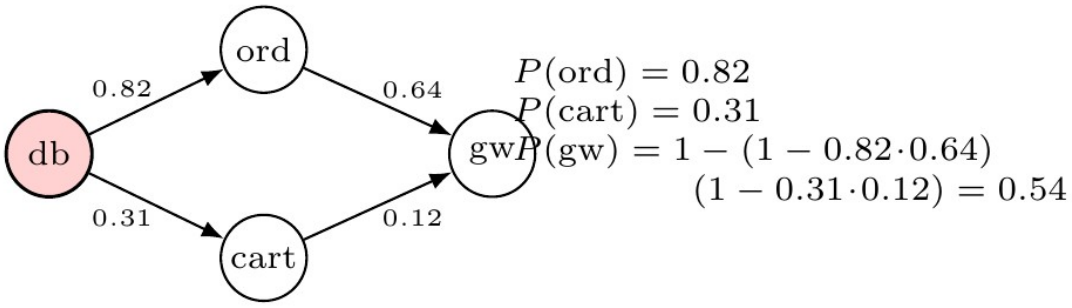


Figure 4.1: The cascade model on a four-service example. Edge labels are learned propagation probabilities $\hat{p}(c \leftarrow x)$; risk at the gateway combines its two routes by noisy-OR.

4.6.9 Early warning, change points and fault typing

For each rising-only feature, a least-squares line fitted to the load-aware residual z over the last two minutes and projected two minutes ahead (only lines with $R^2 \geq 0.6$ count) flags a service that has not reached the alarm level but will. Online Page–Hinkley change-point detection [44] on the raw metrics can also raise the alarm before the model's line is crossed. At the alarm, 35 features of the top suspect's telemetry (container-state history, exit code, number of separate outages, memory slope, CPU and throttling change, latency and error

ratios against a pre-fault baseline, network drop, number of anomalous callers) are input into a random forest [50] that names the fault type (crash, stop, pause, crash-loop, flap, memory, brownout, latency, partition). A short runbook of two to four concrete checks per type is attached. This layer changes neither detection nor ranking.

4.7 Error Sources and Mitigation

Table 4.3: Error sources and how each is handled.

Source of error	Mitigation
A socket is seen from both ends	Orient by the side that holds the listening port; fall back to the ephemeral-port range; mark unresolved pairs with lower confidence
Short-lived connections between samples	Count recently closed sockets (TIME_WAIT) as well as established ones
A stopped container exits with code 0	Treat “was running, now exited” as stopped
A crash loop looks healthy between kills	Two outages within three minutes mark a crash loop
Services differ by orders of magnitude	Per-service baselines; robust median and MAD
A fault becoming the baseline	Train only on readings below the alarm line; quarantine large shifts for thirty minutes
Restarts lower memory and CPU	One-sided features: only increases count as the service’s own fault
A loaded host answers slowly	Command timeouts sized for a loaded host; the check tool reports a host that cannot answer
Optimistic evaluation	Event-level scoring; held-out synthetic split scored once; bootstrap and paired tests; negative results reported
Ledger parsing	A regression test after a bug that hid half of the real faults (Section 4.10.6)

4.8 Data Processing and Analysis

Readings keep missing values as missing. Each recorded trace stores, per window, the readings, container facts and the engine inputs, so the engines can be replayed deterministically. Warm-up (the first five minutes of each run) is excluded from false-alarm and ROC counts. Intervals are 95% percentile bootstrap intervals over faults [54]. Comparisons between engines are *paired* bootstrap differences over the same faults with two-sided p values, and seed-averaged results are means over seeds. Overlapping faults within a run are not independent, so intervals are, if anything, optimistic. With 62 real faults, no difference below about 8 points is resolvable, and we claim none below that.

4.9 Computational Cost and Complexity

Table 4.4 states the cost of one decision window in terms of N services, E edges and K open sockets. We read the growth rates off the algorithms and checked them by timing

random graphs of 10 to 320 services (Section 6.1.12): about 0.37 ms per service per window, linear in N, on synthetic readings.

Table 4.4: Cost per decision window.

Step	Cost	Note
Orient sockets	$O(K)$	One pass over the socket rows of all containers
Load factor and slopes	$O(N F W)$	$F = 8$ signals, window $W = 120$; one median over the carrying signals, one ridge sum per slope
Score a reading	$O(N F)$	Two hypotheses per signal against cached baselines
Refresh the baselines	$O(N F W \log W)$ every 4 readings	A median and a sort per signal and hypothesis; independent per service
Rule scores	$O(N)$	Median and MAD over a short window
Random walk	$O(I(N+E))$	$I = 60$ iterations
Noisy-OR risk	$O(\text{sweeps} \cdot E)$	Four sweeps
Fusion and sorting	$O(N \log N)$	Ranking of suspects

4.10 Evaluation Design

4.10.1 Systems and traffic

Two containerised applications were built for the study. ShopFlow is a 13-service retail checkout system (HTTP fan-out, two PostgreSQL instances, two Redis caches, a NATS consumer). LedgerLine is a 12-service retail-banking transfer system (a saga orchestrator, a double-entry ledger, a third-party FX provider, a batch worker). Both ran for 217 minutes. Under open-loop traffic with a daily wave, bursts, and flash crowds, each scenario has its own EC2 host, Fault Lab, and Grafana (Section 5.10).

4.10.2 Blind fault injection

Faults arrive by a non-homogeneous Poisson process whose rate follows the load, each with a class, a role-weighted target, a heavy-tailed duration and a severity; with some probability, a follower is spawned in the target’s one-hop neighbourhood, so incidents overlap. Faults are physical (SIGKILL, pause, cgroup CPU and memory limits, network detach) and nothing is labelled inside the system under test. The injector writes a private ledger with, per fault, the true root cause and the services structurally able to be affected; this ledger is read only after the run. The two systems contribute 62 faults over seven classes (Figure 4.2).

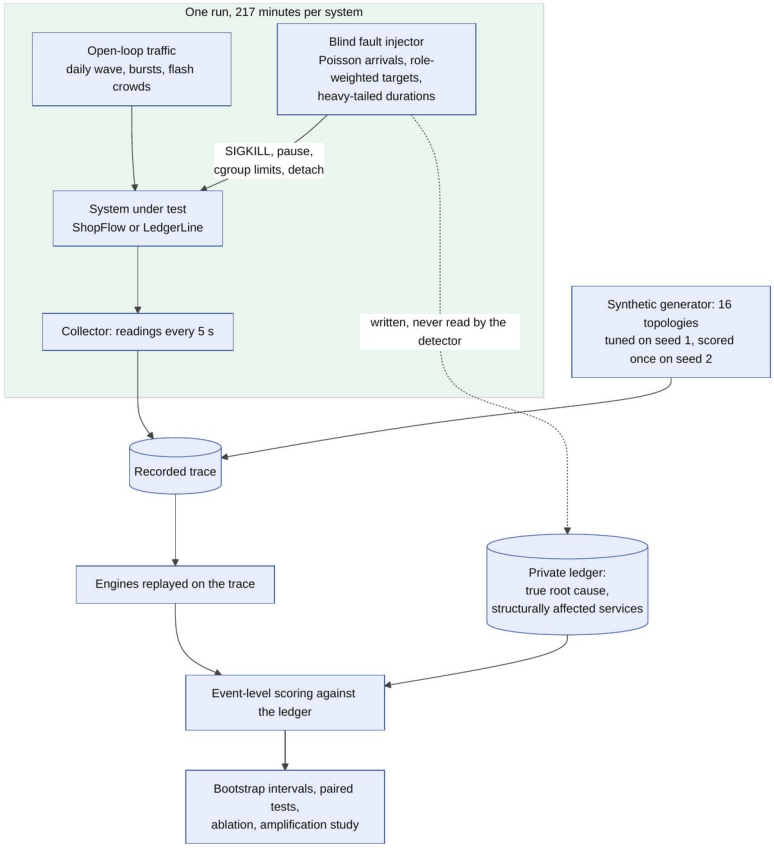


Figure 4.2: Evaluation method. The ledger is written by the injector and read only when scoring.

4.10.3 Synthetic benchmark

To enlarge the sample, a generator writes runs in the identical recorded format. It combines the two topologies with their measured per-service baselines, random topologies, nine fault classes that propagate up and down the graph, overlapping faults and unlabelled noise (noisy neighbours, deploys, missing readings). Settings were chosen on a 16-campaign tuning split (170 faults, seed 1), and every reported synthetic number comes from a disjoint 16-campaign split (178 faults, seed 2) scored once. The generator also takes an *amplification factor* A that scales how strongly callers feel a fault (retry storms) and a class filter; the amplification study rests on it.

4.10.4 Live runs and fault-free workload recordings

We added two kinds of recording to test the corrected engine. *Live campaigns:* both benches ran in Docker on one machine, the mapper reached them over SSH exactly as it reaches a remote host, and the Fault Lab replayed one plan (seed 11, 21 faults of four classes) four times (Section 6.2). The observer recorded what the mapper's API showed, with every service's metrics. *Workload recordings*: the same setup with *no fault at all* while the amount of work changed, either with the organic generator started and stopped or with a closed-loop generator holding $2, 8, 2, 0, 20, 2$ concurrent visitors for several minutes each.

Every alarm in these recordings is false by construction, so the measure is the share of time in alarm (Table 4.1). The engine knows neither the plan nor the phases.

4.10.5 Questions the evaluation answers

1. Does the hybrid name the true cause more often than the rule layer, and how does it compare with ranking by anomaly alone and with the ML engine?

2. Which model of normal separates faulted from healthy services best, and which one does not mistake a change of workload for a fault?

3. How much does each mechanism contribute (ablation), and does graph-aware ranking help when victims are louder than the cause (amplification)?

4. Do learned propagation probabilities predict cascades better than fixed ones, and are they calibrated?

5. Do learned directions remain correct when names carry no information, and do change-point alarms reduce detection delay?

4.10.6 Two measurement bugs

Each recorded run contains two fault campaigns that both number their faults from one. The ledger reader kept only the second, hiding 14 of 31 faults per system for every engine and counted their outages as false alarms. We fixed the reader, added a regression test, and now every number in this report uses all 62 faults.

The second bug hid the very failure this chapter corrects. The replay that feeds recorded runs to the engine skipped a service’s reading whenever it matched the previous one. An idle service reports the same zeros again and again, so the replay deleted exactly the idle stretches, and the old forest looked well behaved when traffic stopped. The live server skips nothing, because the collector reports every cycle. The replay now feeds every recorded moment as one reading, and we recomputed every number of the first version of this report that depended on it.

CHAPTER 5

IMPLEMENTATION & EXPERIMENTAL WORK

5.1 Implementation Environment

Table 5.1: Implementation environment.

Part	Technology
Language and runtime	TypeScript on Node.js 22 (server run with tsx); a small amount of JavaScript in the traffic generator; optional Python 3
Server	Express 5, ws for WebSocket, ssh2 for SSH (no ssh binary needed)
Dashboard	React 19, Vite 8, @xyflow/react for the 2D graph, Three.js with react-three-fiber for the 3D scene, Recharts, Framer Motion
Persistence	JSON files under server/data (graph state, remote configuration, per-target connection events); metric ring buffers in memory
Source size	About 18 400 lines of TypeScript in the server and 17 200 in the dashboard; 5 400 in the evaluation tools; 2 500 in the v2 studies
Tests	339 automated tests in 19 suites, run with Node's built-in test runner
Version control	Git, with GitHub as the remote

5.2 Module-wise Implementation

5.2.1 Connection and collection

The connection manager maintains one SSH session per target and runs commands from a fixed catalogue. The catalogue is a list of command builders; every argument that reaches a shell is sanitised (container identifiers must match the shapes Docker itself produces, timestamps and tail lengths are validated). Timeouts are sized for a loaded host: on a swap-thrashing 2 GB machine with eighteen JVM services, 'docker ps' was measured at 45 seconds, and a timeout shorter than that does not degrade gracefully; it makes the whole target disappear from the graph.

Each cycle, the collector asks for the container list, inspects containers it has not seen, reads statistics, and reads each container's socket table. Reading the table through the container's process id on the host (/proc/<pid>/net/tcp) works for every image, including images with no shell; before we switched to it, that gap had silently hidden one benchmark's front end for weeks. The same command returns the interface byte counters. They are exact, where docker stats prints values rounded to three significant figures.

5.2.2 Edge direction, weights and the graph

Every connection appears in the socket tables of both ends. Recording “scanned container calls peer” on both sides produced a reverse duplicate of every real link. The graph became effectively undirected, with a cycle on every pair, so “downstream of X” reached almost everything and blast radius and criticality saturated. We now recover direction from listening ports: the end that owns the listening port is the callee. The ephemeral port range is a fallback when a container’s listen table cannot be read, and an edge that neither rule settles stays marked unresolved, not given a coin flip dressed as a measurement.

Observed and declared edges are merged, and an observed edge expires ten minutes after it was last seen. A connection that is seen only as a recently closed socket (TIME_WAIT, which lingers about a minute) is counted once by its connection key, so a burst of short calls does not look like many links. A call made through the host’s address to a port that one service publishes is attributed to that service (a connection to a private or bridge-gateway address on a published port).

5.2.3 Container facts

The container runtime’s state becomes one of a few facts: down (exited with a non-zero code, dead or vanished), paused, restarting, or detached from its network while running. Repeated restarts in a short window raise a separate crash-loop flag. A load generator and a container that finished cleanly are *benign*, because people start and stop traffic, not faults.

5.2.4 The load factor and the load-aware level

Listing 5.1 reads the project’s load and the slope of each signal on it: the median shift of the signals that carry traffic, and a ridge estimate pulled towards the prior. Listing 5.2 judges one reading. It tries a signal under both hypotheses about how the signal follows the load and counts it as unusual only if neither explains it. The strongest signal sets the service’s level and stays as the evidence.

Listing 5.1: The load factor and its slopes (loadModel.ts).

```
    // 1. load factor: the median shift, over services and load signals
    const shifts: number[] = [];
    let contributing = 0;
    if (p.mode !== 'control') {
      for (const [id, x] of readings) {
        const r = refs.get(id); if (!r) continue;
        let used = false;
        // only a signal that carries traffic says anything about load: an idle database
reads the same zeros either way
        for (const f of p.loadFeatures) if (x[f] !== null && r[f] !== null && (r[f] as
number) - p.idle[f] >= p.active) { shifts.push((x[f] as number) - (r[f] as number));
used = true; }
        if (used) contributing++;
      }
```

```
    }
    const load = contributing >= p.minServices ? med(shifts) : 0;

    // 2. slopes: pooled over services and over the window (their stored loads against
their shifts from the reference)
    const slopes = p.prior.slice();
    if (p.mode === 'pooled' && contributing >= p.minServices) {
      for (let f = 0; f < slopes.length; f++) {
        if (!p.loadFeatures.includes(f) && p.prior[f] === 0) continue;
        let sxy = 0, sxx = 0;
        for (const [id, s] of this.svc) {
          const r = refs.get(id); if (!r || r[f] === null || (r[f] as number) -
p.idle[f] < p.active) continue;
          for (const e of s.win) if (e.x[f] !== null) { const d = (e.x[f] as number) -
(r[f] as number); sxy += d * e.load; sxx += e.load * e.load; }
        }
        slopes[f] = (sxy + p.ridge * p.prior[f]) / (sxx + p.ridge);
      }
    }
    const usedSlopes = p.mode === 'control' ? p.prior.map(() => 0) : slopes;
    const shifted = Math.abs(load) >= 1;
```

Listing 5.2: Judging a reading against what the load predicts (loadModel.ts).

```
      let top: Level['top'] = null;
      const zs = new Array<number>(NAMES.length).fill(0);
      if (ready) {
        for (let f = 0; f < NAMES.length; f++) {
          const xf = x[f];
          if (xf === null) continue;
          // The signal is judged under each hypothesis about how it follows the load:
not at all (b = 0), or by the slope the
          // project shows (b = slope). It counts as unusual only if NEITHER explains
it: a service that ignores the load
          // (a message bus idling at the same rate) and one that follows it (a gateway)
are both handled before either has
          // been seen to move. With the load unchanged, both hypotheses agree.
          const betas = usedSlopes[f] === 0 || p.mode === 'control' ? [0] : [0,
usedSlopes[f]];
          let zBest = Infinity;
          for (const b of betas) {
            const ck = `${f}|${b.toFixed(2)}`;
            let c = s.cache.get(ck);
            if (!c || this.tick - c.at >= p.refresh || Math.abs(c.n - s.win.length) >
20) {
              const rho = s.win.map((e) => (e.x[f] === null ? null : (e.x[f] as number)
- b * e.load)).filter((v): v is number => v !== null);
              if (rho.length < p.minTrain * 0.6) continue;
              const base0 = med(rho);
              let spread0 = mad(rho, base0);
              if (p.scale === 'tail') {
                const sorted = [...rho].sort((a, c2) => a - c2), q = (u: number) =>
sorted[Math.min(sorted.length - 1, Math.floor(u * (sorted.length - 1)))];
                const up = (q(0.9) - base0) / 1.2816, down = (base0 - q(0.1)) / 1.2816;
                spread0 = Math.max(spread0, p.oneSided && UPPER_ONLY.has(f) ? up :
Math.max(up, down));
              }
              c = { base: base0, spread: Math.max(spread0, p.floors[f]), at: this.tick,
n: s.win.length };
              s.cache.set(ck, c);
            }
            const base = c.base, spread = c.spread;
            let zb = ((xf - b * load) - base) / spread;
            if (p.oneSided && UPPER_ONLY.has(f)) zb = Math.max(0, zb);
            zBest = Math.min(zBest, Math.abs(zb));
          }
          if (!Number.isFinite(zBest)) continue;
          let z = zBest;
          zs[f] = z;
          const lv = z / p.zLine;
          if (lv > level) { level = lv; score = z; top = { feature: NAMES[f], z }; }
          if (OWN.includes(f)) own = Math.max(own, lv); else symptom = Math.max(symptom,
lv);
        }
```

5.2.5 Ranking by random walk

Listing 5.3 is the personalised random walk. The “stay” weight lets the true source keep the walker: a service whose trouble nothing downstream explains collects the most time.

Listing 5.3: Root cause by random walk (randomWalk.ts).

```
export function rootCauseWalk(ids: string[], a: Map<string, number>, edges: WalkEdge[],
p: Partial<WalkParams> = {}, own?: Map<string, number>): Map<string, number> {
  const { alpha, rho, iterations } = { ...WALK_DEFAULTS, ...p };
  const n = ids.length, index = new Map(ids.map((id, i) => [id, i]));
  const an = ids.map((id) => Math.max(0, Math.min(1, a.get(id) ?? 0)));
  const ao = ids.map((id) => Math.max(0, Math.min(1, own?.get(id) ?? 0)));
  const total = an.reduce((s, x) => s + x, 0);
  const out = new Map<string, number>(ids.map((id) => [id, 0]));
  if (!n || total <= 0) return out;

  const callees: { j: number; w: number; mutual: boolean }[][] = ids.map(() => []);
  const callers: number[][] = ids.map(() => []);
  for (const e of edges) {
    const i = index.get(e.from), j = index.get(e.to);
    if (i === undefined || j === undefined || i === j) continue;
    callees[i].push({ j, w: Math.max(0, e.weight), mutual: !!e.mutual });
    callers[j].push(i);
  }
  // transition rows
  const rows: { j: number; p: number }[][] = ids.map((_, i) => {
    const moves: { j: number; p: number }[] = [];
    let worstCallee = 0;
    // a link whose direction is unknown lets the walker move, but cannot explain this
service away
    for (const { j, w, mutual } of callees[i]) { moves.push({ j, p: w * an[j] }); if (!
mutual) worstCallee = Math.max(worstCallee, an[j]); }
    for (const k of callers[i]) moves.push({ j: k, p: rho * an[k] });
    // stay: trouble not explained by anything it calls, and its own (resource) trouble,
which nothing downstream can cause
    moves.push({ j: i, p: Math.max(0, an[i] - worstCallee, ao[i]) });
    const sum = moves.reduce((s, m) => s + m.p, 0);
    return sum > 0 ? moves.map((m) => ({ j: m.j, p: m.p / sum })) : [{ j: i, p: 1 }];
  });
  const restart = an.map((x) => x / total);
  let pi = [...restart];
  for (let it = 0; it < iterations; it++) {
    const next = restart.map((r) => alpha * r);
    for (let i = 0; i < n; i++) for (const m of rows[i]) next[m.j] += (1 - alpha) *
pi[i] * m.p;
    pi = next;
  }
  const s = pi.reduce((x, y) => x + y, 0) || 1;
  ids.forEach((id, i) => out.set(id, pi[i] / s));
  return out;
}
```

5.2.6 Learned propagation

The cascade model has two halves (Algorithm 5.2). While the system runs, it counts trials and successes per edge. When an incident is open, it spreads risk up the graph by noisy-OR from the suspected root.

5.2.7 Hybrid fusion and the three engines in parallel

Listing 5.4 fuses rules and ML. The default detection mode is or-strict: the alarm value is the larger of the ML level and the rule score divided by 1.25. Every cycle the incident manager records each engine’s verdict (alarm, peak against its own line, top five suspects),

whichever engine is active, and keeps the last 90 peaks for the Detection engines page. Only the active engine opens and closes incidents.

Listing 5.4: Fusing rules and ML (hybrid.ts).

```
export function fuseMoment(x: MomentInputs, p: FusionParams): Fused {
  const { detection, ranking, rulesAlone, mlWeight } = { ...HYBRID_DEFAULTS, ...p };
  const ids = x.services;
  const r = (id: string) => Math.max(0, x.rules.get(id) ?? 0), l = (id: string) =>
x.level.get(id) ?? 0;
  const maxR = Math.max(0, ...ids.map(r)), maxL = Math.max(0, ...ids.map(l));
  const fact = x.hard.size > 0 ? 10 : 0;

  let max: number;
  switch (detection) {
    case 'rules': max = maxR; break;
    case 'ml': max = maxL; break;
    case 'or': max = Math.max(maxR, maxL); break;
    case 'and': max = Math.max(Math.min(maxR, maxL), fact); break;
    case 'or-strict': max = Math.max(maxL, maxR / rulesAlone); break;
    case 'agree': max = Math.max(fact, ...ids.map((id) => Math.min(r(id), l(id))));
break;
  }

  const rankIn = (order: string[]) => new Map(order.map((id, i) => [id, i + 1]));
  const rulesOrder = [...ids].sort((a, b) => r(b) - r(a) || a.localeCompare(b));
  const topW = Math.max(1e-9, ...ids.map((id) => x.walk.get(id) ?? 0)), topR =
Math.max(1e-9, maxR);
  const blend = new Map(ids.map((id) => [id, mlWeight * ((x.walk.get(id) ?? 0) / topW) +
(1 - mlWeight) * (r(id) / topR)]));
  let score: (id: string) => number;
  if (ranking === 'rules') score = (id) => r(id);
  else if (ranking === 'ml') { const k = rankIn(x.mlRanked); score = (id) => -(k.get(id)
?? 1e6); }
  else if (ranking === 'rrf') { const a = rankIn(x.mlRanked), b = rankIn(rulesOrder);
score = (id) => 1 / (60 + (a.get(id) ?? ids.length)) + 1 / (60 + (b.get(id) ??
ids.length)); }
  else score = (id) => blend.get(id)!;
  const ranked = [...ids].sort((a, b) => (Number(x.hard.has(b)) - Number(x.hard.has(a)))
|| (score(b) - score(a)) || a.localeCompare(b));
  return { max, ranked, scores: ids.map((id) => blend.get(id)!), blend };
}
```

5.2.8 Incidents

The incident manager, once per target per cycle, runs the three engines' shared step, builds the evidence for the active engine, calls the root-cause engine, and either updates the open incident or opens a new one. A new incident takes the status DEGRADED or INCIDENT depending on whether any service is critical. It resolves when the alarm clears and stays in the history. Severity rises when more than a configured number of services are critical and falls for a service that nothing depends on and that depends on nothing, since it has no blast radius.

5.2.9 Traffic generation

The traffic controller starts and stops load through the traffic generator's HTTP API. Each application has a workflow file (a set of weighted user journeys with think times) and a set of profiles (baseline, moderate, heavy, stress, ramp) that fix the number of virtual users and their pace. An application without a file of its own gets a generic front-page workflow.

Before that change, the server refused every project outside five named ones with “no workflow defined”.

5.3 Algorithms

Algorithm 5.1 is one decision window and Algorithm 5.2 the learning and prediction step of the cascade model.

Algorithm 5.1: One decision window of the diagnosis assistant

Require: readings $x_s(t)$, container states, graph G with learned directions

Ensure: alarm, ranked causes, likely-next set, fault type, runbook

facts: mark each service stopped, paused, restarting, detached or crash-looping

score: model levels λ_s (own and symptom), rule scores ρ_s, anomalies a_s

alarm: apply the hybrid rule; if no alarm, update the propagation counts and stop

rank: services with a fact first, then by the walk share π_s over G

explain: report own versus symptom evidence and what the neighbours explain

predict: $P(c)$ by Equation. 4.9 from the top suspect; early-warning flags

diagnose: classify the fault type of the top suspect; attach its runbook

update $\hat{p}(c \leftarrow x)$ from services that newly turned abnormal

Algorithm 5.2: Learning propagation and predicting the next services

Require: set of services newly abnormal O, set of abnormal services A, call graph

Ensure: edge probabilities $\hat{p}$; risk $P(c)$ for healthy services

for each open trial (c, x) do

if $c \in A$ then record a success else ifthe window of 12 steps ran out record a failure end if

end for

for each $x \in O$ and each healthy caller c of x do open the trial (c, x) end for

$\hat{p}(c \leftarrow x) \leftarrow (k + m\pi_0)/(n + m)$ for every edge

for a suspected root r: $P(r)=1$; sweep the callers upwards, $P(c)=1-\prod_{x \in C(c)} \left(1-P(x)\,\hat{p}(c \leftarrow x)\right)$

drop services that are already abnormal; report the rest above the risk cut

5.4 Application Programming Interfaces

The server exposes 64 REST endpoints and one WebSocket. The endpoints fall into ten areas: the graph (topology, node detail, metric history), targets (registered hosts, discovery), reliability (cascade risk, what-if failure, blast radius), incidents (list, evidence, propagation, timeline), engines (active engine, per-engine verdicts, the switch), traffic (start, stop, status, entry points), experiments, the optional ML sidecar, configuration (hosts, keys, reset) and ingest (push from the optional standalone collector). The WebSocket carries graph-update (nodes, edges, targets, every 5 seconds), incident.updated, trace-events (the live connection feed) and the embedded terminal's data, so the dashboard never polls for graph state.

5.5 Configuration and Deployment

A few environment variables configure the server: PORT (3001), POLLING_INTERVAL_MS (5000), METRIC_HISTORY_SIZE (720 readings, one hour), INGEST_TOKEN (change it when others can reach the server), MAPPER_ENGINE (engine at start), MAPPER_BASELINE_MIN (30 minutes of baseline before a project's learned rule model applies) and MAPPER_AUTO_RESTART (off stops the mapper restarting crashed containers).

A host is added in the dashboard with its address, SSH user, and key path. The configuration lands in server/data/remote_config.json. Private keys never enter the repository, and *.pem is git-ignored as a backstop. npm start installs what is missing and starts the server and the dashboard, npm run doctor checks the machine, npm run hosts lists hosts and tests port 22, and npm run check-host and npm run mapper-edges diagnose a host and what the mapper holds for it (Section 5.11).

Table 5.2: Engine parameters (defaults).

Parameter	Value	Meaning
Training window	120 readings	About ten minutes of normal behaviour
Slow reference	360 readings	Median that shifts are measured from (half an hour)
Readings before judging	30	First baseline
Baseline refresh	every 4 readings	Median and spread per signal and hypothesis

Parameter	Value	Meaning
Signal units u_f	CPU 5, memory 1, network 2000, requests 0.1	Below about one unit a change is noise
Carrying signal	0.4 above idle	A signal that says something about the load
Slope prior / ridge weight	1 (network, requests), 0.4 (CPU) / 60	Load response before any load change has been seen
Alarm line z_{line}	4 robust spreads	Level 1; spread is the larger of MAD and the 90th-percentile distance
Alarm level / persistence	2 / 2 windows	Model condition of the hybrid alarm
Rules alone trusted at	1.25 × line	Rule condition of the hybrid alarm
Drift	24 or 360 readings	Mild shift accepted in two minutes; large one quarantined for thirty
Walk restart / iterations	0.15 / 60	Random walk
Propagation trial window	12 windows	One minute
Propagation prior	mean 0.5, weight 2	Uninformative
Forecast window / horizon / R^2	24 / 24 / 0.6	Two minutes fitted, two minutes ahead

5.6 Experimental Setup and Test Cases

The experiments run on the two benches of Section 5.10 (ShopFlow with 13 services and LedgerLine with 12, 217 minutes each, open-loop traffic, 62 blind faults), on a synthetic tuning split (16 campaigns, 170 faults, seed 1) and on a disjoint test split scored once (16 campaigns, 178 faults, seed 2). The collection period is 5 seconds, and Section 4.10 gives the design. Intervals are 95% percentile bootstrap, and engine contrasts use the paired bootstrap. Section 5.9 describes the test cases that hold the implementation.

5.7 Prototype and System Implementation

The figures below are screenshots of the running dashboard. The 2D pages and the Detection engines page come from the shop that the ground-truth simulator drives (10 services, 9 edges), which lets the same data appear with and without a fault. The 3D views come from the live ShopFlow bench.

5.7.1 Architecture view

The 2D view draws the dependency graph with a choice of arrangements (clusters, hierarchy, tiers, grid, circle, polygon, radial, spiral) and filters by service type. Nodes show CPU and memory against the container’s allowance; edges show the dependency and, when observed, its activity.

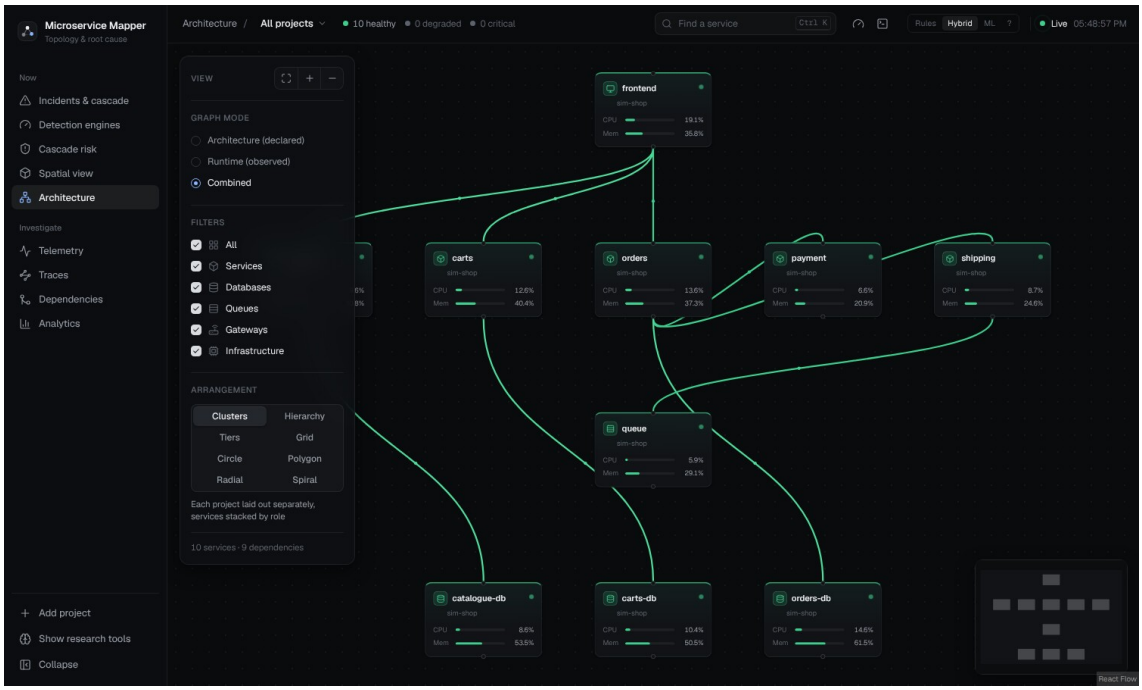


Figure 5.1: Architecture view: the dependency graph of the simulated shop, with arrangement and filter controls.

5.7.2 Incidents and cascade

When the alarm is raised, the page shows the most likely root cause, the confidence, the severity, and the number of affected services. The cascade panel below it shows the source, the services already affected (observed) and those next in line (predicted). The ranking lists each service with the strongest signal behind its score. The “Why” panel splits a score into its own-level, symptom-level and walk components and lists the evidence behind it, and the next steps give concrete commands for the host.

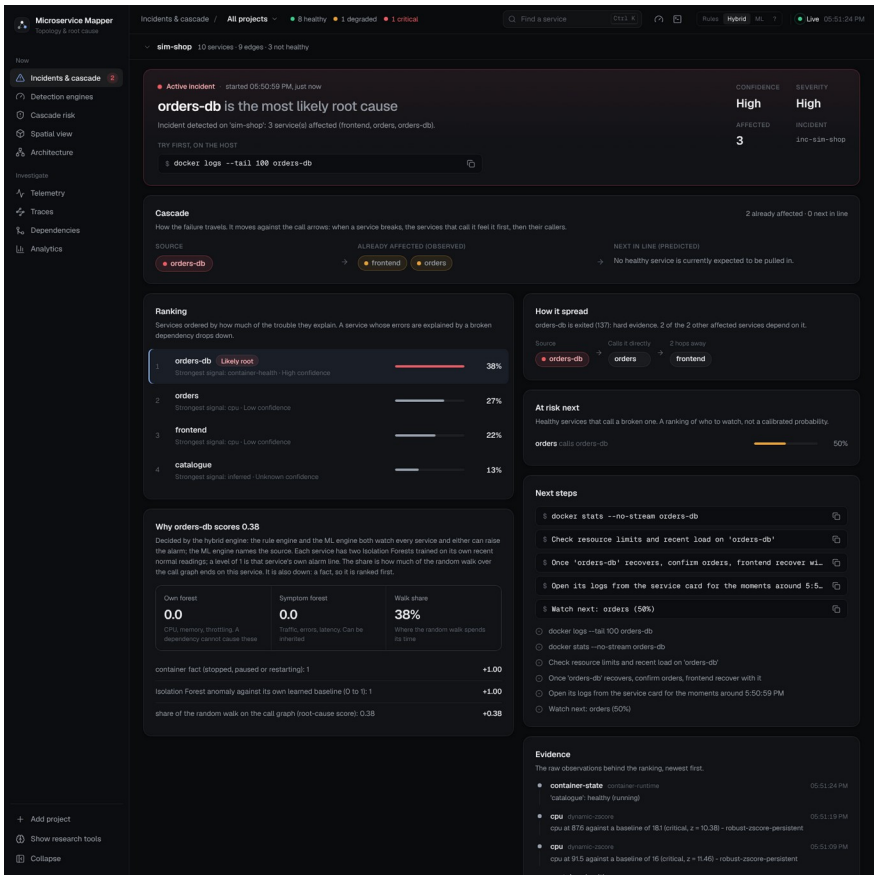


Figure 5.2: An active incident after orders-db was killed: root cause, cascade, ranking, evidence breakdown and next steps.

5.7.3 Detection engines

The Detection engines page (Figure 5.3) lets a developer check what the engine switch actually changes. The top half explains the three engines in four short lines each (what decides, how the source is named, the strength, the weakness) and states the Hybrid rule as a formula. Below it, the three engines run side by side on the same readings every cycle, whichever one is active. Each column names its top suspect, and the ML column also says *why*; in Figure 5.3 it reads “payment, cpu 10.3 spreads from what the load predicts”. A small chip beside the project name reports whether the shared workload moved (“workload steady” or “traffic ×2.4, explained, not a fault”), so a traffic swing visibly leaves before any service gets judged.

Six charts sit under the columns: each engine’s peak against its alarm line, a strip showing the cycles on which the engines agreed, the level of every service under every engine, a source-or-victim scatter (own resources against symptoms), the share of blame from the random walk, and a trail for one chosen service. Every chart has an enlarge button. The enlarged view (Figure 5.4) adds a time brush for zooming into a few cycles and closes with Esc. The page ends with the configuration of both engines in plain words.

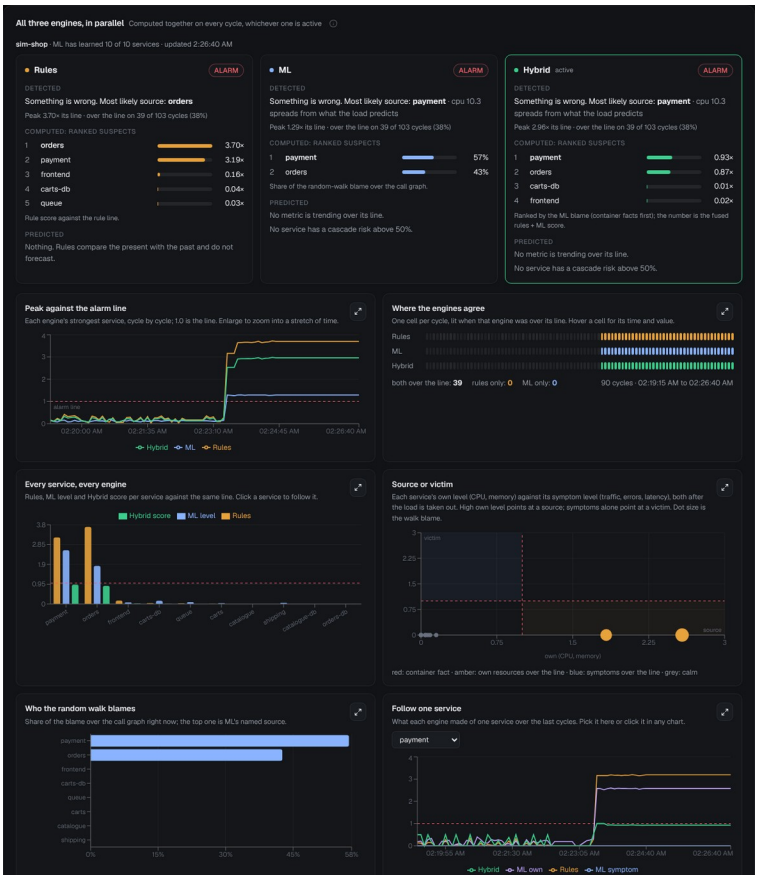


Figure 5.3: The parallel view during a slow-payment fault. Rules blame orders (the louder victim); ML and Hybrid name payment and say which signal moved. Below: peak against the line, agreement strip, every service, source or victim, walk blame and one service's trail.

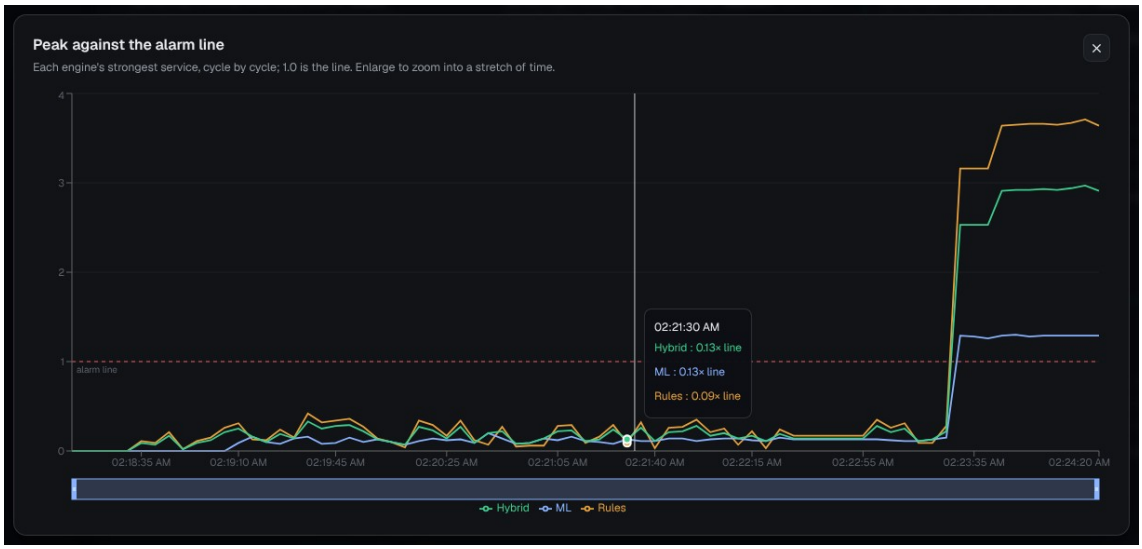


Figure 5.4: An enlarged chart with its time brush and hover read-out. The ML peak crosses the line once and stays just above it; the rule score climbs higher but on the wrong service.

5.7.4 Cascade risk

The cascade-risk page shows what is failing now and on its own measurements, what is exposed, and what would happen if a chosen service failed (what-if), with the model's name and parameters beside every prediction rather than bare numbers.

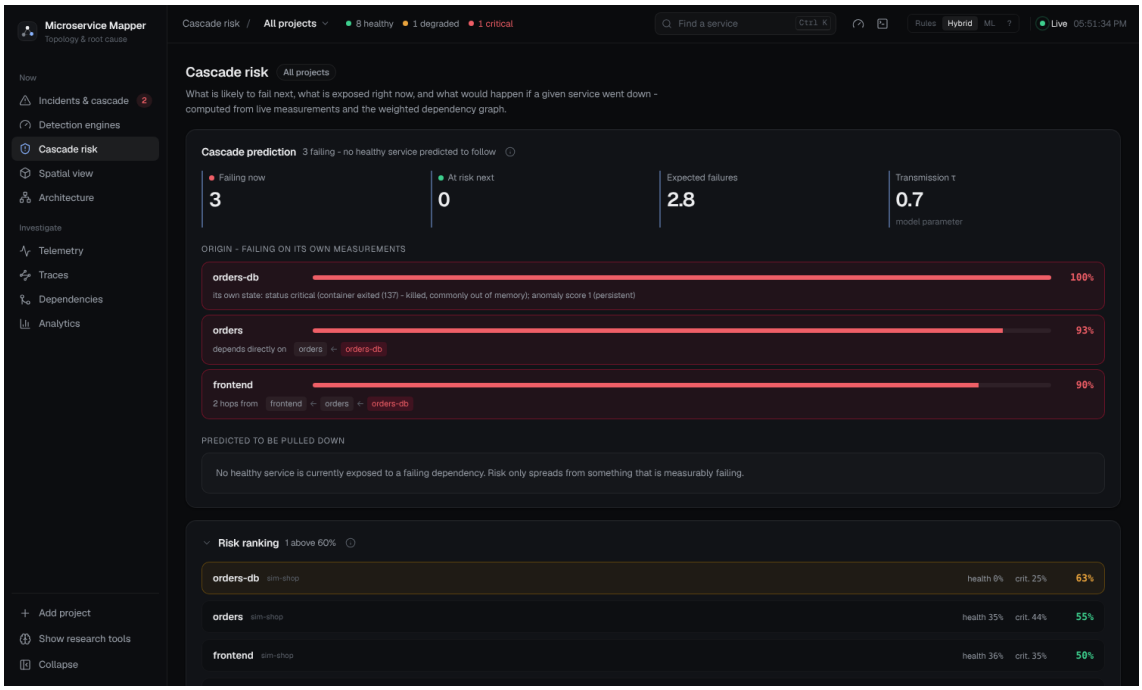
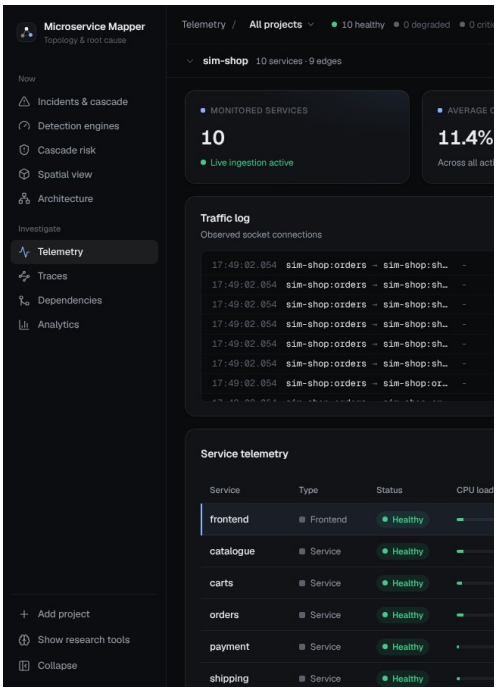


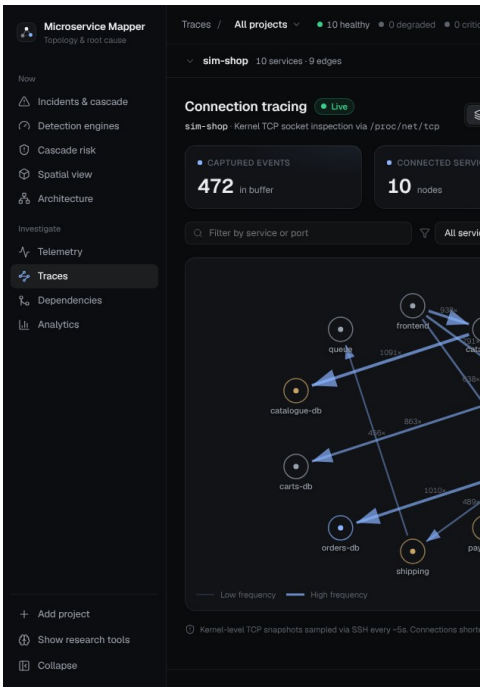
Figure 5.5: Cascade risk during the fault: the origin and the services failing on their own measurements, expected failures and transmission.

5.7.5 Telemetry, traces, dependencies and analytics

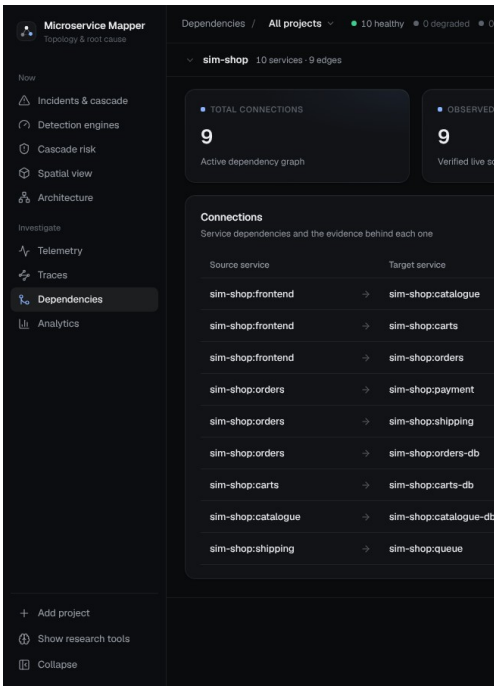
Four pages support the investigation (Figure 5.6). Telemetry: fleet totals, the live connection log and a sortable per-service table with a history chart. Traces shows connection tracing as a ring, a matrix and an event stream; these are sampled TCP sockets, not requests. Dependencies lists every connection with its declared and observed flags and its evidence sources. Analytics: system health index, counts of healthy, degraded and critical services, the most connected services and the top CPU and memory users.



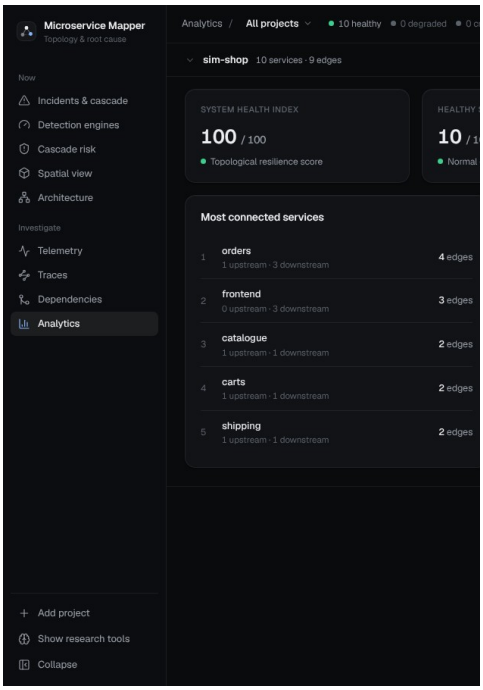
(a) Telemetry



(b) Traces



(c) Dependencies



(d) Analytics

Figure 5.6: The investigation pages: (a) telemetry with fleet totals and a per-service table, (b) connection traces as ring, matrix and event stream, (c) dependencies with their evidence (here *network-tcp* and *direction:listen-port*), (d) analytics.

5.7.6 The 3D spatial view

The 3D scene draws the same graph as a small city, and each arrangement answers a different question a developer asks while debugging. Services are units on platforms. Links are cables: thickness shows the dependency weight, colour shows the heat of the link, and pulses move at a rate driven by the link's request rate. A floating card over each platform gives its role, how many services it holds and their state, plus a line on what a fault there would mean (callers reached, services depending on it, calls in and out of the layer). The *Cards* button hides the cards, and the figures use that setting because the cards cover units and cables. We took them on the two live benches, each running on its own Docker daemon and registered as its own project, so each scene holds only that project's services.

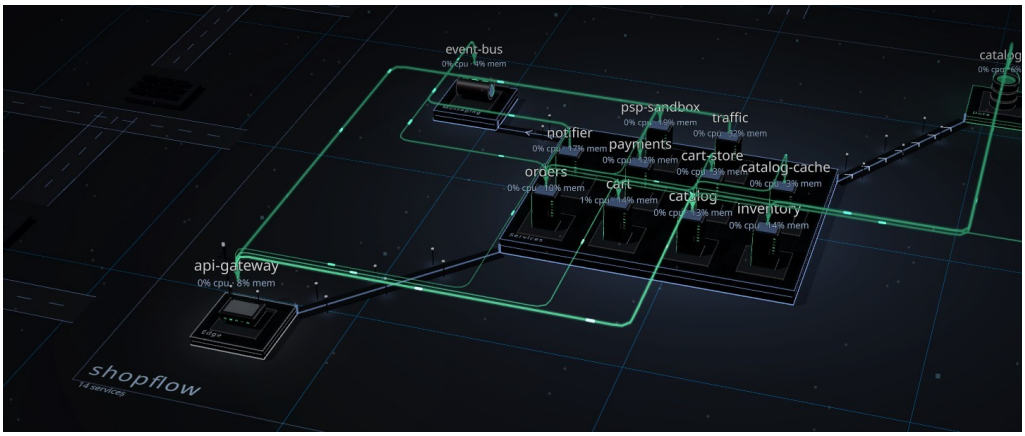
- Islands groups services by role (edge, services, messaging, data), each on its own raised platform. With every project shown (Figure 5.9a), each project is a district of its own.

- Flow lays the platforms out in the order a request travels: the entry points, then the services one hop away, two hops away and so on. Each card says how many callers a fault at that hop can reach, so the platform to look at first is the one whose card says the most.

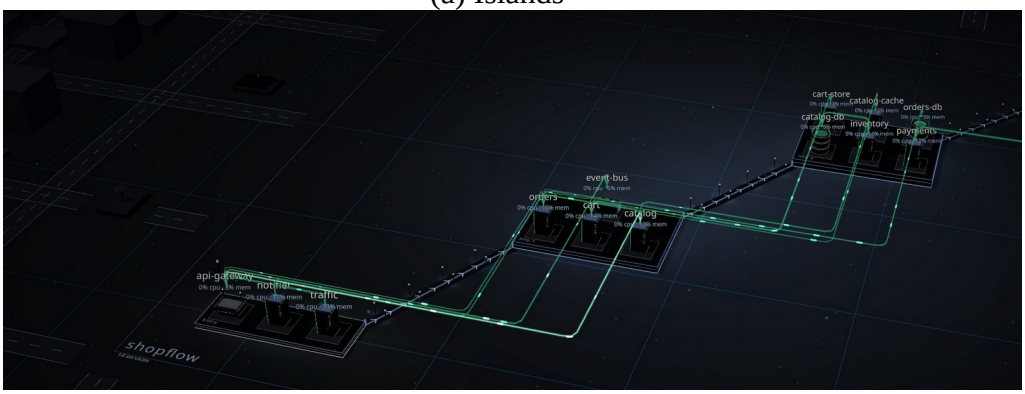
- Topology sorts services by how much depends on them: the most depended-on, the depended-on, and those nothing depends on. It is the blast-radius view.

• Tiers stacks the architectural layers as terraces from edge (L4) down to data (L1), with the calls in and out of each layer on its card, so a call that skips a layer or goes up the stack stands out.

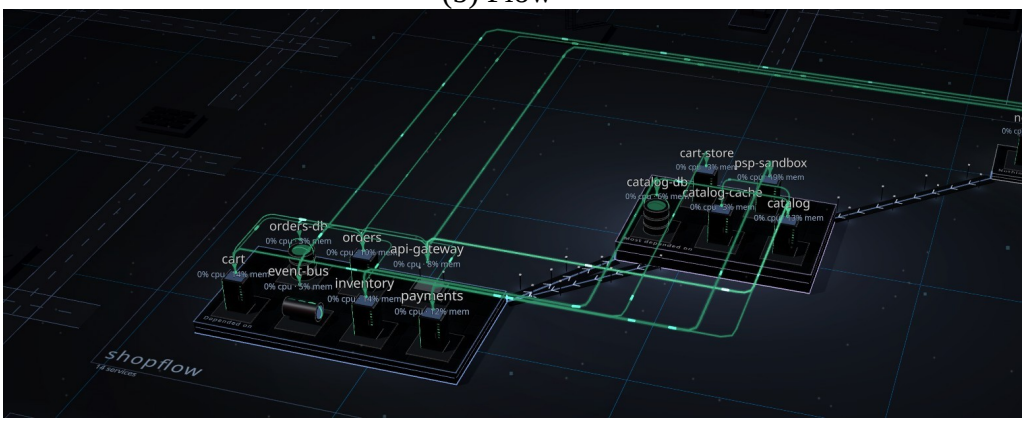
Figures 5.7 and 5.8 show the four arrangements for each bench. During a fault the failing unit gets a red box with a callout naming it and its score, its platform takes a tint, and the bell in the corner counts what needs attention (Figure 5.9b).



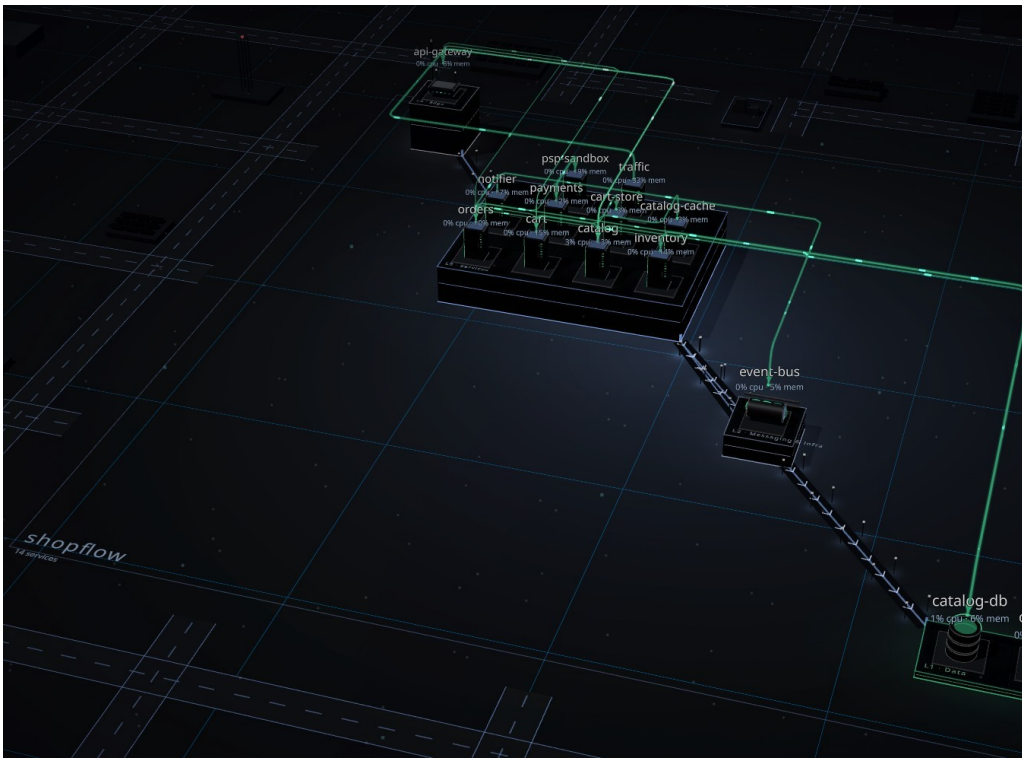
(a) Islands



(b) Flow

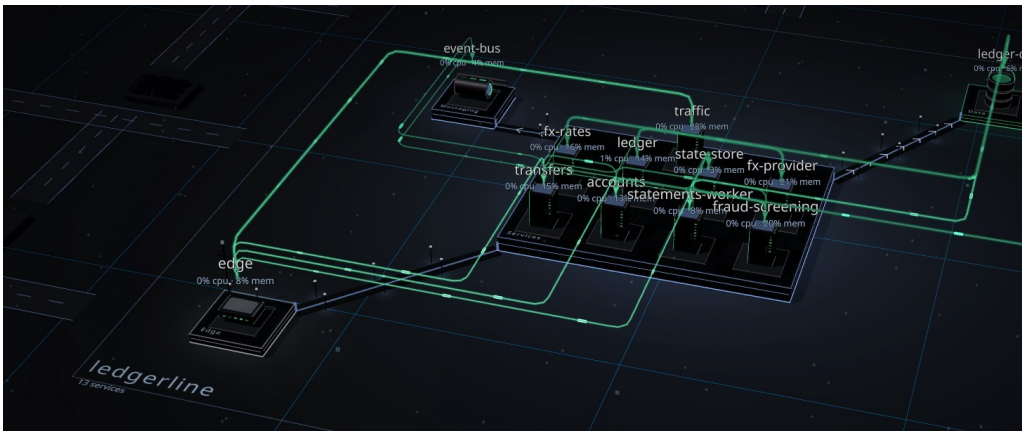


(c) Topology

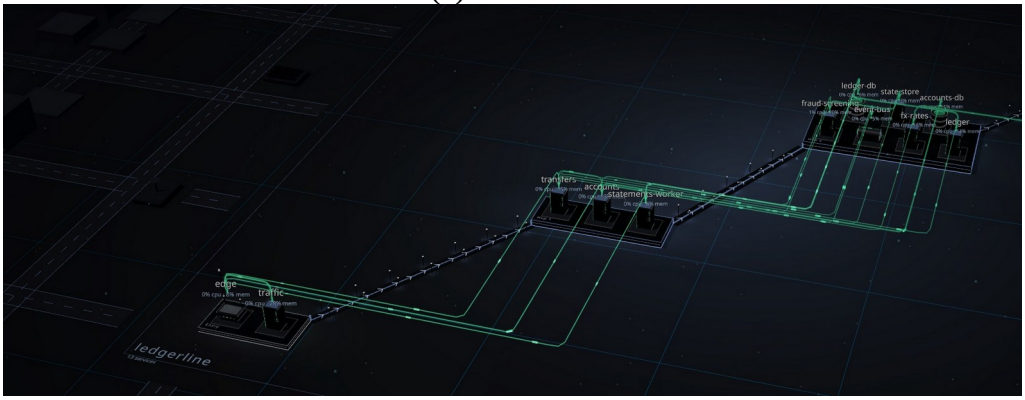


(d) Tiers

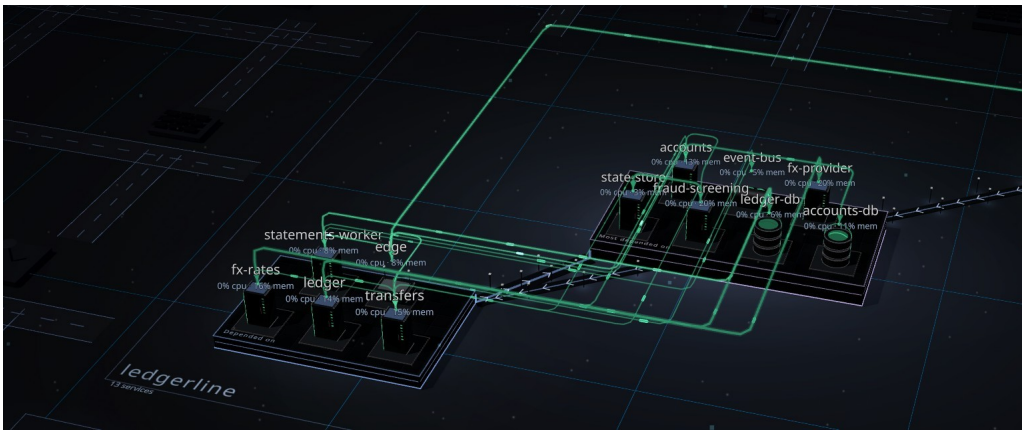
Figure 5.7: The live ShopFlow bench (13 services and its load generator) in the four 3D arrangements, shown without the section cards: (a) Islands, platforms by role; (b) Flow, platforms in request order from entry to data; (c) Topology, services grouped by how much depends on them; (d) Tiers, layers as terraces from edge (top) to data (bottom).



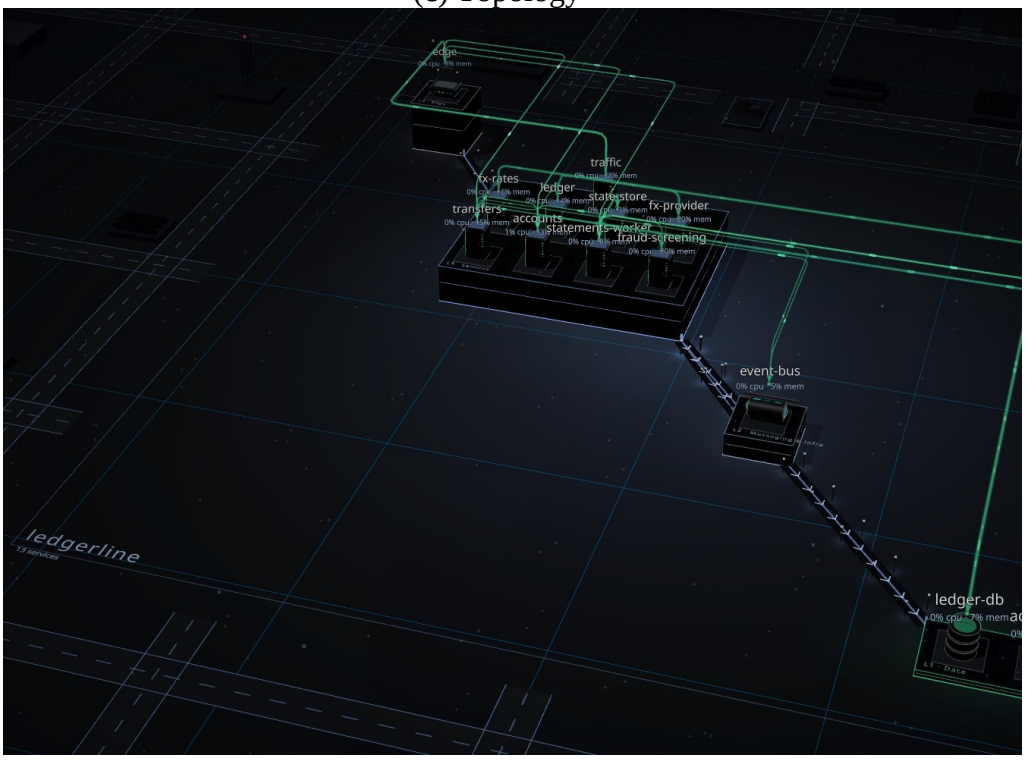
(a) Islands



(b) Flow

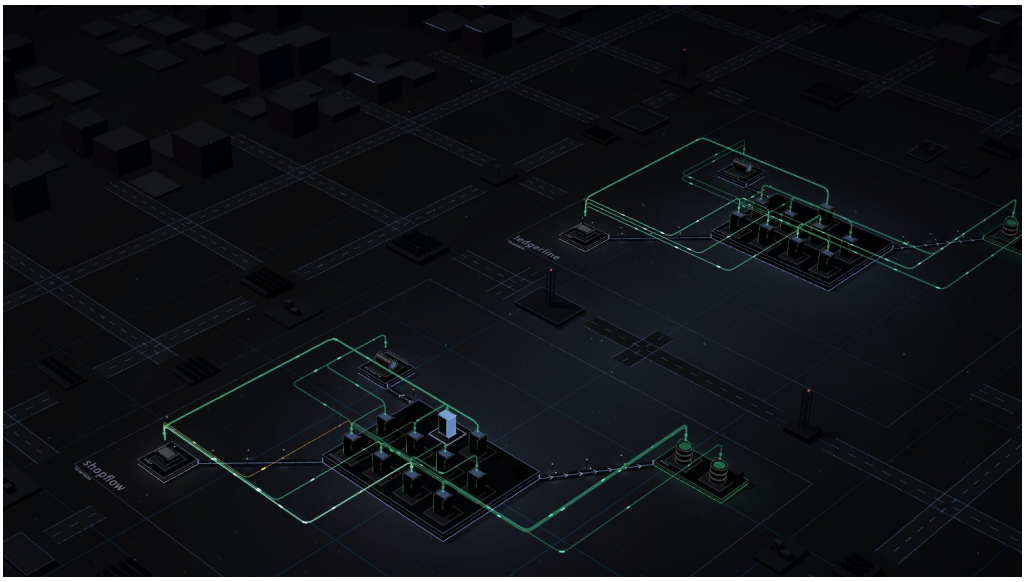


(c) Topology



(d) Tiers

Figure 5.8: The live LedgerLine bench (12 services and its load generator) in the four 3D arrangements, shown without the section cards: (a) Islands, platforms by role; (b) Flow, platforms in request order from entry to data; (c) Topology, services grouped by how much depends on them; (d) Tiers, layers as terraces from edge (top) to data (bottom).



(a) All projects



(b) ShopFlow during a fault: payments paused

Figure 5.9: (a) The Islands view with every project shown: each bench is a district with its own role platforms. (b) With payments paused, the unit is boxed in red with a callout naming it and its score, and its platform is tinted.

5.7.7 Research pages

Three pages that belong to the study rather than daily operation are hidden until the operator switches them on: the ML pipeline (what the hybrid engine sees per service), the model findings of the optional Python sidecar, and the experiment history.

5.8 Security Considerations

The tool maintains SSH access to hosts, so it was designed to expose as little as possible.

- Keys stay on the operator’s machine. The configuration stores only the path of the key file. Private keys are never copied into the repository, and *.pem files are git-ignored as a backstop.

- A fixed command catalogue. Every command the server can run on a host is built by a function in one file, and every argument that reaches a shell is checked against the shapes

Docker itself produces (container identifiers, timestamps, tail lengths, paths). A test ensures that an identifier with shell metacharacters is refused before a command is built.

- Bounded output and time. Each command has a timeout and a maximum output size, so a misbehaving host cannot make the server wait or buffer without limit.

- Local control surfaces. The Fault Lab panel listens on localhost only, and its requests need a header that the page itself sets. Grafana and Prometheus also listen on localhost and are reached through an SSH tunnel.

- Ingest token. The push endpoint is protected by a token. The built-in default is for local use and must be changed if the server can be reached by others.

- One write per host. The only change the mapper makes on a host is restarting a container that exited with a non-zero code, with a few attempts at most per container, never for load generators, and is switched off with MAPPER_AUTO_RESTART=off.

- Load limits. The traffic generator clamps the number of virtual users to a fixed ceiling and only starts from the traffic controller.

5.9 Testing and Validation

The suite has 339 tests in 19 suites and runs in under a minute with npm test; Table 5.3 groups them by area. Three kinds of tests deserve a note. Purity tests check that algorithm modules do not touch the filesystem or the network, so they can be replayed. Regression tests hold each bug that was found, for example, the ledger-reader bug and the tier layout. Security tests check that the command catalogue refuses unsafe identifiers.

Table 5.3: The test suites.

Suite(s)	What is held
logic, riskEngine, rca	Layouts (2D and 3D), edge heat scale, risk, cascade, what-if, root-cause scoring
ml-engine, ml, load-model, rca-research	Load-aware model (workload invariance, a fault seen during a load change), walk, propagation, hybrid fusion, learned directions, change points
pipeline, eval, purity	End-to-end replay, scoring against a ledger, purity of algorithm modules
telemetry-platform.* (eight files)	SSH connection, command catalogue, Docker parsers, CPU, units, service identity, storage, discovery, metric collector
sockets, requestStats, freshness	Socket direction and recently closed sockets, request statistics, staleness
security, platform	Command sanitising, portability across operating systems

Suite(s)	What is held
edge-cases, sparse-traffic, target-model	Missing data, idle services, per-target models

5.10 The Two Custom Benchmark Hosts: ShopFlow and LedgerLine

The two real systems of the evaluation are not borrowed demos. We built them for this study as *test benches for root-cause-analysis tooling*, each in its own repository and each running on its own EC2 host. Their code is correct and defensive: every failure comes from configuration, data volume, sizing, topology or a third party, and every failure ships with a machine-readable ground truth (the true root cause, what a monitor should and should not see, and the fix). Each bench has the same four parts: the application, a monitoring stack with Grafana, an always-on traffic generator, and a Fault Lab.

Table 5.4: The two benchmark hosts.

	ShopFlow	LedgerLine
Domain	Online retail checkout	Retail-banking transfers
Services	Gateway, catalogue, cart, inventory, payments, orders, an order-event consumer, a third-party payment sandbox; two PostgreSQL, two Redis, NATS (13 services)	Edge, accounts, transfers, fraud screening, FX rates, a double-entry ledger, a statements worker, a third-party FX provider; two PostgreSQL, Redis, NATS (12 services)
Public port	8080 (storefront and API)	8081 (gateway)
Host	EC2 c7i-flex.large, Ubuntu 24.04; about 0.6 GiB of containers	the same
Scenarios with ground truth	12 (one baseline control)	7 (one baseline control)
Monitoring	Prometheus, Grafana, Loki, Promtail; 25-panel dashboard	the same stack; 25-panel dashboard

5.10.1 Monitoring, traffic and the Fault Lab

Each bench carries a monitoring stack as a *separate Compose project*, which keeps the application project exactly what a mapper should discover. The stack holds Prometheus, Grafana with a provisioned 25-panel dashboard (Figure 5.10), Loki with Promtail, and a small exporter that reads per-container CPU, memory, network, restarts and health from the Docker API. Grafana gives the operator’s ordinary view of a healthy and a faulted bench, and an independent source for checking the mapper’s readings on the same time axis. Everything listens on localhost and reaches the outside through an SSH tunnel.

Each bench has an always-on, open-loop traffic generator. The default mode is organic: visitor sessions with think times, popular products, returning users, and a day-and-night

wave (about twelve workers); a steady mode gives a flat load. A quiet mode stops the generator so that only a person's own clicks reach the gateway.

The Fault Lab (lab/faultlab in each bench repository, Figure 5.11) is the evaluation's fault injector. The mapper never learns what is injected, and a private ledger, read only when scoring, records everything. Faults act on real containers (kill, repeated kills, stop, pause, resource limits, network detach, pause-and-release cycles). A campaign draws faults from a Poisson process with a profile, rate and seed, and one seed always gives the same campaign. The control page runs on the bench host, listens on localhost only and needs a header that only the page itself sets, so another site open in the same browser cannot drive it.

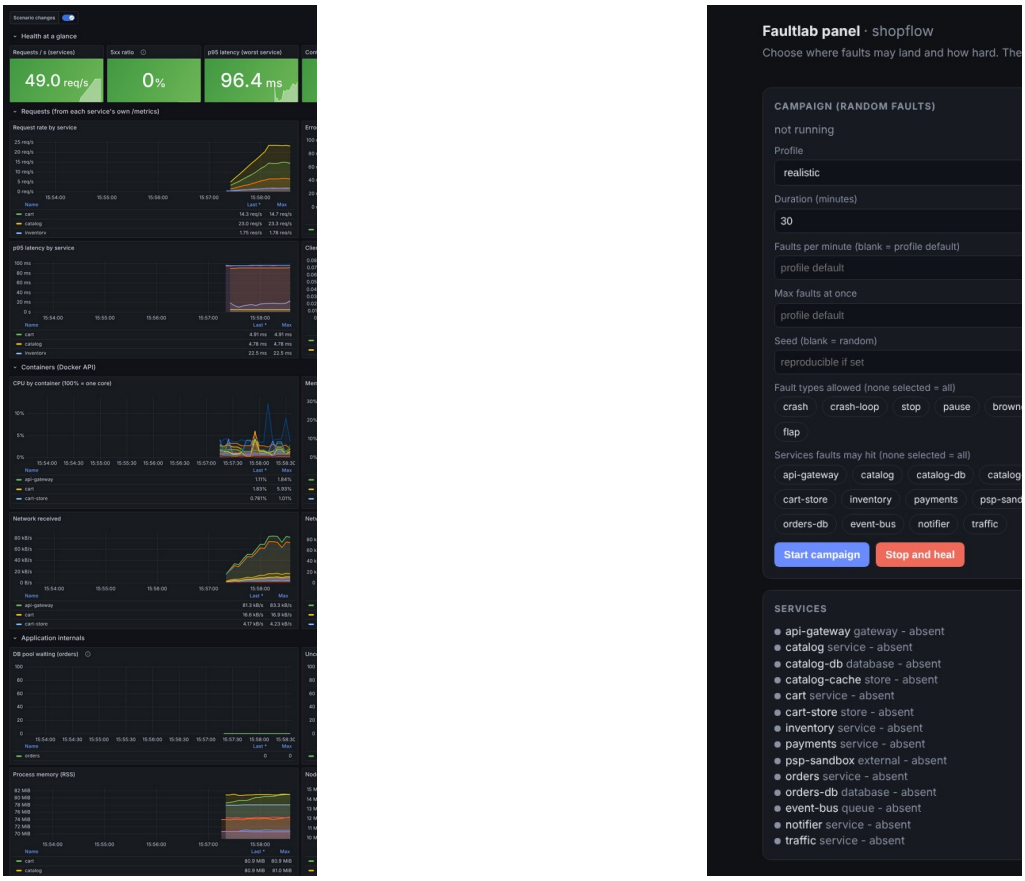


Figure 5.10: The Fault Lab control page of the ShopFlow bench: a random campaign, one fault by hand, traffic control and the service list.

5.10.2 Scenarios with ground truth

Besides random campaigns, each bench offers scripted scenarios. A scenario applies a defined cause, drives the load that triggers it, and writes a report next to the expected result in its scenario.yaml. Table 5.5 lists ShopFlow's; LedgerLine has seven more (a pool-starvation cycle between two healthy services, a third-party outage behind a circuit breaker, a hot row, a connection budget, a month-end job that runs out of memory, a shadow dependency, and a baseline). They cover cascades, retry amplification, silent degradation

and undocumented dependencies, the cases where a diagnosis tool most easily goes wrong. The reported numbers come from the random campaigns of Chapter 4.

Table 5.5: ShopFlow scenarios and their true root causes.

Scenario	What happens	Root cause
sf-01-missing-index	Order history is fast in staging, melts the database in production	orders-db
sf-02-cache-thrash	A cache sized for staging evicts itself in production	catalog-cache
sf-03-unbounded-cache	Memory climbs for an hour, then the container is killed	cart
sf-04-retry-storm	A third party throttles; our own retries make it ten times worse	psp-sandbox
sf-05-secret-rotation	A rotated database password; one service still has the old one	orders
sf-06/07	A dependency that exists only in the code; a declared one nobody calls	none
sf-08-noisy-neighbor	A reporting job on the shared database slows checkout	catalog-db
sf-09-cpu-throttle	Slow but healthy: a container throttled at a quarter of a core	catalog
sf-10-poison-message	One un-processable message stops every confirmation e-mail	notifier
sf-11-wrong-hostname	A renamed service still referenced by its old name	none

5.11 Host and Mapper Diagnostic Tools

Operating the benches showed that “nothing appears in the dashboard” has two very different causes: the host cannot answer, or the mapper has a problem. **npm run check-host** walks one instance step by step and stops at the first failure (port 22, key permissions, login, memory and disk, whether docker answers within 25 seconds, whether the application answers, which peers each front end calls). **npm run mapper-edges** prints what the running mapper holds for a project: the target’s status, its nodes, and every edge with its evidence and last-seen time. Together they tell a host problem from a mapper problem in minutes.

5.12 The Evaluation Harness

Table 5.6: The evaluation harness and the experiments built on it.

Command or folder	Purpose
npm run eval	Record a fault-injection campaign and score the live mapper (observe, score, report)
npm run ml:compare	Compare detectors and root-cause methods on the recorded runs

Command or folder	Purpose
npm run ml:hybrid	Fusion variants of rules and ML
npm run ml:predict	Early warning and cascade prediction
npm run ml:synth	Synthetic generator (amplification factor, class filter)
npm run ml:bench	Cost of one decision window
v2/	Studies: learned directions, change-point onset, fault diagnosis
paper/exp	Curves, amplification, cascade, router and scaling data for the figures
run/recorded-runs	The two recorded runs

CHAPTER 6

RESULTS, ANALYSIS AND DISCUSSION

6.1 Results

6.1.1 A change of workload is not a fault: the ML correction

The first ML engine judged each service with two Isolation Forests fitted on that service's recent readings. On the benchmark faults it looked fine, but an operator who stopped the traffic generator and switched to the ML engine saw every service with an anomaly score of 100%. We reproduced it on recordings made for the purpose. Both benches ran live in Docker, reached by the mapper over SSH, with *no fault at all*; only the work changed (run/scripts/workload-shift-experiment.sh and workload-steps-experiment.sh). In the first pair of recordings (two benches, 62 minutes) the organic traffic generator ran, stopped, ran, stopped longer and ran again. In the second pair (two benches, 72 minutes) a closed-loop generator held a constant number of visitors and stepped it $2 \rightarrow 8 \rightarrow 2 \rightarrow 0 \rightarrow 20 \rightarrow 2$, which moved the entry service's network traffic between 0 and about 3×10^5 B/s (Figure 6.1a). Every alarm in these recordings is a false alarm by construction.

The forest was in alarm for 23% of the organic recording and for 52% of the steady-load recording (Figure 6.1b, Table 6.1). The other workload-blind detectors do the same or worse, from 38% to 75% on the organic recording and from 76% to 96% on the steady one, because all of them model the readings themselves and a new workload is outside what they were fitted on. It is not a tuning matter. A forest scores a value beyond its training range no higher than its most extreme training point, so the old engine had added a range guard. That guard fires at once for every service when the traffic changes for all of them, and a large shift needed thirty minutes to be accepted as the new normal. On the fault-free stretches of the live fault campaigns the same engine raised 40.9 alarm episodes per hour.

The corrected engine (Section 4.6.3) does not alarm on either recording (0% and 0% of the time; one false episode in 72 minutes of steady-load steps, none in 62 minutes of organic traffic). Figure 6.1c shows the load factor it read: it follows the steps of Figure 6.1a. It rises to about $+2$ at the twenty-visitor step (traffic about seven times the reference) and falls to about -1.2 when the traffic stops (the log scale saturates at the network unit). That is how the shift gets explained instead of judged. The model finds the change by itself: no setting names the load generator, and nothing is installed on the hosts.

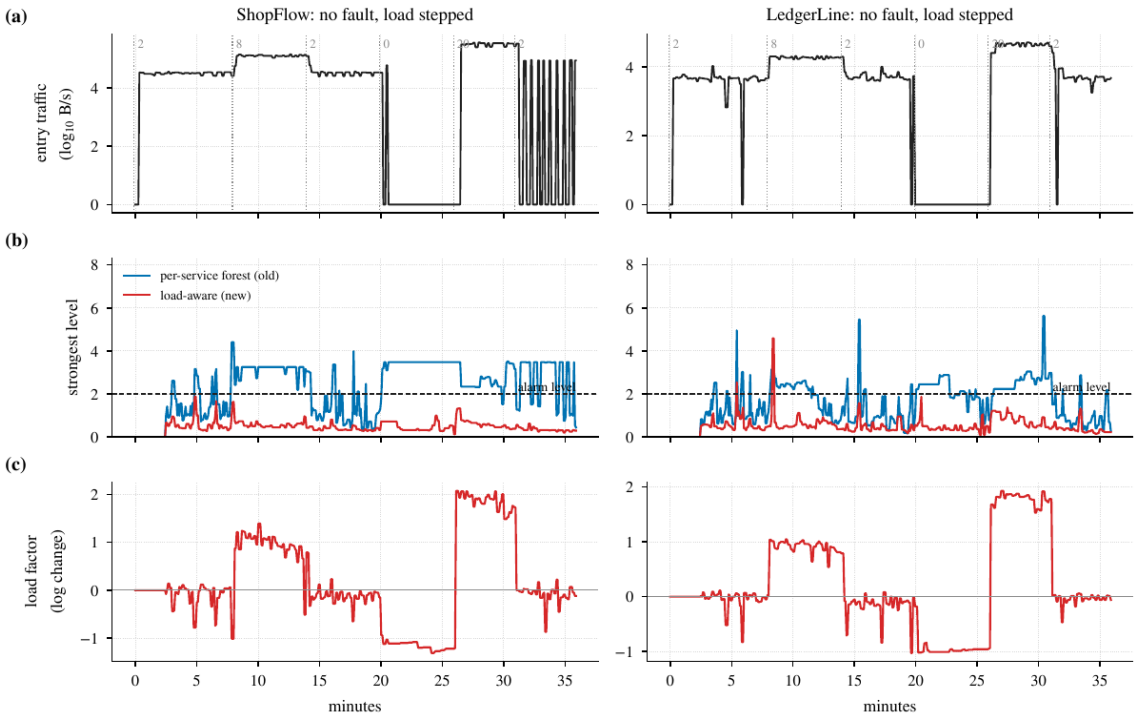


Figure 6.1: Two fault-free recordings with the load stepped (left ShopFlow, right LedgerLine; dotted lines mark a new level of concurrent visitors). (a) Network traffic into the entry service. (b) The strongest service level of the old per-service forest and of the load-aware model; the dashed line is the alarm level. (c) The load factor $L(t)$ the new model read, as a logarithmic change from its reference.

6.1.2 Which model of normal

Every candidate of Section 4.6.4 runs inside the same engine and is judged on four kinds of recording (Table 6.1, Figure 6.2): the 62 real faults, the 84 faults of the live Fault Lab campaigns (Section 6.2), 125 synthetic faults, and the two fault-free workload recordings. The replay feeds every recorded moment to the engine as one reading, as the live server does; the earlier replay dropped consecutive identical readings, which removed the idle stretches from the data (Section 4.10.6).

Table 6.1: Models of normal compared inside the same engine. For each set of injected faults: share of faults caught with the true cause first (C+N, %) and false-alarm episodes per fault-free hour. Last two columns: share of the time in alarm in the fault-free recordings of organic traffic and of steady-load steps (%); every alarm there is false.

	Real (62)		**Live (84)**		**Synthetic (125)**		**No faults, load moves**	
Model	C+N	FA/h	C+N	FA/h	C+N	FA/h	organic	steps
Rule engine (z-score, no model)	63	9.2	45	16.3	75	2.6	12	35
Isolation Forest (old default)	69	4.6	46	40.9	76	0.2	23	52
Ensemble of four detectors	71	20.0	44	81.7	78	2.6	47	88
k-NN	69	13.8	44	57.2	72	5.2	73	96
Mahalanobis	71	23.8	45	65.4	84	2.6	38	76
HBOS	69	34.6	44	57.2	77	1.4	75	82
Control chart, MAD spread	73	10.0	44	24.5	79	0.6	13	31

	Real (62)		Live (84)		Synthetic (125)		No faults, load moves	
Control chart, tail spread	71	2.3	46	0.0	78	0.2	0	0
Subspace residual, 1 component	68	3.1	50	8.2	78	0.4	1	3
Subspace residual, 2 components	68	3.8	50	0.0	77	0.2	1	4
Load factor, MAD spread	71	3.8	43	16.3	79	0.6	10	6
Load factor, tail spread (adopted)	71	2.3	48	0.0	78	0.2	0	0

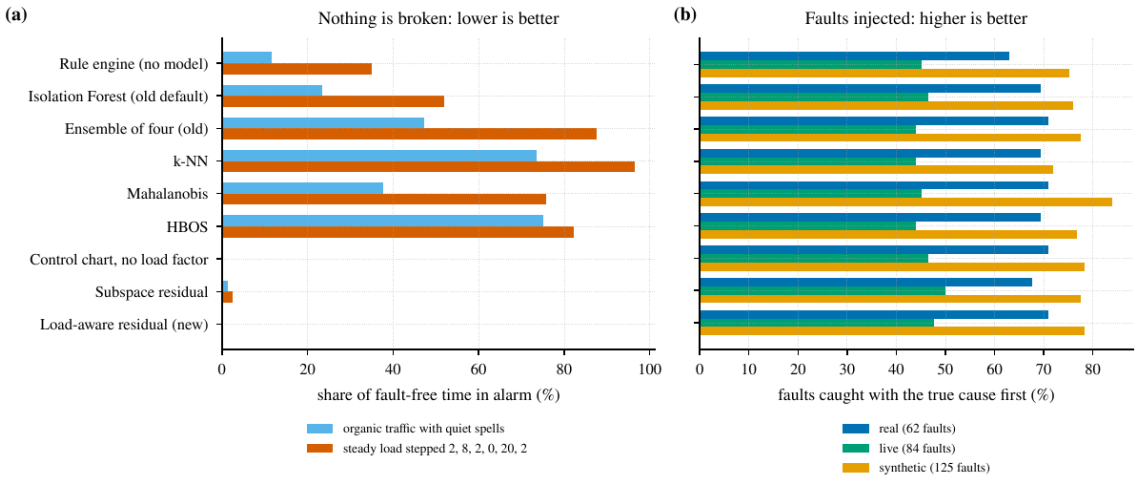


Figure 6.2: The candidate models. (a) Share of fault-free time in alarm on the two workload recordings (lower is better). (b) Share of injected faults caught with the true cause first on the real, live and synthetic sets (higher is better).

1. On injected faults the models are alike. The caught-and-named rate is 68–73 % on the real faults, 43–50 % on the live ones and 72–84 % on the synthetic ones for every candidate; differences of a few points are inside the noise of 62 to 125 faults (the intervals of Table 6.3 are ±10 points). The workload-blind models differ in false alarms: the forest-family detectors raise 4.6 to 34.6 episodes per hour on the real systems and 40.9 to 81.7 on the live ones, against 2.3 and 0.0 for the adopted model.

2. On workload shifts they are not alike. The adopted model is in alarm for 0% of both recordings; the workload-blind family for 23–75% and 52–96%. Replacing the forests by a robust residual with the tail-aware spread is the larger part of the improvement (control chart: 0% and 0%); the MAD spread alone is not enough (13% and 31%), because it treats bursty signals as Gaussian.

3. The load factor removes what is left. The control chart without it still raises 1.2 and 1.9 false episodes per hour on the two recordings; with it 0.0 and 1.0, and it is the better of the two on the live faults (47.6% against 46.4% caught and named). The subspace residual is the strongest alternative (3% and 4% of the steady recording in alarm; 50% caught and

named on the live faults). It raises 7.7 to 10.7 false episodes per hour on the steady steps, is slower to adapt, and its cost grows faster than linearly (Section 6.1.12).

We therefore adopted the load factor with the tail spread. We report what it does not do: it is not more accurate than the forest on the injected faults (71% against 69% on the real ones, inside the noise). A fault that begins during a large change of load can also be masked, because the model then accepts a deviation that either hypothesis about the load explains. Brownouts of nearly idle services remain the open problem for every model (no signal moves), as in the earlier version.

Sensitivity. Figure 6.3 sweeps the alarm line. A looser line (2 spreads) raises detection to 94% on the real faults but also the false alarms on the workload recordings (15.0 episodes per hour); a tighter one (6 or 8 spreads) removes all false alarms on the workload recordings but loses detection (87% and 85% on the real faults, 83% and 75% on the synthetic ones). The adopted line of 4 spreads is the loosest that keeps the workload recordings quiet. The training window (60, 120 or 240 readings) and the slow reference (120, 360 or 720) matter little once they are long enough; a window of 60 readings makes the workload recordings noisy again (10.4 false episodes per hour). We also tried carrying the range guard of the first model over to the residual (a value several floors beyond everything in the window counts as level one): it changed no result on any set of injected faults and added false alarms on the steady-load steps (4.8 episodes per hour at four floors, 1.0 without it), so the adopted model has none.

How fast a mild shift becomes normal. A deviation that stays between level one and three is accepted as the new normal after 24 readings (two minutes). Waiting longer, 48 to 240 readings, changed nothing on the injected faults (C+N 71 % on the real ones and 78 % on the synthetic ones at every value; synthetic detection 88 % against 89–90 %) and nothing on the workload recordings, but doubled the time in alarm on the real systems (6 % to 13 %) and raised their false alarms from 2.3 to 3.1 per hour, because a service whose footprint changes after a restart stays in alarm until it is accepted. The cost of the short setting is that a slow fault that stays only mildly above the line is absorbed after two minutes, as in the simulated slow payment of Section 6.1.14; a larger shift (level three or more) still needs thirty minutes.

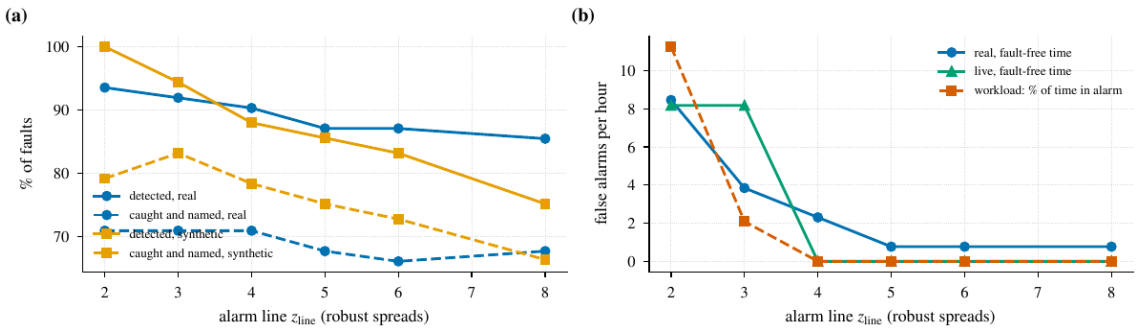


Figure 6.3: Sensitivity of the load-aware model to its alarm line z_{line} (the engine alarms at level 2, that is, at $2\,z_{line}$ spreads). (a) Faults detected, and caught and named, on the real and synthetic sets. (b) False alarms per fault-free hour on the real and live sets, and the share of time in alarm on the workload recordings.

6.1.3 Rules, the old ML and the new ML side by side

Table 6.2 puts the three engines a user can pick in the top bar next to each other. The same harness replays them over the same recordings: the rule engine (robust z-score per metric, alarm at three spreads, no model), the first ML engine (per-service Isolation Forests) and the corrected ML engine (load-aware residual). Both ML engines name the source with the same random walk, so any difference between them comes from the model of normal. A longer version with the reasoning in plain words is kept in the repository as wiki/44-ml-comparison.md.

Table 6.2: The three engines on the same recordings. C+N: faults caught with the true cause named first (%). FA/h: false-alarm episodes per fault-free hour. Workload columns: share of the fault-free workload recordings spent in alarm (%). ROC: per-reading AUROC and average precision on the real run and the held-out synthetic set. Cost: mean milliseconds per decision window for 10 and 320 services.

	Rules	**Old ML (forest)**	**New ML (load-aware)**
Injected faults			
Real, 62 faults: C+N / FA/h	63% / 9.2	69% / 4.6	**71% / 2.3**
Live, 84 faults: C+N / FA/h	45% / 16.3	46% / 40.9	**48% / 0.0**
Synthetic, 125 faults: C+N / FA/h	75% / 2.6	76% / 0.2	**78% / 0.2**
Detected, real / live / synthetic	90% / 96% / 83%	87% / 99% / 87%	90% / 99% / 88%
Median delay, real	15 s	12 s	12 s
No fault, only the workload moves			
Organic on/off: time in alarm	12%	23%	**0%**
Steps 2–8–2–0–20–2: time in alarm	35%	52%	**0%**
Per-reading ranking			
AUROC / AP, real	0.69 / 0.34	0.63 / 0.29	0.66 / 0.31
AUROC / AP, synthetic test	0.87 / 0.67	0.86 / 0.53	0.84 / 0.66
Cost, 10 / 320 services (ms)	–	8.9 / 296	4.0 / 119

The table reads in three parts. On injected faults the rule engine catches about as many faults as the ML engines. With container facts ranked first it names the right service for 63% of the real faults, against 69% for the old ML and 71% for the new one. It also raises four times as many false alarms as the new engine on the real run and more than ten times as many on the synthetic set. Every swing of traffic is a deviation to it, and when no container fact is present the loudest service, often a victim, gets the blame. The two ML engines are close on the injected faults; the new one is two points ahead on each set, with fewer false alarms, mostly on the live campaigns, where restarts and traffic pauses are frequent. On the workload recordings the difference is the whole point of the correction: the old ML is in alarm for about half of the steady-load recording, the rule engine for a third of it, and the new ML not at all. Per reading, the rule z-score ranks best on both sets; a single high reading is cheap, and the rule engine loses only once alarms need persistence and a named source. Between the ML engines, the new one has the higher average precision on the held-out set and a slightly lower AUROC, and it is the cheaper of the two.

The new engine differs from the old one in five ways. Before judging any service, it estimates how much the whole project’s workload moved and explains that move away. A fault during a load change still shows, because the load is a median that one or two services cannot drag, and a signal counts as normal only if it either ignores the load or follows it. Bursty signals get a fair spread, the engine never learns from readings it alarmed on, and each alarm says which signal moved and by how much. The rules the mapper was built on stay: no labels, no agent on the hosts, only the telemetry the mapper already reads.

6.1.4 Detection and root cause

Table 6.3 compares the engines in one harness, now with the corrected ML engine inside. On the real systems the hybrid and the ML engine name the true cause first for 71% of all faults, against 63% for the rule layer and 68% for ranking by anomaly alone. The paired differences are in Figure 6.5. They are smaller than in the first version of this report and no longer reach significance. Hybrid over rules is +8.1 points on the real systems (95% CI $[-1.6, +17.7)$, $p = 0.117$) and +5.6 on the synthetic set ($[-0.6, +11.8)$, $p = 0.093$). Hybrid over the severity ranking is +3.2 and +1.1 points, and hybrid over the ML engine 0.0 and +0.6. With 62 real faults a difference below about ten points cannot be resolved. What the corrected engine changes is the false-alarm column: the ML engine alarms 2.3 times per fault-free hour on the real systems and 0.2 on the synthetic set, the quietest row, and Section 6.1.1 shows that it stays quiet when only the load moves. The hybrid raises more episodes (6.2 and 0.9) because the rule layer may also trigger it.

Table 6.3: Detection and root cause in one harness (point estimates with 95% bootstrap intervals over faults). Det.: faults detected (%). A@1, A@3: true cause first or in the top three among detected (%). C+N: caught and named first over all faults (%). FA/h: false-alarm episodes per fault-free hour. Alarm: share of fault-free time in alarm (%).

Engine	Det. [95% CI]	A@1	A@3	C+N [95% CI]	FA/h	Alarm
Real benches (62 faults, 217 min)						
Rules	90 [82, 97]	70	91	63 [52, 74]	9.2	9
Severity ranking	90 [82, 97]	75	86	68 [56, 79]	6.2	8
ML engine	90 [82, 97]	79	89	71 [60, 81]	2.3	6
Hybrid	90 [82, 97]	79	89	71 [60, 81]	6.2	8
Hybrid + change-point	95 [89, 100]	76	88	73 [61, 82]	5.4	16
Hybrid + learned dir.	90 [82, 97]	75	89	68 [55, 77]	6.2	8
Synthetic held-out (178 faults, 16 topologies)						
Rules	89 [84, 93]	78	95	70 [62, 76]	2.5	2
Severity ranking	93 [89, 97]	80	92	74 [67, 80]	0.9	5
ML engine	92 [87, 96]	82	94	75 [69, 81]	0.2	4
Hybrid	93 [89, 97]	81	93	75 [69, 81]	0.9	5
Hybrid + change-point	94 [90, 98]	80	92	75 [69, 81]	0.8	6
Hybrid + learned dir.	93 [89, 97]	81	93	75 [69, 81]	0.9	5

The operating curves (Figure 6.4) show why the comparison is delicate. The engines trade detection for false alarms along nearly overlapping curves. The hybrid and the ML engine sit on or near the upper envelope in both sets, above the rules at almost every false-alarm budget, so their advantage lies in the operating point and not in a single threshold. Figure 6.6 breaks C+N down by fault class. The gain over the rules is concentrated in stop faults (16 of 16 against 9 of 16 on the real systems), which is the effect of the docker stop fact, and, on the synthetic set, in flap faults (12 of 12 against 2 of 12).

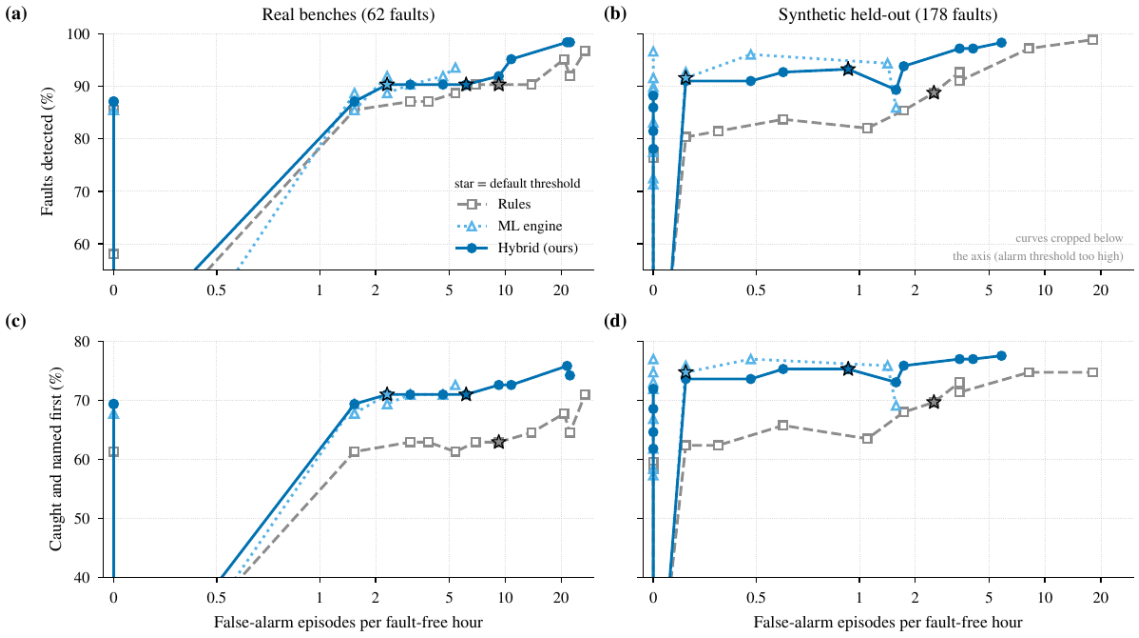


Figure 6.4: Operating curves. Sweeping the alarm threshold of each engine traces detection rate (top) and caught-and-named rate (bottom) against false-alarm episodes per fault-free hour

(symmetric-log axis); stars mark the default threshold. The hybrid sits on the upper envelope in both the real systems (a, c) and the synthetic held-out set (b, d).

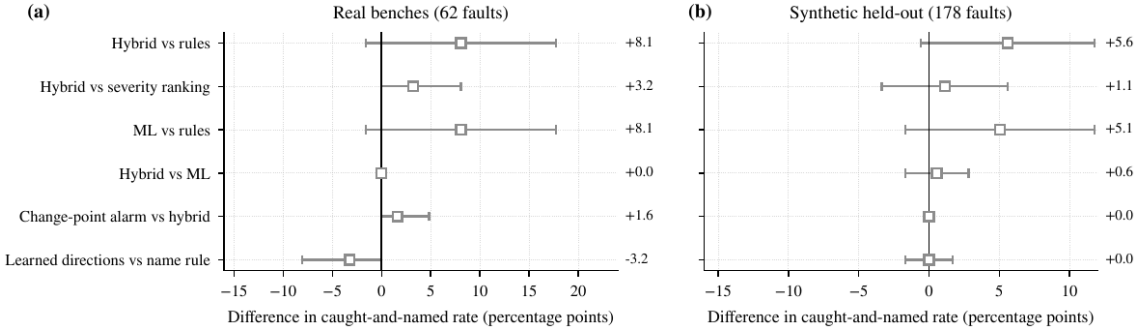


Figure 6.5: Paired differences in caught-and-named rate between engines (percentage points, 95% paired-bootstrap interval); filled markers exclude zero.

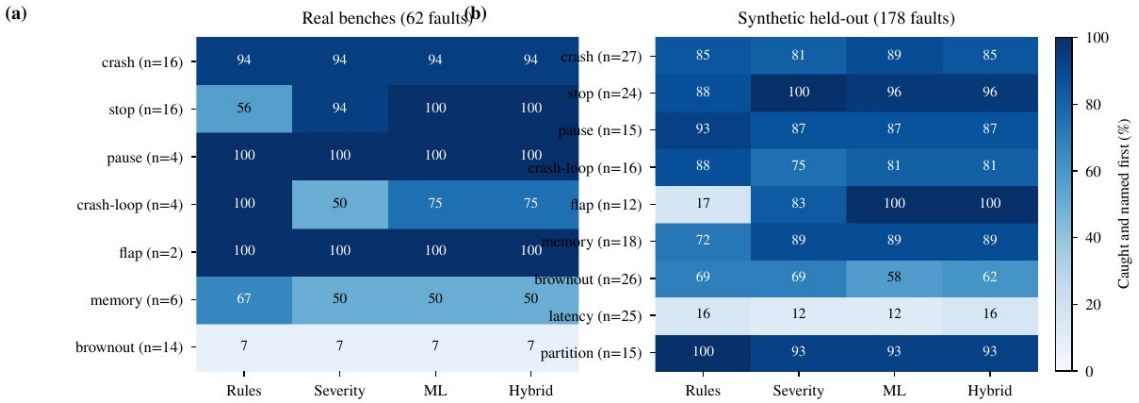


Figure 6.6: Caught-and-named rate by fault class (percent; *n* faults per class), for four rankers on the real systems (a) and the synthetic held-out set (b). Brownouts are the open problem on the real systems.

6.1.5 Which detector

Table 6.4 scores each model of normal as a service-level classifier, now including the control chart, the subspace residual and the adopted load-aware model. As before, the plain rule z-score is as good as or better than every learned detector at separating a faulted service from a healthy one by AUROC (0.695 against 0.629 for the old forest and 0.665 for the load-aware model on the real systems; 0.873 against 0.862 and 0.842 on the synthetic set). On average precision, which weighs the rare faulted services, the load-aware model is above every forest-family detector on both sets (0.313 against at most 0.290 on the real systems; 0.665 against at most 0.608 on the synthetic set). It is level with the rules on the synthetic set (0.666). Separation on the real systems is weak for every detector (AUROC 0.62 to 0.70), because many real faults hit services that are nearly idle.

Discrimination is therefore not what chose the model; behaviour under a change of workload was (Section 6.1.2). The adopted model is quiet (alarm time of 6% against 9% for the rules on the real systems, Table 6.3); it separates own from symptom evidence; and its

alarms complement those of the rules, so their union detects more faults than either on the synthetic set (93% against 89%).

Table 6.4: Service-level detection by model of normal (mean ± s.d. over three seeds for the randomised detectors; the rule layer and the three residual models are deterministic).

	Real		**Synthetic**	
Detector	AUROC	AP	AUROC	AP
Rule z-score	0.695±0.000	0.337±0.000	0.873±0.000	0.666±0.000
Isolation Forest (old)	0.629±0.004	0.288±0.003	0.862±0.004	0.528±0.020
k-NN	0.668±0.010	0.193±0.004	0.860±0.001	0.517±0.004
Mahalanobis	0.650±0.003	0.290±0.004	0.868±0.001	0.608±0.006
HBOS	0.620±0.005	0.274±0.006	0.853±0.004	0.473±0.030
Ensemble of four	0.650±0.006	0.287±0.009	0.865±0.001	0.593±0.002
Control chart (no load factor)	0.669±0.000	0.302±0.000	0.842±0.000	0.657±0.000
Subspace residual	0.696±0.000	0.298±0.000	0.847±0.000	0.629±0.000
Load-aware residual (new)	0.665±0.000	0.313±0.000	0.842±0.000	0.665±0.000

6.1.6 Detection delay and root-cause depth

Figure 6.7 shows detection delay as a cumulative distribution. On the real systems the hybrid's median delay is 11.3 s against 14.9 s for the rules, and the change-point alarm lifts the detected fraction from 90% to 95% and moves the median to 4.8 s. On the synthetic set, whose faults are abrupt, the curves almost coincide (8.2 s against 9.5 s for the rules). The change-point alarm has a price visible in Table 6.3: alarm time rises from 8% to 16% on the real systems. In the top k (the A@1 and A@3 columns of Table 6.3) the ML-based rankers reach the true cause first more often than the rules on the real systems (79% against 70% of the detected faults), and the rankers converge by $k = 3$ near the 90% detection ceiling, so the remaining misses are mostly faults the engine does not see (brownouts on near-idle services), and few are faults it ranks badly.

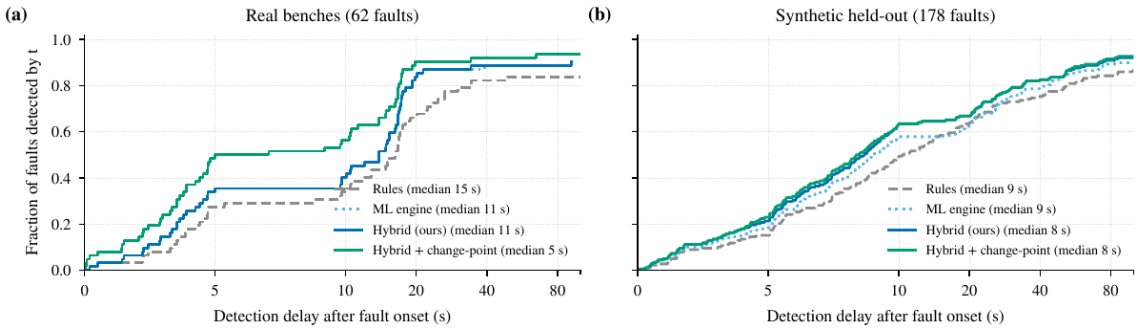


Figure 6.7: Detection delay after fault onset as a cumulative distribution (symmetric-log axis); legends give the median over detected faults. The curves plateau at each engine's detection rate.

6.1.7 Ablation

Figure 6.8 removes one mechanism of the corrected ML engine at a time. The most important single component is still the docker stop fact: without it C+N falls by 18 points on the real systems and 15 on the synthetic set, because every stopped service goes unnamed. The crash-loop fact (0 and 7 points) and the random walk (3 and 2 points) each contribute. Two mechanisms act on the contract rather than on accuracy. Without the load factor nothing changes on injected faults (the load does not move in them) and the effect appears only on the workload recordings of Section 6.1.1: false alarms rise from 0.0 to 1.2 and from 1.0 to 1.9 per hour there. With the MAD spread instead of the tail-aware one, alarm time on the real systems doubles (6% to 12%) and, on the workload recordings, rises from 0% to 10% and 6% (13% and 31% when the load factor is also removed). Removing the one-sided features raises alarm time on the real systems from 6% to 24%; the two-speed drift keeps it at 6% instead of 13% and false alarms at 2.3 instead of 3.1 per hour. Alarming at level one instead of two raises C+N by 2 points on both sets but more than doubles the false alarms on the real systems (5.4 per hour). The one-sided features do not matter on the synthetic set, where restarts do not lower memory: an effect of the real systems that the simulator does not reproduce. The range guard of the first version has no counterpart: the robust spread takes its place.

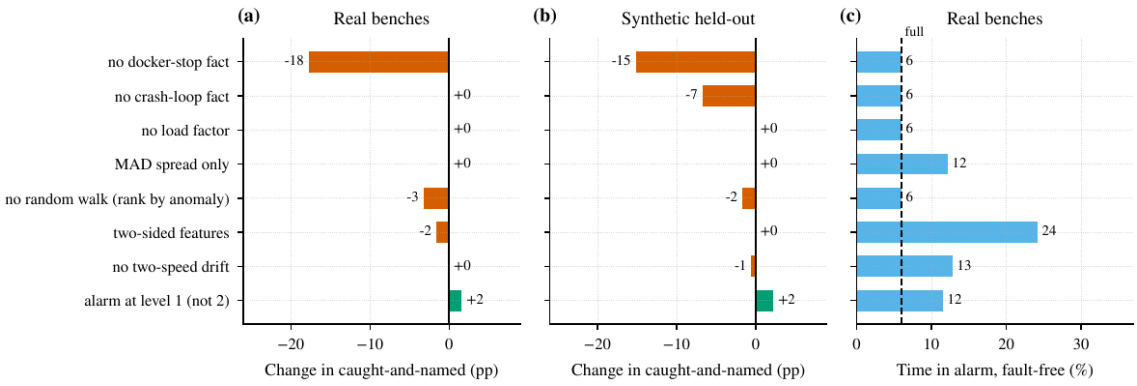


Figure 6.8: Ablation of the corrected ML engine. Each bar removes one mechanism from the full engine. (a), (b): change in caught-and-named rate (percentage points) on the real systems and the synthetic held-out set. (c): share of fault-free time in alarm on the real systems; the dashed line marks the full engine.

6.1.8 When does the graph matter? The amplification study

Standard benchmarks have faults whose own service is also the loudest, which is the case where ranking by severity cannot fail. To probe the regime in which it can, soft faults (brownout, latency, memory) were regenerated with the amplification factor $A \in \{1, 1.5, 2, 3, 4\}$, the hybrid's alarms were kept identical for every ranker, and only the ranking varied (Figure 6.9). As A grows, the share of faults in which some victim is louder

than the true cause rises from 10% to 21% (panel c), the severity ranking falls from 76% to 71%, and the walk holds at 75 to 76%. The gain of the walk over severity therefore grows from zero to +3.8 points (95% CI $[-0.9, +9.4]$ at $A = 4$) and that of the walk with learned directions to +4.7 ($[-0.9, +10.4]$) (panel b). The intervals include zero at every level. In the first version of the study the own-versus-explained decomposition of the rule layer was the strongest ranker on soft faults; with the corrected ML engine it is not (71 to 75%, −1.9 to 0.0 points against severity, intervals including zero), because the engine's own and symptom levels now carry the same separation. A router that uses it whenever no container fact holds did not change the full benchmarks (within ±2 points), so the shipped ranking is unchanged. Graph-aware ranking remains a modest, regime-dependent gain that this sample is too small to confirm.

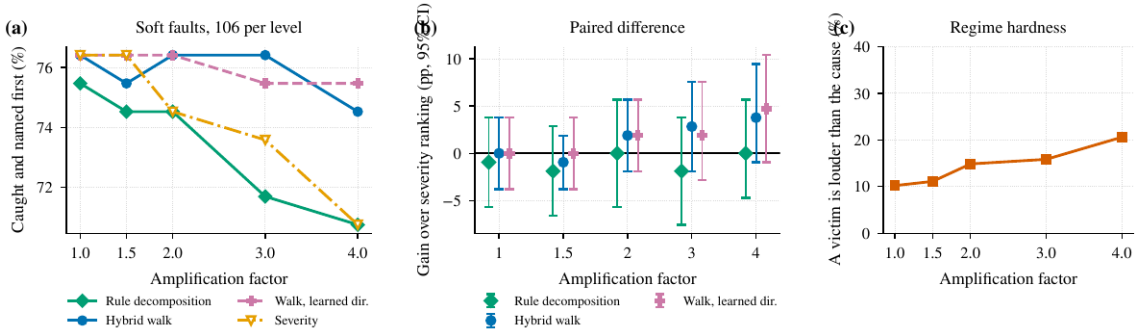


Figure 6.9: Amplification study on soft faults (106 per level). (a) Caught-and-named rate of four rankers on identical alarms. (b) Gain over the severity ranking with 95% paired-bootstrap intervals. (c) Share of faults in which a victim is louder than the true cause.

6.1.9 The cascade model

Figure 6.10 evaluates who-is-next predictions at the first alarm of each fault: for every service still healthy, the probability that it turns unhealthy before the fault ends. On the synthetic set (163 faults, 1 289 predictions, 69 cascades) the learned propagation model reaches AUROC 0.841 against 0.827 for the same noisy-OR with every edge fixed at 0.7, a Brier score of 0.047 against 0.052 and an ECE of 0.033 against 0.051. The calibration is better than that of the fixed model, the ranking only slightly. The reliability curve is noisy in the upper bins, which hold few predictions. *On the real systems the experiment cannot be done well:* only 10 of 535 predictions were followed by a cascade (56 faults), so the real-system comparison (AUROC 0.742 against 0.633; the live mapper's own risk values 0.591) is not informative. The real faults are mostly single-service outages that the systems absorb; whether learned propagation helps on a real cascade is untested.

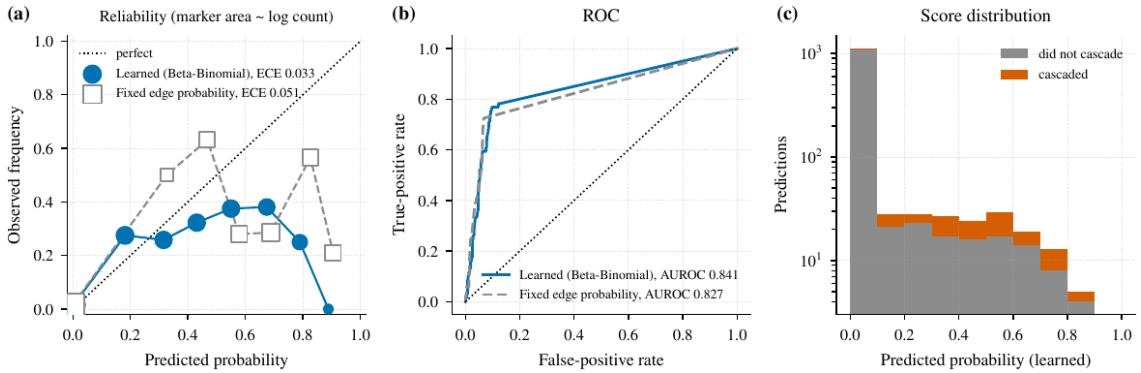


Figure 6.10: Who-is-next prediction on the synthetic held-out set. (a) Reliability diagram with equal-width bins and marker area proportional to the logarithm of the bin count. (b) ROC curves. (c) Distribution of the learned predicted probabilities.

6.1.10 Learned directions

Figure 6.11 reports direction accuracy on observed pairs that are true calls. With names available the name rule is correct for 21 of 31 real pairs (and leaves 10 undecided); the learned rule is correct for 27 of 31 (4 wrong). With service names hidden the name rule is undecided for every pair while the learned rule is unchanged; the same holds on the synthetic set (95 of 204 against 198 of 204 correct). The effect on ranking is smaller than The effect on directions. With names available the learned directions do *not* beat the name rule (68% against 71% on the real systems, −3.2 points, not significant). With names hidden they reach 68% against 66% for the name rule and 71% for the true graph. Learned directions are a robustness mechanism, not an accuracy gain.

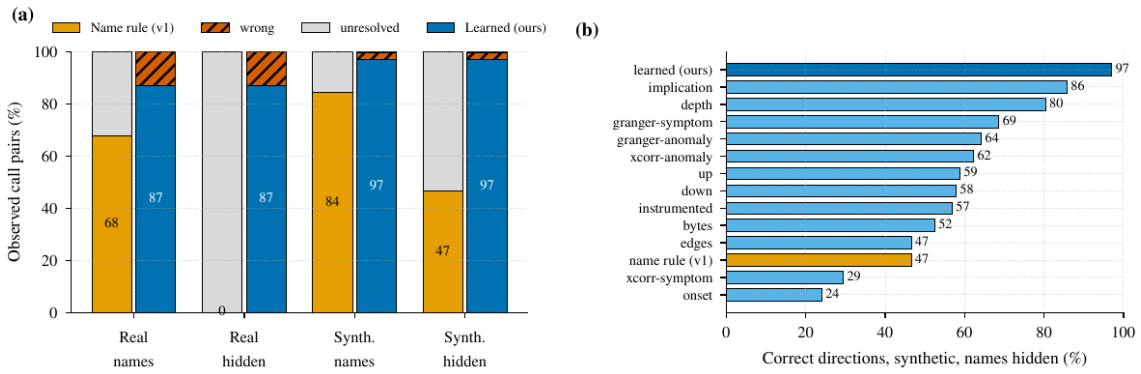


Figure 6.11: Call directions. (a) Share of observed true-call pairs oriented correctly, wrongly, or left unresolved by the name rule and by the learned rule, with service names shown or hidden. (b) Correct share of each individual signal on the synthetic set with names hidden.

6.1.11 Early warning and fault diagnosis

The trend forecast (now a straight line through the recent deviations from what the load predicts) warns before the alarm for 13 of 178 synthetic faults (7 of 18 memory leaks, 4 of 25 latency faults, 2 of 26 brownouts) with a median lead of 25 s and 0.8 false warning episodes per fault-free hour (Figure 6.12); the first version, on the raw readings, warned for 16 (5 of 26 brownouts). On the real systems it fired before the alarm for none of the 62

faults, since those faults are abrupt, and raised 6.9 false warning episodes per hour; early warning of gradual faults is the weakest part of the system and is reported as such. Figure 6.13 shows fault-type diagnosis. With the true service the forest is correct for 99% of synthetic faults 60 s after the alarm; on the engine's own suspect it is 79%, because a wrong suspect is rarely typed correctly. On the real systems a hand-written decision list generalises best (64% on the suspect, 84% on the true service), because real flaps are pause cycles and real brownouts on idle services show no throttling: differences between the simulator and the real systems that the learned models absorb as synthetic-specific patterns.

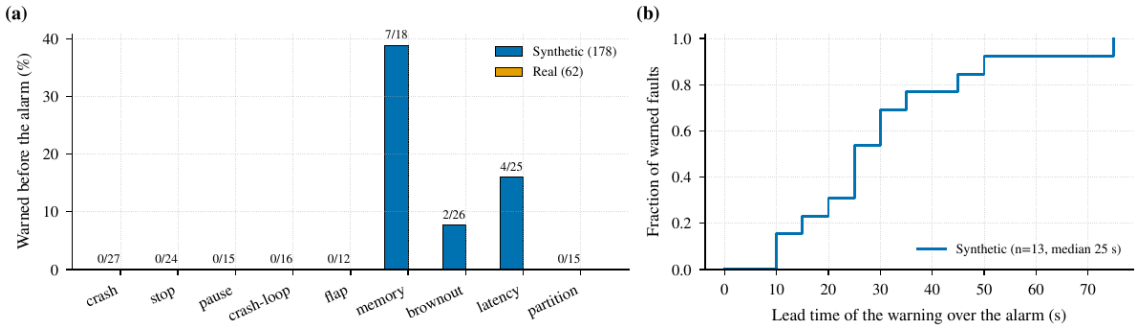


Figure 6.12: Early warning. (a) Share of faults warned before the alarm, by fault class. (b) Lead time of the warnings over the alarm as a cumulative distribution.

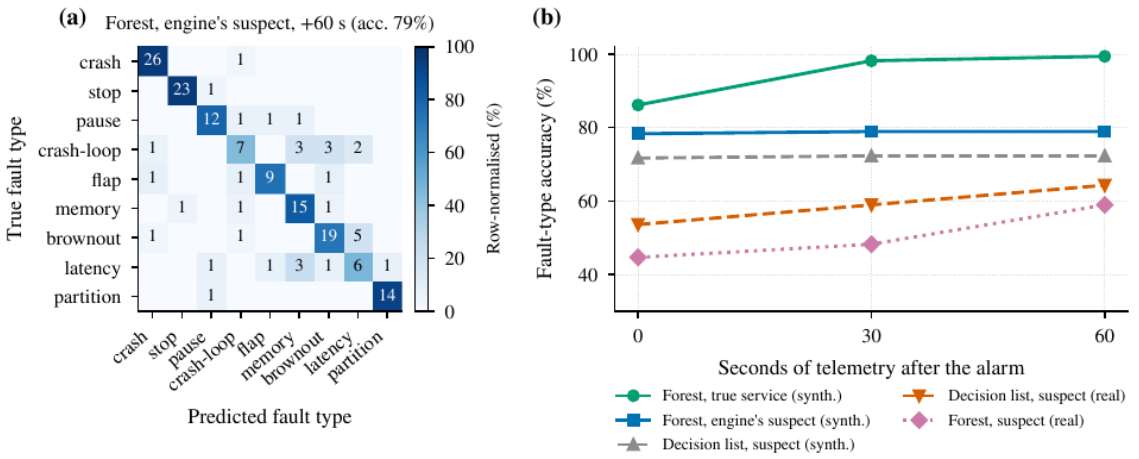


Figure 6.13: Fault-type diagnosis. (a) Confusion matrix (counts; shading is row-normalised) of the random forest on the engine's suspect 60 s after the alarm, synthetic held-out set. (b) Accuracy against seconds of telemetry after the alarm, for the true service and the engine's suspect.

6.1.12 Cost

The pipeline is unsupervised, needs no GPU and runs on one core. Figure 6.14 times one complete decision window (container facts, the model of normal, random walk, forecast) on random call graphs of 10 to 320 services, after a warm-up long enough for every service to have a model. The adopted load-aware model costs about 0.37 ms per service per window and grows linearly: 4.0 ms for 10 services, 26.8 ms for 80 and 119 ms for 320 (95th percentile 356 ms), always far below the 5 s collection period. The old forest cost about 0.9 ms per service (296 ms for 320) with 95th-percentile windows nine times the mean,

because every service refits at once every ten readings. The load-aware model refreshes its baselines every four readings with a much smaller peak (about three times the mean). The subspace residual grows faster than linearly (1.2 s for 320 services, with 11 s windows at the 95th percentile) because its covariance has one row per service-signal pair. On the two benches (12 and 13 services) a window takes about 5 ms. These are wall-clock times on one core of a shared four-core virtual machine, not a tuned benchmark.

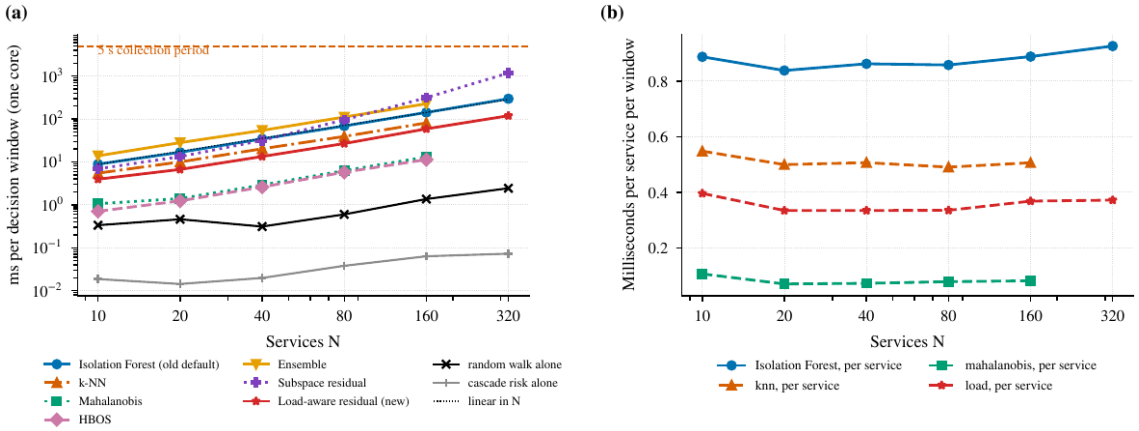


Figure 6.14: Cost of one decision window against the number of services (single core, logarithmic axes; the dashed line is the 5 s collection period). (a) Mean milliseconds per window for each model of normal, the random walk and the cascade risk alone. (b) Milliseconds per service per window.

6.1.13 Case studies

Figure 6.15 shows three real incidents, chosen by class before looking at the outcome. In (a) a memory leak overlaps several outages: the engine names the stopped containers first, as it should, and only a model of concurrent faults could do better. In (b) a crash loop is named correctly about 30 s after its first kill and stays named between kills. In (c) a brownout on an almost idle service never raises an alarm, which is the open problem of Figure 6.6.

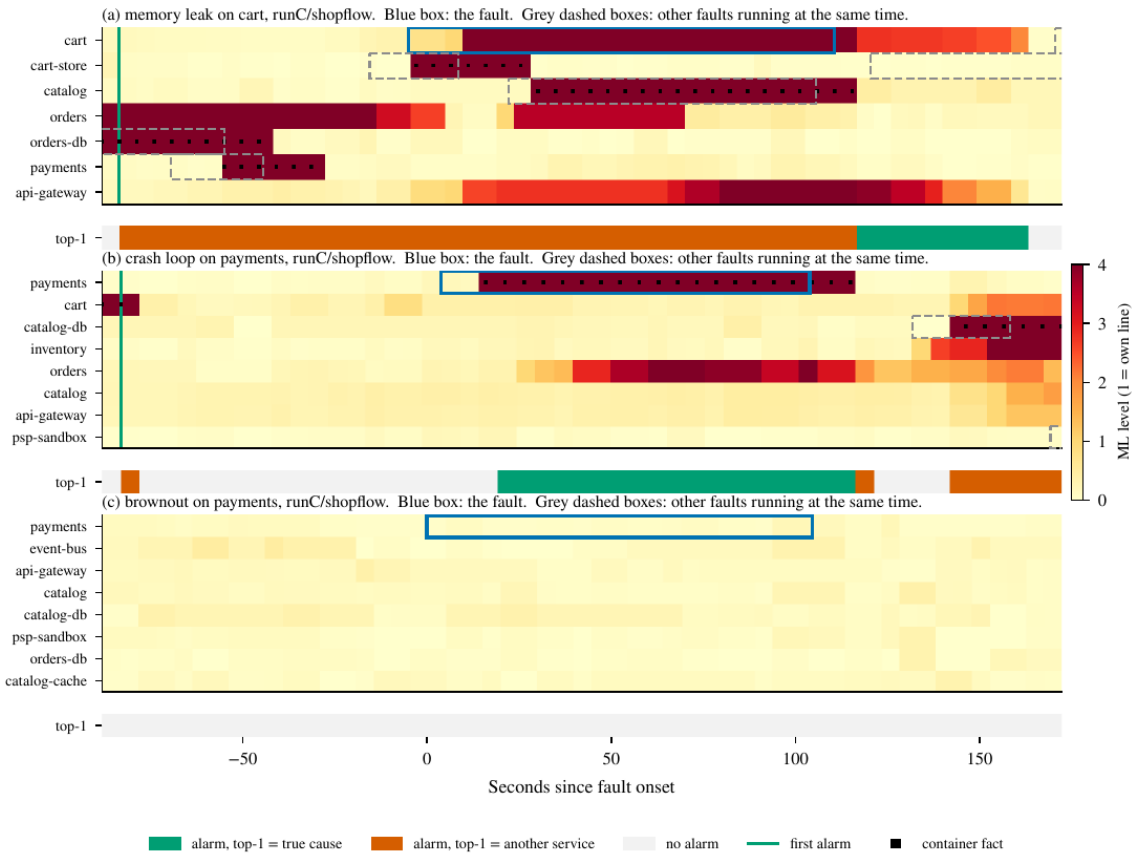


Figure 6.15: Three real incidents. Heat maps show the ML level of the most relevant services; the blue box is the fault, grey dashed boxes are other faults running at the same time, black squares mark container facts, the green line is the first alarm in the window (an alarm already ongoing from an earlier fault starts at the left edge), and the strip below each map shows whether the engine's top-ranked service was the true cause.

6.1.14 The three engines on one live fault

To show what the Detection engines page adds, the simulator was run with a slow payment service (CPU saturated at 97% from second 30 of the fault run; orders follows at 86% twelve seconds later), the ground-truth root cause being payment. The three engines were read every four seconds through the page's API after a 260-second quiet period (Table 6.5).

Table 6.5: One simulated fault (slow payment): each engine's first suspect and its peak against its own alarm line, at three moments after the onset. The simulator reports every two seconds, so 24 readings are 48 s here (two minutes on a real host).

Engine	+5 s: payment only	+20 to +70 s: both saturated	after +75 s
Rules	payment (3.0×)	orders (3.7×), payment second (3.0×)	the same
ML	payment (88% of the walk), peak 1.3	payment (57%), orders (43%), peak 1.3	absorbed: no alarm
Hybrid	payment	payment (0.91), orders (0.87)	orders (alarm kept by the rules)

This is one simulated fault and an illustration, not evidence. It shows the mechanism the evaluation measures at scale: the rule layer ranks the loudest service first, so it blames

orders as soon as both are saturated, while the ML engine lets the walk keep the blame on the source. It also shows the price of the two-minute drift rule (Section 6.1.2): a saturation that lasts longer than the acceptance time is absorbed as the new normal and the ML alarm clears. The hybrid keeps alarming because of the rule layer, but its ranking then follows the rules.

6.1.15 Operational findings from the benchmark hosts

Operating the benchmark hosts produced three findings that changed the tools.

- A host too small for its workload looks like a mapper fault. The OpenTelemetry demo ran on a 2-vCPU, 4 GiB instance, with a load average of about 25 and 3.6 of 3.8 GiB of memory in use, no swap, and Docker did not answer within 25 seconds; the dashboard showed no services. A second host with 4 GiB of swap and a load average of 0.2 ran 27 containers of a social-network benchmark normally. The lesson is operational: the mapper depends on the host's Docker daemon answering, and a host that cannot answer shows nothing. The check tool now says so in one step.

- Idle front ends have no edges. On a healthy social-network host, the front-end containers showed no outgoing sockets when sampled, because nothing was calling them; an undeclared edge expires ten minutes after its last traffic. A front end that is idle therefore has no edge until traffic arrives, which is correct behaviour but looks like a fault. Declared edges from the Compose file keep an idle link on the screen.

- Hard-coded project names blocked traffic; the traffic controller accepted only five named projects, so benchmark hosts with other names could not be driven. Unknown projects now get a generic workflow.

6.2 Live Fault Lab Experiments: Auto-restart On and Off

The 62-fault evaluation above was made on recorded campaigns. To test the mapper once more against a system it cannot see into, and to settle whether its optional auto-restart changes the results, both benchmark benches (Section 5.10) were run live inside Docker, reached by the mapper over SSH exactly like a remote host (auto-restart exists only on that path). The Fault Lab replayed the *same* plan every time (seed 11, the stormy profile, crash, stop, pause and brownout faults, 18 minutes, 21 faults), once with MAPPER_AUTO_RESTART on and once with it off, after a 6-minute warm-up. The hybrid engine decided the incidents and the harness scored them against the injector's

private ledger. Only one bench ran at a time. The run scripts and every recorded file are in the repository (run/scripts/autorestart-experiment.sh, run/recorded-runs/autorestart/).

Table 6.6: Live campaigns, same fault plan, auto-restart on and off (21 faults each; 95% Wilson intervals).

Bench	Auto-restart	Detected	Top-1 (%)	Top-3 (%)	Delay (s)	FA/h
ShopFlow	on	100%	33 [17, 55]	71 [50, 86]	3.3	9.2
ShopFlow	off	100%	24 [11, 45]	71 [50, 86]	2.6	0.0
LedgerLine	on	95%	55 [34, 74]	65 [43, 82]	9.7	0.0
LedgerLine	off	95%	40 [22, 61]	70 [48, 85]	3.9	3.1

Delay is the median time from fault onset to the first alarm; FA/h is false-alarm episodes per fault-free hour.

Table 6.7: Top-1 and Top-3 root-cause accuracy (%) by fault class in the live campaigns (on / off).

			ShopFlow	LedgerLine
Fault class	**Faults**		**Top-1 / Top-3**	**Top-1 / Top-3**
Brownout	6	on / off	0 / 17 0 / 0	0 / 0 0 / 20
Crash	6	on / off	33 / 83 33 / 100	80 / 80 33 / 83
Pause	7	on / off	57 / 100 29 / 100	86 / 100 86 / 100
Stop	2	on / off	50 / 100 50 / 100	50 / 100 0 / 50

What the numbers show. Every fault but one was detected in both systems, and the true cause was within the top three suspects in 65 to 71 % of the faults. The differences between the on and off rows are 9 and 15 points of Top-1, and the false-alarm column moves in opposite directions (more with auto-restart on in ShopFlow, fewer in LedgerLine). With 21 faults the intervals overlap completely, so none of these differences can be called an effect of auto-restart. Brownouts are the weak class in all four runs (no Top-1 at all), which agrees with the 62-fault results of Section 6.1.

Why there was no effect to find. The log of each run shows what auto-restart actually did. It restarted no container of the bench under test. The one restart in each “on” run was of the *other* bench’s message bus, which had been stopped by hand to keep one bench per run and which exited with code 1; the mapper, which sees every container on the host, restarted it. That is an artefact of the setup and is reported as such. Three properties of the mapper and the benches explain the rest:

- the benches set restart: always, so Docker brings a crashed container back within seconds, before the mapper’s next pass;

- the mapper only looks for dead containers once per discovery cycle (180 s), longer than the 20-second crash faults;

- it skips containers that exited with code 0, and a container stopped with docker stop exits with code 0.

A direct test with long outages. To give auto-restart a chance to act, cart and payments were each stopped for 5 minutes by the Fault Lab (run/scripts/autorestart-stop-test.sh), with the state of each container sampled every 2 seconds. Table 6.8 shows no difference: the mapper left the containers down in both modes.

Table 6.8: Time a stopped container stayed down, with auto-restart on and off (injector releases it after 300 s).

Container	On (s)	Off (s)
cart	299	298
payments	301	301

Conclusion. On these benchmarks auto-restart is inert: it neither helped nor hurt detection or root-cause accuracy beyond run-to-run variation, and the rule that skips exit code 0 means a deliberate stop is never undone. It would matter on a host whose containers have no restart policy and exit with a failure code, which these benches do not model, so that case is untested. Switching it off for evaluation remains the safe default, because the mapper then cannot heal what the injector breaks.

6.3 Analysis

What the evidence supports. The first ML engine mistook a change of workload for a fault: on fault-free recordings it sat in alarm for 23 to 52% of the time, and every workload-blind detector we tried did the same or worse (Section 6.1.1). The load-aware model that replaced it stays quiet on both recordings and loses nothing on injected faults. Container facts carry more weight than any other component; the ablation costs 18 and 15 points when the docker stop fact goes. And learned propagation calibrates better than fixed edge probabilities on observed cascades (ECE 0.033 against 0.051, AUROC 0.841 against 0.827).

What did not work. Several expected gains never appeared. The corrected engine is no more accurate than the first on injected faults. Hybrid over rules was significant in the first version (+9.7 points, $p = 0.032$) and is not now (+8.1, $p = 0.117$), so the improvement sits in robustness, not in the benchmark score. The random walk does not beat ranking by anomaly alone, learned directions do not beat the name rule when names exist, and onset-precedence

re-ranking changed nothing. The subspace residual, the principled multivariate alternative, discriminates as well as the adopted model but raises more false alarms on the load steps and costs more. We report these because an operator gains more from a pipeline that states where it stops. The plain rule z-score remains a strong detector. The load-aware model earns its place by staying quiet under changing load and by catching faults the rules miss, not by separating faulted services better.

The measurement bugs. Before we fixed the ledger reader, every engine scored against only part of the real faults (14 of 31 per system stayed hidden), and the outages of the hidden faults counted as false alarms. Every number in this report therefore uses all 62 faults, and a regression test guards the reader. A second bug in the replay (Section 4.10.6) deleted the idle stretches from the recordings. That bug explains why the benchmarks never exposed the first ML engine's failure; an operator found it on a live system. Both bugs flattered the engines.

6.4 Discussion and Threats to Validity

Construct validity. The injector's ledger is the ground truth. It names the structural root cause and the services able to suffer. Whether a service visibly suffers depends on protections the ledger does not model, so cascade recall understates true propagation. The offline recordings keep no edge evidence, which leaves the evidence weighting of Section 4.6.1 unmeasured.

Internal validity. The authors chose the settings. We swept the alarm line, window and slow reference of the load-aware model over all recordings together (Figure 6.3), which amounts to tuning on test data; the adopted values sit in the sweep's flat region, not at its maximum. On the synthetic track a disjoint tuning split picked the settings and the test split was scored once. On the real track no held-out system exists, so the real numbers run optimistic by an unknown margin. We found the two container facts by inspecting the real runs.

External validity. Two real applications, one capture and live recordings on one machine limit the claim. Generators produced the workload recordings (organic or closed-loop), so a real day of users stays untested, and a fault that begins during a large change of load can hide behind the load-aware model. Architecture independence rests on the cross-system check and on synthetic topologies. It is argued, not proven. The simulator is also cleaner than a real system: restarts do not lower memory in it.

Conclusion validity. With 62 real faults, no difference below about ten points is resolvable, and overlapping faults make the intervals optimistic. The workload claims rest on four fault-free recordings of 62 and 72 minutes. The effect is large (0% against 23 to 96% of the time in alarm), but four recordings are few.

Auto-restart. The mapper can restart crashed containers on a target (Section 5.11, Chapter 3). Live campaigns with the feature on and off showed no effect on these benches (Section 6.2). Even so, switch it off for evaluation runs so the mapper cannot heal the faults the injector applies.

6.5 Comparison with Existing Approaches

The published methods in Table 2.1 were not re-implemented on our recordings, so there is no head-to-head accuracy table against them. What is compared head to head are the baselines that those methods build on: the rule layer, ranking by anomaly alone (the severity ranking), the walk, and the ML engine in its two versions. Table 6.9 sets the measured results beside the qualitative properties of Chapter 2.

Table 6.9: This project beside the approaches it builds on. Accuracy figures are caught-and-named on the 62 real faults.

Approach	C+N (real)	Notes
Rule layer (z-score)	63%	Fast, 9 false-alarm episodes per hour, 9% alarm time
Severity ranking (loudest service)	68%	Cannot fail when the cause is loudest; weakens as victims get louder
Per-service Isolation Forest + walk (first version)	69%	4.6 false alarms per hour; in alarm for 23 to 52% of the time when only the load moves
ML engine (load-aware model + walk)	71%	Fewest false alarms (2.3 per hour), 6% alarm time; 0% when only the load moves
Hybrid (this project)	**71%**	Paired $+8.1$ points over rules (not significant); names stops 16 of 16
Learned fusion methods [4, 5, 6]	not run	Need labelled failures and GPU; not auditable
Trace-based rankers [34]	not run	Need tracing in every service

6.6 Significance and Implications

An instrumentation-free tool can recover a usable call graph and a useful ranking, and for small teams that is the difference between having a diagnosis tool and not. Container facts are worth more than any statistical refinement: a stopped container is something one knows, and folding it into an anomaly score throws information away. And the right operating

contract for an alarm is the one the operator experiences, which is why alarm time is reported beside false-alarm episodes.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Summary of Work

We built a platform that watches containerised microservices from outside, recovers their call graph and diagnoses incidents without labels. A collector reads Docker state, statistics and socket tables over SSH. The graph orients each edge by its listening port. A load-aware residual model and a robust rule layer share the alarm, and container facts count as evidence. Each deviation is split into what neighbours explain and what they do not, and a random walk over the call graph orders the suspects. Learned Beta–Binomial edge probabilities in a cascade model predict who falls next. The dashboard draws the result in 2D and 3D, and the Detection engines page runs Rules, ML and Hybrid side by side. We tested it on 62 faults injected blind into two real systems and 178 held-out synthetic faults, and operated it on further benchmark applications on EC2.

7.2 Conclusions

1. The first ML engine, a per-service Isolation Forest, mistook a change of workload for a fault. It sat in alarm for 23 to 52% of fault-free recordings in which only the load moved, and every other workload-blind detector did the same or worse. A load-aware residual model estimates the work all services share and judges only what remains. It stays in alarm for 0% of those recordings and matches the forest on injected faults (71% against 69% caught and named on the real ones).

2. On 62 real faults the hybrid names the true cause first for 71% against 63% for the rule layer (paired $+8.1$ points, 95% CI $[-1.6, +17.7]$, $p = 0.117$); on the synthetic set the difference is $+5.6$ points, also not significant.

3. Container facts matter most. Removing the docker stop fact costs 18 points on the real systems and 15 on the synthetic set.

4. Learned propagation is better calibrated than fixed edge probabilities on 69 observed cascades (expected calibration error 0.033 against 0.051; AUROC 0.841 against 0.827), with a lower Brier score. On the real systems, the experiment is uninformative because almost no real fault cascaded.

5. Learned directions stay correct when service names carry no information (27 of 31 real pairs against none for the name rule), but they do not improve ranking when names exist.

6. Change-point alarms cut the median detection delay on the real systems from 11.3 s to 4.8 s, at the price of more alarm time (8% to 16%).

7. The simple rule z-score is a strong detector on its own; the load-aware model earns its place through quietness and by surviving changes of workload.

8. The pipeline costs about 0.37 ms per service per five-second window (about 5 ms for the systems tested, 119 ms for 320 services), linear in the number of services, on one core, with no GPU.

7.3 Contributions

- An observation layer that recovers topology and direction from the host alone.
- A probabilistic cascading-failure model with learned propagation.
- A load-aware model of normal, found by comparing nine unsupervised models on fault-free workload recordings as well as on injected faults, and a hybrid detector with an honest operating contract (alarm time as well as false-alarm episodes).
- A fact-aware, decomposition-based ranker and learned call directions.
- A detection engines page showing three engines in parallel.
- A blind, reproducible evaluation that includes the negative results and a corrected measurement bug.
- A working platform with 339 automated tests, a traffic generator and host-diagnostic tools.

7.4 Utility and Applications

Small teams running Docker Compose stacks that nobody wants to instrument can use the platform for on-call support. It also works as a first-line triage layer in front of heavier observability products, giving a ranked suspect and a runbook. Its what-if and blast-radius views serve resilience reviews. With a blind injector and replayable recordings it is a lab for teaching cascading failure, and a benchmark harness for new detectors and rankers, with ground truth the detector never sees.

7.5 Limitations

- Coarse signals. Eight metrics every five seconds. A brownout on an almost idle system. service never raises an alarm; no engine here catches it.

- Two real systems. Architecture independence is argued from a cross-system check and synthetic topologies, not proven. Real cascades were too rare in the recordings to test learned propagation on real data.

- Edge evidence is not isolated offline. The recordings do not retain edge evidence, so the effect of the evidence weighting is not measured.

- No scaling measurement. The cost was measured on random graphs of 10 to 320 services with synthetic readings, on a shared virtual machine; no real system of that size was observed.

- Workload recordings are generated. The load changes of the workload recordings come from two traffic generators on two benches, not from real users; a fault that begins during a large change of load can be masked by the load-aware model.

- Tuned by the authors. Real-system settings were not tuned on a held-out system, so those numbers are optimistic by an unmeasured margin.

- Auto-restart. The mapper restarts crashed containers by default (switch off with MAPPER_AUTO_RESTART=off); live runs with it on and off (Section 6.2) showed no difference on the two benches, but it should still be switched off when measuring detection.

- The push endpoint has a built-in token, which must be changed when the server is reachable by others.

7.6 Scope for Future Work

- A larger campaign on more real systems with real multi-service cascades, a public labelled data set, and edge evidence kept in the recordings so that its weighting can be measured.

- Real multi-service cascades to test learned propagation where it matters; promotion of learned directions and change-point alarms into the live system behind its operator switch.

- A language-model agent on top of the structured evidence (cause, evidence, runbook) to answer follow-up questions or draft a change, with the evidence as its only

grounding, evaluated through a user study; any action it proposes would go through an approval gate.

- Support for Kubernetes through the same observation idea (node-level access to container sockets), and a scaling benchmark.

- Workload recordings from a production-like day (a daily wave, a release, a flash crowd) and a model of a fault that begins during a load change.

- Brownout detection on idle services, perhaps with active probing of the service.

References

[1] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: Yesterday, today, and tomorrow,” in *Present and Ulterior Software Engineering*.1em plus 0.5em minus 0.4emSpringer, 2017, pp. 195–216.

[2] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*.1em plus 0.5em minus 0.4emO’Reilly Media, 2016.

[3] P. Notaro, J. Cardoso, and M. Gerndt, “A survey of AIOps methods for failure management,” *ACM Trans. Intelligent Systems and Technology*, vol. 12, no. 6, pp. 81:1–81:45, 2021.

[4] S. Zhang, P. Jin, Z. Lin, Y. Sun, B. Zhang, S. Xia, Z. Li, Z. Zhong, M. Ma, W. Jin, D. Zhang, Z. Zhu, and D. Pei, “Robust failure diagnosis of microservice system through multimodal data,” *IEEE Trans. Services Computing*, 2023.

[5] Y. Sun, Z. Lin, B. Shi, S. Zhang, S. Ma, P. Jin, Z. Zhong, L. Pan, Y. Guo, and D. Pei, “Interpretable failure localization for microservice systems based on graph autoencoder,” *ACM Trans. Software Engineering and Methodology*, vol. 34, no. 2, 2024.

[6] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, “Eadro: An end-to-end troubleshooting framework for microservices on multi-source data,” in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2023, pp. 1750–1762.

[7] Y. Gan, Y. Zhang, D. Cheng *et al.*, “An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems,” in *Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 3–18.

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[9] ——, “Isolation-based anomaly detection,” *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 1, pp. 3:1–3:39, 2012.

[10] A. Lakhina, M. Crovella, and C. Diot, “Diagnosing network-wide traffic anomalies,” *ACM SIGCOMM Computer Communication Review*, vol. 34, no. 4, pp. 219–230, 2004.

[11] J. E. Jackson and G. S. Mudholkar, “Control procedures for residuals associated with principal component analysis,” *Technometrics*, vol. 21, no. 3, pp. 341–349, 1979.

[12] S. Ramaswamy, R. Rastogi, and K. Shim, “Efficient algorithms for mining outliers from large data sets,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2000, pp. 427–438.

[13] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying density-based local outliers,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2000, pp. 93–104.

[14] M. Goldstein and A. Dengel, “Histogram-based outlier score (HBOS): A fast unsupervised anomaly detection algorithm,” in *KI-2012: Poster and Demo Track*, 2012, pp. 59–63.

[15] H. Hotelling, “The generalization of Student’s ratio,” *Annals of Mathematical Statistics*, vol. 2, no. 3, pp. 360–378, 1931.

[16] P. C. Mahalanobis, “On the generalised distance in statistics,” *Proc. National Institute of Sciences of India*, vol. 2, no. 1, pp. 49–55, 1936.

[17] O. Ledoit and M. Wolf, “A well-conditioned estimator for large-dimensional covariance matrices,” *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2004.

[18] F. R. Hampel, “The influence curve and its role in robust estimation,” *Journal of the American Statistical Association*, vol. 69, no. 346, pp. 383–393, 1974.

[19] P. J. Rousseeuw and C. Croux, “Alternatives to the median absolute deviation,” *Journal of the American Statistical Association*, vol. 88, no. 424, pp. 1273–1283, 1993.

[20] ——, “Alternatives to the median absolute deviation,” *Journal of the American Statistical Association*, vol. 88, no. 424, pp. 1273–1283, 1993.

[21] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[22] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, 2009.

[23] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, “Unsupervised real-time anomaly detection for streaming data,” *Neurocomputing*, vol. 262, pp. 134–147, 2017.

[24] N. Laptev, S. Amizadeh, and I. Flint, “Generic and scalable framework for automated time-series anomaly detection,” in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2015, pp. 1939–1947.

[25] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, “Robust anomaly detection for multivariate time series through stochastic recurrent neural network,” in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2019, pp. 2828–2837.

[26] R. Wu and E. J. Keogh, “Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress,” *IEEE Trans. Knowledge and Data Engineering*, vol. 35, no. 3, pp. 2421–2429, 2023.

[27] S. Kim, K. Choi, H.-S. Choi, B. Lee, and S. Yoon, “Towards a rigorous evaluation of time-series anomaly detection,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 36, no. 7, 2022, pp. 7194–7201.

[28] N. Tatbul, T. J. Lee, S. Zdonik, M. Alam, and J. Gottschlich, “Precision and recall for time series,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[29] M. Kim, R. Sumbaly, and S. Shah, “Root cause detection in a service-oriented architecture,” in *Proc. ACM SIGMETRICS Int. Conf. Measurement and Modeling of Computer Systems*, 2013, pp. 93–104.

[30] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, “MicroRCA: Root cause localization of performance issues in microservices,” in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020, pp. 1–9.

[31] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, “CloudRanger: Root cause identification for cloud native systems,” in *Proc. IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGRID)*, 2018, pp. 492–502.

[32] J. Lin, P. Chen, and Z. Zheng, “Microscope: Pinpoint performance issues with causal graphs in micro-service environments,” in *Proc. Int. Conf. Service-Oriented Computing (ICSOC)*, 2018, pp. 3–20.

[33] M. Ma, J. Xu, Y. Wang, P. Chen, Z. Zhang, and P. Wang, “AutoMAP: Diagnose your microservice-based web applications automatically,” in *Proc. The Web Conference (WWW)*, 2020, pp. 246–258.

[34] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, and X. Li, “MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments,” in *Proc. The Web Conference (WWW)*, 2021, pp. 3087–3098.

[35] A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu, “Root cause analysis of failures in microservices through causal discovery,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[36] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, and D. Pei, “Causal inference-based root cause analysis for online service systems with intervention recognition,” in *Proc. ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2022, pp. 3230–3240.

[37] R. Xin, P. Chen, and Z. Zhao, “CausalRCA: Causal inference based precise fine-grained root cause localization for microservice applications,” *Journal of Systems and Software*, vol. 203, p. 111724, 2023.

[38] H. Wang, Z. Wu, H. Jiang, Y. Huang, J. Wang, S. Kopru, and T. Xie, “Groot: An event-graph-based approach for root cause analysis in industrial settings,” in *Proc. IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2021, pp. 419–429.

[39] J. Soldani and A. Brogi, “Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey,” *ACM Computing Surveys*, vol. 55, no. 3, pp. 59:1–59:39, 2022.

[40] D. Kempe, J. Kleinberg, and É. Tardos, “Maximizing the spread of influence through a social network,” in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2003, pp. 137–146.

[41] G. W. Brier, “Verification of forecasts expressed in terms of probability,” *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

[42] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2005, pp. 625–632.

[43] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 70, 2017, pp. 1321–1330.

[44] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[45] R. P. Adams and D. J. C. MacKay, “Bayesian online changepoint detection,” arXiv:0710.3742, 2007.

[46] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank citation ranking: Bringing order to the web,” Stanford InfoLab, Tech. Rep., 1999.

[47] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758–759.

[48] C. W. J. Granger, “Investigating causal relations by econometric models and cross-spectral methods,” *Econometrica*, vol. 37, no. 3, pp. 424–438, 1969.

[49] T. Schreiber, “Measuring information transfer,” *Physical Review Letters*, vol. 85, no. 2, pp. 461–464, 2000.

[50] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[51] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*.1em plus 0.5em minus 0.4emWadsworth, 1984.

[52] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, “Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study,” *IEEE Trans. Software Engineering*, vol. 47, no. 2, pp. 243–260, 2021.

[53] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, “Chaos engineering,” *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[54] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

Appendix A: Engine Constants and Thresholds

Table A.1: Rule-layer thresholds (defaults, adjustable on the Incidents page).

Name	Default	Meaning
minHistorySamples	12	Samples a service needs before it is judged
zScoreAnomaly	3.5	Deviation counted as anomalous (noise-normalised)
zScoreHigh	5.0	Deviation at or above this is HIGH
zScoreCritical	8.0	Deviation at or above this is CRITICAL
minDeltaPercent	5	Smallest change worth a flag
absoluteHighPercent	70	Absolute CPU or memory share counted as high
absoluteCriticalPercent	85	Absolute share counted as critical
flatlineStdDev, flatlineDeltaPercent	0.001, 30	A series that goes flat after moving
criticalServicesForCritical	2	Critical services before the incident is CRITICAL
silenceSamples	3	Missed readings before a service counts as silent
monitoringGapShare	0.8	Share of silent services that means the monitoring itself is down
explainAwayGamma	1.0	Weight of what a sick callee explains (γ)
cascadeEdgeProbability	0.5	Starting propagation probability of an edge

Table A.2: Edge evidence confidence.

Evidence	Confidence $c(e)$
Seen in request logs, or oriented by a listening port	1.00
Oriented only by the ephemeral-port range	0.85
Observed, no direction evidence	0.70
Observed, direction unresolved	0.50
Declared in Compose, never seen carrying traffic	0.40

Appendix B: Fault Classes

Table B.1: Fault classes, how they are applied, and how many faults of each class the evaluation sets contain.

Class	Applied as	Real (62)	Synthetic (178)
crash	Process killed (SIGKILL); the container exits with a non-zero code	16	27
stop	Container stopped with docker stop (exit code 0)	16	24
brownout	Resource limit that starves the service without stopping it	14	26
memory	Memory pressure on the service (a leak, or a limit)	6	18
crash-loop	Repeated outages within minutes	4	16
pause	Container paused	4	15
flap	Repeated pause and release cycles	2	12
latency	Slow responses	–	25
partition	Network detached from the service	–	15

Appendix C: User Guide

C.1 Install and start

```
git clone https://github.com/Royson-salis-18/microservice-mapper.git
cd microservice-mapper
npm start        # installs what is missing, starts the server (:3001) and the dashboard
(:5173)
```

Open http://localhost:5173. Node.js 20.19 or 22.12 and later is required. npm run doctor checks the machine.

C.2 Add a host

Click “Add project” and fill in the target identifier, display name, the host’s public IPv4 address, the SSH user and the path to the key file. The key stays on your machine. npm run hosts lists the configured hosts and tests port 22; npm run hosts – set <target> <ip> changes an address.

C.3 Read an incident

Open *Incidents and cascade*. The headline names the most likely root cause with its confidence. The ranking shows what each service’s score is made of; “Why” shows the own, symptom and walk parts and the evidence; the cascade panel shows who is already affected and who is next; the next steps are commands to run on the host.

C.4 Compare the three engines

Open *Detection engines*. The three columns show, for the same readings, what each engine detected, its ranked suspects and what it predicts. The switch in the top bar (or the “Use” buttons) chooses which engine decides the incident.

C.5 Generate load and reproduce results

```
npm test                # the 339 tests
npm run eval            # record and score a campaign against the live mapper
npm run ml:compare      # detectors and root-cause methods on the recorded runs
npm run ml:hybrid       # fusion variants
npm run ml:predict      # early warning and cascade prediction
report/build.sh         # rebuild this report (needs pdflatex; Mermaid optional)
```

C.6 When nothing shows up

```
npm run check-host -- <public-ip> ~/path/to/key.pem 8080
npm run mapper-edges -- <project> <service-name-part>
```

The first says whether the host or the mapper is at fault; the second prints what the running mapper holds for a service.